

Link.Foundation.Links.Notation Class Library

1.1 ./Link.Foundation.Links.Notation/Binary/ArityRange.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.IO;
4 using System.Linq;
5 using System.Diagnostics.CodeAnalysis;
6 using System.Globalization;
7
8 namespace Link.Foundation.Links.Notation.Binary;
9
10 /// <summary>
11 /// How many references the links of a <see cref="Section"/> hold:
12 /// <c>Min..=Max</c>, where a null <see cref="Max"/> means no upper bound.
13 /// </summary>
14 /// <param name="Min">Fewest references in a link, at least 1.</param>
15 /// <param name="Max">Most references in a link, null for no limit.</param>
16 public readonly record struct ArityRange(ulong Min, ulong? Max)
17 {
18     /// <summary>The largest minimum arity a section shape can hold.</summary>
19     public const ulong MaxMin = ulong.MaxValue >> 4;
20
21     /// <summary>Links of exactly two references.</summary>
22     public static ArityRange Doublets { get; } = Exactly(2);
23
24     /// <summary>Links of exactly <paramref name="arity"/> references.</summary>
25     public static ArityRange Exactly(ulong arity) => new(arity, arity);
26
27     /// <summary>Links of at least <paramref name="min"/> references.</summary>
28     public static ArityRange AtLeast(ulong min) => new(min, null);
29
30     /// <summary>Links of <c>min..=max</c> references.</summary>
31     public static ArityRange Between(ulong min, ulong max) => new(min, max);
32
33     /// <summary>True when a link of <paramref name="length"/> references fits the
34     ↪ range.</summary>
35     public bool Contains(ulong length) => length >= Min && (Max is not { } max || length <= max);
36
37     /// <summary>True when every link has the same number of references, so links need no length
38     ↪ prefix.</summary>
39     public bool IsFixed => Max == Min;
40
41     /// <summary>0 for no maximum, otherwise <c>Max - Min</c>; only for variable
42     ↪ arities.</summary>
43     internal ulong Extra => Max is { } max ? max - Min : 0;
44
45     /// <summary>Null for a usable range, otherwise why it is not.</summary>
46     internal string? Problem()
47     {
48         if (Min == 0)
49         {
50             return "arity must be at least 1";
51         }
52         if (Min > MaxMin)
53         {
54             return $"arity {Min} is too large";
55         }
56         if (Max is { } max && max < Min)
57         {
58             return $"arity range {this} is empty";
59         }
60         return null;
61     }
62
63     /// <summary>The range a section header describes; <paramref name="extra"/> 0 means no
64     ↪ maximum.</summary>
65     internal static ArityRange FromShape(ulong min, ulong? extra)
66     {
67         ArityRange range;
68         if (extra is not { } added)
69         {
70             range = Exactly(min);
71         }
72         else if (added == 0)
73         {
74             range = AtLeast(min);
75         }
76         else
77         {
78             range = Between(min, min + added);
79         }
80     }
81 }
```

```

74         if (min > ulong.MaxValue - added)
75         {
76             throw BinaryNotationException.Malformed("arity range overflows 64 bits");
77         }
78         range = Between(min, min + added);
79     }
80     if (range.Problem() is { } problem)
81     {
82         throw BinaryNotationException.Malformed(problem);
83     }
84     return range;
85 }
86
87 /// <summary>Formats the range as <c>n</c>, <c>min..max</c> or <c>min..</c>.</summary>
88 public override string ToString() => Max switch
89 {
90     { } max when max == Min => max.ToString(CultureInfo.InvariantCulture),
91     { } max => $"{Min}..{max}",
92     null => $"{Min}..",
93 };
94
95 /// <summary>Parses <c>n</c>, <c>min..max</c> (inclusive) or <c>min..</c> (no
96 ↪ maximum).</summary>
97 /// <exception cref="FormatException">The text is not a usable arity range.</exception>
98 public static ArityRange Parse(string text) =>
99     TryParse(text, out var range, out var error) ? range : throw new FormatException(error);
100
101 ↪ /// <summary>Parses <c>n</c>, <c>min..max</c> (inclusive) or <c>min..</c> (no
102 ↪ maximum).</summary>
103 public static bool TryParse([NotNullWhen(true)] string? text, out ArityRange range) =>
104     TryParse(text, out range, out var ignored);
105
106 ↪ /// <summary>Parses an arity range, explaining in <paramref name="error"/> why it
107 ↪ failed.</summary>
108 public static bool TryParse([NotNullWhen(true)] string? text, out ArityRange range,
109 ↪ [NotNullWhen(false)] out string? error)
110 {
111     range = default;
112     error = $"invalid arity '{text}': expected n, min..max or min..";
113     if (text is null)
114     {
115         return false;
116     }
117     var separator = text.IndexOf("..", StringComparison.Ordinal);
118     if (separator < 0)
119     {
120         if (!TryParseNumber(text, out var arity))
121         {
122             return false;
123         }
124         range = Exactly(arity);
125     }
126     else
127     {
128         var maxText = text[(separator + 2)..];
129         if (!TryParseNumber(text[..separator], out var min))
130         {
131             return false;
132         }
133         if (maxText.Length == 0)
134         {
135             range = AtLeast(min);
136         }
137         else if (TryParseNumber(maxText, out var max))
138         {
139             range = Between(min, max);
140         }
141         else
142         {
143             return false;
144         }
145     }
146     error = range.Problem();
147     return error is null;
148 }
149
150 // Like Rust's u64::from_str: ASCII digits with an optional leading '+'.
151 private static bool TryParseNumber(string text, out ulong value)

```

```

148     {
149         var digits = text.StartsWith('+') ? text[1..] : text;
150         return ulong.TryParse(digits, NumberStyles.None, CultureInfo.InvariantCulture, out
↪ value);
151     }
152 }

```

1.2 ./Link.Foundation.Links.Notation/Binary/BinaryLinoCodec.cs

```

1 using System.Collections.Generic;
2 using LinoLink = Link.Foundation.Links.Notation.Link<string>;
3
4 namespace Link.Foundation.Links.Notation.Binary;
5
6 /// <summary>Convenience codec for complete LiNo packets.</summary>
7 public sealed record BinaryLinoCodec
8 {
9     public BinaryLinoCodec() { }
10
11     public BinaryLinoCodec(BinaryLinoOptions options) => Options = options;
12
13     /// <summary>Options used when writing.</summary>
14     public BinaryLinoOptions Options { get; init; }
15
16     /// <summary>Limits applied when reading and writing.</summary>
17     public DecodeLimits Limits { get; init; } = DecodeLimits.Default;
18
19     /// <summary>Encodes the supplied model without canonicalizing its groups.</summary>
20     public byte[] Encode(IReadOnlyList<LinoLink> document) =>
↪ LinoMapping.EncodeDocument(document, Options, Limits).ToBytes();
21
22     /// <summary>Decodes exactly one complete packet, rejecting trailing bytes.</summary>
23     public IReadOnlyList<LinoLink> Decode(byte[] bytes) =>
24         LinoMapping.DecodeDocument(LinksPacket.FromBytes(bytes, Limits), Limits);
25 }

```

1.3 ./Link.Foundation.Links.Notation/Binary/BinaryNotationException.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.IO;
4 using System.Linq;
5 namespace Link.Foundation.Links.Notation.Binary;
6
7 /// <summary>What went wrong while encoding, decoding or exchanging a LiNo message.</summary>
8 public enum BinaryErrorKind
9 {
10     /// <summary>The underlying transport failed.</summary>
11     Io,
12     /// <summary>The peer sent bytes that are not a valid message.</summary>
13     Malformed,
14     /// <summary>The message text is not valid LiNo.</summary>
15     InvalidLino,
16     /// <summary>The message exceeds one of the configured <see cref="DecodeLimits"/>.</summary>
17     LimitExceeded,
18     /// <summary>The document cannot be represented with the chosen options.</summary>
19     Unencodable,
20 }
21
22 /// <summary>Raised by the binary links notation codecs.</summary>
23 public sealed class BinaryNotationException : Exception
24 {
25     public BinaryNotationException()
26         : this(BinaryErrorKind.Malformed, "unknown protocol error")
27     {
28     }
29
30     public BinaryNotationException(string message)
31         : this(BinaryErrorKind.Malformed, message)
32     {
33     }
34
35     public BinaryNotationException(string message, Exception innerException)
36         : this(BinaryErrorKind.Io, message, innerException)
37     {
38     }
39
40     public BinaryNotationException(BinaryErrorKind kind, string detail, Exception?
↪ innerException = null)
41         : base($"{Describe(kind)}: {detail}", innerException)
42     {
43     }
44 }

```

```

43     Kind = kind;
44     Detail = detail;
45 }
46
47 /// <summary>The category of the failure.</summary>
48 public BinaryErrorKind Kind { get; }
49
50 /// <summary>The message without the category prefix.</summary>
51 public string Detail { get; }
52
53 internal static BinaryNotationException Malformed(string detail) =>
↪ new(BinaryErrorKind.Malformed, detail);
54
55 internal static BinaryNotationException Limit(string detail) =>
↪ new(BinaryErrorKind.LimitExceeded, detail);
56
57 internal static BinaryNotationException Unencodable(string detail) =>
↪ new(BinaryErrorKind.Unencodable, detail);
58
59 private static string Describe(BinaryErrorKind kind) => kind switch
60 {
61     BinaryErrorKind.Io => "I/O error",
62     BinaryErrorKind.Malformed => "malformed message",
63     BinaryErrorKind.InvalidLiNo => "invalid LiNo",
64     BinaryErrorKind.LimitExceeded => "limit exceeded",
65     BinaryErrorKind.Unencodable => "cannot encode",
66     => kind.ToString(),
67 };
68 }

```

1.4 ./Link.Foundation.Links.Notation/Binary/LinksPacket.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.IO;
4  using System.Linq;
5  using System.Buffers.Binary;
6
7  namespace Link.Foundation.Links.Notation.Binary;
8
9  /// <summary>One reference inside a <see cref="LinksPacket"/>.</summary>
10 /// <param name="IsExternal">True for an external value (a number or a code point).</param>
11 /// <param name="Value">The address inside the packet, or the external value.</param>
12 public readonly record struct PacketReference(bool IsExternal, ulong Value)
13 {
14     /// <summary>The null reference.</summary>
15     public static readonly PacketReference Null = Internal(LinksPacket.NullAddress);
16
17     /// <summary>An address inside the packet (0 null, 1..5 markers, then links).</summary>
18     public static PacketReference Internal(ulong address) => new(false, address);
19
20     /// <summary>An external value, e.g. a number or a Unicode code point.</summary>
21     public static PacketReference External(ulong value) => new(true, value);
22
23     public override string ToString() => IsExternal ? $"External({Value})" :
↪ $"Internal({Value})";
24 }
25
26 /// <summary>Safety limits applied while decoding untrusted input.</summary>
27 public sealed record DecodeLimits
28 {
29     /// <summary>Maximum number of links in a packet.</summary>
30     public ulong MaxLinks { get; init; } = 1UL << 22;
31
32     /// <summary>Maximum number of references in all links of a packet.</summary>
33     public ulong MaxReferences { get; init; } = 1UL << 24;
34
35     /// <summary>Maximum number of LiNo nodes a packet may expand to.</summary>
36     public long MaxNodes { get; init; } = 1L << 22;
37
38     /// <summary>Maximum total UTF-8 bytes in expanded references, including link ids.</summary>
39     public long MaxStringBytes { get; init; } = 64L << 20;
40
41     /// <summary>Maximum LiNo nesting depth.</summary>
42     public int MaxDepth { get; init; } = 64;
43
44     internal void Validate()
45     {
46         if (MaxNodes < 0 || MaxStringBytes < 0 || MaxDepth < 0)
47             throw new ArgumentOutOfRangeException(nameof(DecodeLimits), "Limits must be
↪ non-negative.");

```

```

48     }
49
50     /// <summary>The default limits.</summary>
51     public static DecodeLimits Default { get; } = new();
52
53     /// <summary>Limits for trusted input such as a store archive: only the address space bounds
54     ↪ the packet.</summary>
55     public static DecodeLimits Unlimited { get; } = new()
56     {
57         MaxLinks = ulong.MaxValue,
58         MaxReferences = ulong.MaxValue,
59         MaxNodes = long.MaxValue,
60         MaxStringBytes = long.MaxValue,
61         MaxDepth = int.MaxValue,
62     };
63
64     /// <summary>A run of links at consecutive addresses sharing an arity range and a reference
65     ↪ width.</summary>
66     public sealed class Section
67     {
68         /// <summary>Addresses skipped before the first link of the section.</summary>
69         public ulong Gap { get; set; }
70
71         /// <summary>The number of references each link may hold.</summary>
72         public ArityRange Arity { get; set; } = ArityRange.Doublets;
73
74         /// <summary>Bytes per reference: 1, 2, 4 or 8.</summary>
75         public byte Width { get; set; } = 1;
76
77         /// <summary>The links, in address order.</summary>
78         public List<PacketReference[]> Links { get; } = new();
79
80         internal void WriteHeader(List<byte> output)
81         {
82             var shape = LinksPacket.WidthCode(Width) | (Arity.Min << LinksPacket.ShapeMinArityShift);
83             if (Gap != 0)
84             {
85                 shape |= LinksPacket.ShapeHasGap;
86             }
87             if (!Arity.IsFixed)
88             {
89                 shape |= LinksPacket.ShapeVariableArity;
90             }
91             LinksPacket.WriteLeb128(output, shape);
92             if (Gap != 0)
93             {
94                 LinksPacket.WriteLeb128(output, Gap);
95             }
96             if (!Arity.IsFixed)
97             {
98                 LinksPacket.WriteLeb128(output, Arity.Extra);
99             }
100             LinksPacket.WriteLeb128(output, (ulong)Links.Count);
101         }
102
103         /// <summary>Reads a section header, returning the still empty section and its link
104         ↪ count.</summary>
105         internal static (Section Section, ulong Count) ReadHeader(PacketReader reader)
106         {
107             var shape = LinksPacket.ReadLeb128(reader);
108             var width = LinksPacket.WidthFromCode(shape & LinksPacket.ShapeWidthBits);
109             var gap = (shape & LinksPacket.ShapeHasGap) != 0 ? LinksPacket.ReadLeb128(reader) : 0;
110             ↪ ulong? extra = (shape & LinksPacket.ShapeVariableArity) != 0 ?
111             LinksPacket.ReadLeb128(reader) : null;
112             var arity = ArityRange.FromShape(shape >> LinksPacket.ShapeMinArityShift, extra);
113             var count = LinksPacket.ReadLeb128(reader);
114             return (new Section { Gap = gap, Arity = arity, Width = width }, count);
115         }
116
117         internal void Validate()
118         {
119             LinksPacket.WidthCode(Width);
120             if (Arity.Problem() is { } problem)
121             {
122                 throw BinaryNotationException.Unencodable(problem);
123             }
124             foreach (var link in Links)

```

```

122     {
123         if (!Arity.Contains((ulong)link.Length))
124         {
125             throw BinaryNotationException.Unencodable(
126                 $"a link of {link.Length} references in a section of arity {Arity}");
127         }
128     }
129 }
130
131
132 /// <summary>
133 /// Binary links notation: a self-delimiting packet of links.
134 /// </summary>
135 /// <remarks>
136 /// A packet stores links, each a tuple of one or more references, at
137 /// implicit consecutive addresses. It knows nothing about LiNo; the LiNo
138 /// protocol (<see cref="LinoMapping"/>) is one use of it.
139 /// <code>
140 /// byte 0      0x10 | flags      high nibble 1 = format version 1
141 ///                                     bit 0      external references (Hybrid encoding)
142 ///                                     bit 1      explicit layout
143 ///                                     bits 2-3    log2 of the width (compact layout only)
144 ///
145 /// compact layout (bit 1 clear): one section of doublets right after the markers
146 /// LEB128      N                  number of links, each `source target`
147 ///
148 /// explicit layout (bit 1 set):
149 /// LEB128      S                  number of sections, then S section headers:
150 /// LEB128      shape              bits 0-1    log2 of the reference width in bytes
151 ///                                     bit 2      a gap follows
152 ///                                     bit 3      variable arity (else every link
153 ///                                     holds exactly min arity references)
154 ///                                     bits 4+    min arity, at least 1
155 /// LEB128      gap                addresses skipped before the section (if bit 2)
156 /// LEB128      extra arity        0 = no maximum, else max - min (if bit 3)
157 /// LEB128      count              number of links in the section
158 ///
159 /// links, section by section; a link in a variable-arity section starts
160 /// with LEB128 (length - min arity); every reference is `width` bytes,
161 /// little-endian
162 /// </code>
163 /// Address 0 is null. The first section starts at <c>1 + gap</c> and every
164 /// other section at <c>previous end + gap</c>, so gaps leave holes. The
165 /// compact layout is exactly one section with gap 5 (the LiNo marker points
166 /// 1..5), arity 2 and the header width. Each section has its own width: the
167 /// narrowest of 1, 2, 4 and 8 bytes that holds every reference in it;
168 /// <see cref="Pack"/> chooses the sections. With external references
169 /// enabled the top bit of a reference marks it as external, exactly like
170 /// <c>Platform.Data.Hybrid<T></c>, which halves every internal range.
171 /// The byte layout is identical to the Rust <c>links notation::binary::LinksPacket</c>.
172 /// </remarks>
173 public sealed class LinksPacket
174 {
175     /// <summary>The high nibble of the header byte; text never starts with 0x10..0x1F.</summary>
176     public const byte BinaryVersion1 = 0x10;
177
178     /// <summary>Null link address.</summary>
179     public const ulong NullAddress = 0;
180     /// <summary>Marker 1: the unary one;  $2^k = (2^{(k-1)} 2^{(k-1)})$ .</summary>
181     public const ulong One = 1;
182     /// <summary>Marker 2: (Number unary) is a non-negative integer.</summary>
183     public const ulong Number = 2;
184     /// <summary>Marker 3: (String code points...) is a Unicode string.</summary>
185     public const ulong String = 3;
186     /// <summary>Marker 4: (List elements...) is a list of links.</summary>
187     public const ulong List = 4;
188     /// <summary>Marker 5: (Identified id values...) is a link with an id.</summary>
189     public const ulong Identified = 5;
190     /// <summary>Address of the first link after the marker points.</summary>
191     public const ulong FirstLinkAddress = 6;
192
193     private const byte FlagExternalReferences = 0b0001;
194     private const byte FlagExplicitLayout = 0b0010;
195     private const int WidthShift = 2;
196     private const byte WidthBits = 0b1100;
197
198     internal const ulong ShapeWidthBits = 0b0011;
199     internal const ulong ShapeHasGap = 0b0100;
200     internal const ulong ShapeVariableArity = 0b1000;

```

```

201     internal const int ShapeMinArityShift = 4;
202
203     /// <summary>The addresses the compact layout skips: the marker points.</summary>
204     private const ulong CompactGap = FirstLinkAddress - 1;
205
206     private static readonly byte[] WidthValues = { 1, 2, 4, 8 };
207
208     /// <summary>The reference widths, in bytes, that a packet may use.</summary>
209     public static IReadOnlyList<byte> Widths => WidthValues;
210
211     /// <summary>An empty packet.</summary>
212     public LinksPacket(bool externalReferences = false) => ExternalReferences =
↪     externalReferences;
213
214     /// <summary>Header bit 0: references may be external (Hybrid encoding).</summary>
215     public bool ExternalReferences { get; set; }
216
217     /// <summary>The links, section by section.</summary>
218     public List<Section> Sections { get; } = new();
219
220     /// <summary>Largest internal address that fits in <paramref name="width"/> bytes.</summary>
221     public static ulong InternalCapacity(byte width, bool externalReferences)
222     {
223         var bits = width * 8 - (externalReferences ? 1 : 0);
224         return bits >= 64 ? ulong.MaxValue : (1UL << bits) - 1;
225     }
226
227     /// <summary>Largest external value that fits in <paramref name="width"/> bytes.</summary>
228     public static ulong ExternalCapacity(byte width) => (1UL << (width * 8 - 1)) - 1;
229
230     /// <summary>The narrowest width able to hold the internal address.</summary>
231     public static byte AddressTier(ulong address, bool externalReferences)
232     {
233         foreach (var width in WidthValues)
234         {
235             if (InternalCapacity(width, externalReferences) >= address)
236             {
237                 return width;
238             }
239         }
240         ↪ throw BinaryNotationException.Unencodable($"address {address} exceeds the internal
range");
241     }
242
243     private static ulong WidthMask(byte width) => InternalCapacity(width, false);
244
245     /// <summary>Encodes an external value at <paramref name="width"/> the way Hybrid<T>
↪     does.</summary>
246     public static ulong EncodeExternal(ulong value, byte width) =>
247         value == 0 ? 1UL << (width * 8 - 1) : (OUL - value) & WidthMask(width);
248
249     /// <summary>Decodes a raw value; returns true and the value for externals.</summary>
250     public static bool TryDecodeExternal(ulong raw, byte width, out ulong value)
251     {
252         var externalZero = 1UL << (width * 8 - 1);
253         value = 0;
254         if (raw == externalZero)
255         {
256             return true;
257         }
258         if (raw > externalZero)
259         {
260             value = (OUL - raw) & WidthMask(width);
261             return true;
262         }
263         return false;
264     }
265
266     internal static ulong WidthCode(byte width)
267     {
268         var code = Array.IndexOf(WidthValues, width);
269         ↪ return code >= 0 ? (ulong)code : throw BinaryNotationException.Unencodable($"invalid
width {width}");
270     }
271
272     /// <summary>The width of a two-bit width code; every code names a width.</summary>
273     internal static byte WidthFromCode(ulong code) => WidthValues[code & 0b11];
274
275     /// <summary>The narrowest width able to hold <paramref name="reference"/>.</summary>

```

```

276 public static byte ReferenceWidth(PacketReference reference, bool externalReferences)
277 {
278     if (!reference.IsExternal)
279     {
280         return AddressTier(reference.Value, externalReferences);
281     }
282     if (!externalReferences)
283     {
284         throw BinaryNotationException.Unencodable("external reference in a packet without
↪ external references");
285     }
286     foreach (var width in WidthValues)
287     {
288         if (ExternalCapacity(width) >= reference.Value)
289         {
290             return width;
291         }
292     }
293     throw BinaryNotationException.Unencodable($"external value {reference.Value} exceeds 63
↪ bits");
294 }
295
296 /// <summary>
297 /// Lays out <paramref name="links"/> - addresses with their references, in
298 /// ascending address order - using the version 1 section planner.
299 /// </summary>
300 /// <remarks>
301 /// Holes between addresses start new sections. Without
302 /// <paramref name="packedWidths"/> every section uses the width of the
303 /// widest reference, so all references have the same size. With it each
304 /// section gets the narrowest width its links need and sections split
305 /// wherever that saves bytes; the result is never larger than the uniform one.
306 /// </remarks>
307 public static LinksPacket Pack(
308     bool externalReferences,
309     IReadOnlyList<ulong Address, PacketReference[] References> links,
310     bool packedWidths)
311 {
312     ArgumentNullException.ThrowIfNull(links);
313     var planner = new SectionPlanner(externalReferences, links);
314     var uniformLayout = planner.Plan(false);
315     var uniform = LayOut(externalReferences, links, uniformLayout);
316     if (!packedWidths)
317     {
318         return uniform;
319     }
320     var packedLayout = planner.Plan(true);
321     if (packedLayout.SequenceEqual(uniformLayout))
322     {
323         return uniform;
324     }
325     var packed = LayOut(externalReferences, links, packedLayout);
326     return packed.ToBytes().Length < uniform.ToBytes().Length ? packed : uniform;
327 }
328
329 /// <summary>Splits <paramref name="links"/> into sections of <c>(link count,
↪ width)</c>.</summary>
330 private static LinksPacket LayOut(
331     bool externalReferences,
332     IReadOnlyList<ulong Address, PacketReference[] References> links,
333     IReadOnlyList<int Count, byte Width> layout)
334 {
335     var packet = new LinksPacket(externalReferences);
336     var nextAddress = 1UL;
337     var first = 0;
338     foreach (var (count, width) in layout)
339     {
340         var start = links[first].Address;
341         var shortest = ulong.MaxValue;
342         var longest = 0UL;
343         var section = new Section { Gap = start - nextAddress, Width = width };
344         for (var index = first; index < first + count; index++)
345         {
346             var length = (ulong)links[index].References.Length;
347             shortest = Math.Min(shortest, length);
348             longest = Math.Max(longest, length);
349             section.Links.Add(links[index].References);
350         }

```

```

351         section.Arity = ArityRange.Between(shortest, longest);
352         packet.Sections.Add(section);
353         nextAddress = start + (ulong)count;
354         first += count;
355     }
356     return packet;
357 }
358
359 /// <summary>Every link with its address, in address order.</summary>
360 public IEnumerable<(ulong Address, PacketReference[] References)> Links()
361 {
362     var nextAddress = 1UL;
363     foreach (var section in Sections)
364     {
365         var start = SaturatingAdd(nextAddress, section.Gap);
366         nextAddress = SaturatingAdd(start, (ulong)section.Links.Count);
367         for (var index = 0; index < section.Links.Count; index++)
368         {
369             yield return (start + (ulong)index, section.Links[index]);
370         }
371     }
372 }
373
374 private static ulong SaturatingAdd(ulong left, ulong right) =>
375     left > ulong.MaxValue - right ? ulong.MaxValue : left + right;
376
377 /// <summary>The number of links in the packet.</summary>
378 public ulong LinkCount => Sections.Aggregate(0UL, (total, section) => total +
↵ (ulong)section.Links.Count);
379
380 private bool IsCompact => Sections.Count switch
381 {
382     0 => true,
383     1 => Sections[0].Gap == CompactGap && Sections[0].Arity == ArityRange.Doublets &&
↵ Sections[0].Links.Count > 0,
384     => false,
385 };
386
387 internal void Validate()
388 {
389     var nextAddress = 1UL;
390     foreach (var section in Sections)
391     {
392         section.Validate();
393         var count = (ulong)section.Links.Count;
394         if (section.Gap > ulong.MaxValue - nextAddress || count > ulong.MaxValue -
↵ nextAddress - section.Gap)
395         {
396             throw BinaryNotationException.Unencodable("addresses overflow 64 bits");
397         }
398         nextAddress += section.Gap + count;
399         foreach (var link in section.Links)
400         {
401             foreach (var reference in link)
402             {
403                 RawReference(reference, section.Width);
404             }
405         }
406     }
407 }
408
409 /// <summary>Serializes the packet.</summary>
410 public byte[] ToBytes()
411 {
412     Validate();
413     var header = BinaryVersion1;
414     if (ExternalReferences)
415     {
416         header |= FlagExternalReferences;
417     }
418     var output = new List<byte>();
419     if (IsCompact)
420     {
421         var width = Sections.Count > 0 ? Sections[0].Width : (byte)1;
422         output.Add((byte)(header | (WidthCode(width) << WidthShift)));
423         WriteLeb128(output, LinkCount);
424     }
425     else
426     {

```

```

427         output.Add((byte)(header | FlagExplicitLayout));
428         WriteLeb128(output, (ulong)Sections.Count);
429         foreach (var section in Sections)
430         {
431             section.WriteHeader(output);
432         }
433     }
434     foreach (var section in Sections)
435     {
436         foreach (var link in section.Links)
437         {
438             if (!section.Arity.IsFixed)
439             {
440                 WriteLeb128(output, (ulong)link.Length - section.Arity.Min);
441             }
442             foreach (var reference in link)
443             {
444                 WriteRaw(output, RawReference(reference, section.Width), section.Width);
445             }
446         }
447     }
448     return output.ToArray();
449 }
450
451 private ulong RawReference(PacketReference reference, byte width)
452 {
453     if (ReferenceWidth(reference, ExternalReferences) > width)
454     {
455         throw BinaryNotationException.Unencodable($"{reference} does not fit {width}
↪ byte(s)");
456     }
457     return reference.IsExternal ? EncodeExternal(reference.Value, width) : reference.Value;
458 }
459
460 /// <summary>Parses a complete packet; trailing bytes are an error.</summary>
461 public static LinksPacket FromBytes(byte[] bytes, DecodeLimits? limits = null)
462 {
463     var reader = PacketReader.FromBytes(bytes);
464     var packet = ReadFrom(reader, limits) ?? throw BinaryNotationException.Malformed("empty
↪ input");
465     if (!reader.AtEnd)
466     {
467         throw BinaryNotationException.Malformed("trailing bytes after the packet");
468     }
469     return packet;
470 }
471
472 /// <summary>Reads one packet; null on a clean end of stream.</summary>
473 public static LinksPacket? ReadFrom(PacketReader reader, DecodeLimits? limits = null)
474 {
475     ArgumentNullException.ThrowIfNull(reader);
476     limits ??= DecodeLimits.Default;
477     limits.Validate();
478     var first = reader.ReadByte();
479     if (first < 0)
480     {
481         return null;
482     }
483     var header = (byte)first;
484     if ((header & 0xF0) != BinaryVersion1)
485     {
486         throw BinaryNotationException.Malformed($"unsupported binary header byte
↪ 0x{header:X2}");
487     }
488     var packet = new LinksPacket((header & FlagExternalReferences) != 0);
489     var counts = new List<ulong>();
490     if ((header & FlagExplicitLayout) == 0)
491     {
492         var width = WidthFromCode((ulong)((header & WidthBits) >> WidthShift));
493         var count = ReadLeb128(reader);
494         if (count > ulong.MaxValue - FirstLinkAddress)
495         {
496             throw BinaryNotationException.Malformed("addresses overflow 64 bits");
497         }
498         if (count > 0)
499         {
500             packet.Sections.Add(new Section { Gap = CompactGap, Arity = ArityRange.Doublets,
↪ Width = width });

```

```

501         counts.Add(count);
502     }
503 }
504 else
505 {
506     if ((header & WidthBits) != 0)
507     {
508         throw BinaryNotationException.Malformed("the explicit layout keeps the header
↪ width bits clear");
509     }
510     var sectionCount = ReadLeb128(reader);
511     if (sectionCount > limits.MaxLinks)
512     {
513         throw TooManyLinks(limits);
514     }
515     var nextAddress = 1UL;
516     for (ulong index = 0; index < sectionCount; index++)
517     {
518         var (section, count) = Section.ReadHeader(reader);
519         if (section.Gap > ulong.MaxValue - nextAddress || count > ulong.MaxValue -
↪ nextAddress - section.Gap)
520         {
521             throw BinaryNotationException.Malformed("addresses overflow 64 bits");
522         }
523         nextAddress += section.Gap + count;
524         packet.Sections.Add(section);
525         counts.Add(count);
526     }
527 }
528 var total = 0UL;
529 foreach (var count in counts)
530 {
531     if (count > limits.MaxLinks - total)
532     {
533         throw TooManyLinks(limits);
534     }
535     total += count;
536 }
537 var referencesLeft = limits.MaxReferences;
538 for (var index = 0; index < packet.Sections.Count; index++)
539 {
540     var section = packet.Sections[index];
541     section.Links.Capacity = (int)Math.Min(counts[index], 4096);
542     for (ulong link = 0; link < counts[index]; link++)
543     {
544         var length = section.Arity.Min;
545         if (!section.Arity.IsFixed)
546         {
547             var extra = ReadLeb128(reader);
548             if (extra > ulong.MaxValue - length || !section.Arity.Contains(extra +
↪ length))
549             {
550                 throw BinaryNotationException.Malformed($"link length outside the
↪ section arity {section.Arity}");
551             }
552             length += extra;
553         }
554         if (length > referencesLeft)
555         {
556             throw BinaryNotationException.Limit($"references exceed the limit of
↪ {limits.MaxReferences}");
557         }
558         referencesLeft -= length;
559         var references = new List<PacketReference>((int)Math.Min(length, 4096));
560         for (ulong item = 0; item < length; item++)
561         {
562             var raw = ReadRaw(reader, section.Width);
563             references.Add(packet.ExternalReferences && TryDecodeExternal(raw,
↪ section.Width, out var value)
564                 ? PacketReference.External(value)
565                 : PacketReference.Internal(raw));
566         }
567         section.Links.Add(references.ToArray());
568     }
569 }
570 return packet;
571 }
572

```

```

573 private static BinaryNotationException TooManyLinks(DecodeLimits limits) =>
574     BinaryNotationException.Limit($"packet declares more than {limits.MaxLinks} links");
575
576 private static void WriteRaw(List<byte> output, ulong value, byte width)
577 {
578     for (var index = 0; index < width; index++)
579     {
580         output.Add((byte)(value >> (8 * index)));
581     }
582 }
583
584 private static ulong ReadRaw(PacketReader reader, byte width)
585 {
586     Span<byte> bytes = stackalloc byte[8];
587     bytes.Clear();
588     reader.ReadExactly(bytes[..width]);
589     return BinaryPrimitives.ReadUInt64LittleEndian(bytes);
590 }
591
592 /// <summary>Appends <paramref name="value"/> as unsigned LEB128.</summary>
593 public static void WriteLeb128(List<byte> output, ulong value)
594 {
595     ArgumentNullException.ThrowIfNull(output);
596     while (true)
597     {
598         var current = (byte)(value & 0x7F);
599         value >>= 7;
600         if (value == 0)
601         {
602             output.Add(current);
603             return;
604         }
605         output.Add((byte)(current | 0x80));
606     }
607 }
608
609 /// <summary>Reads an unsigned LEB128 value of at most 64 bits.</summary>
610 public static ulong ReadLeb128(PacketReader reader)
611 {
612     ArgumentNullException.ThrowIfNull(reader);
613     ulong value = 0;
614     for (var shift = 0; shift < 64; shift += 7)
615     {
616         var next = reader.ReadByte();
617         if (next < 0)
618         {
619             throw BinaryNotationException.Malformed("unexpected end of packet");
620         }
621         var payload = (ulong)(next & 0x7F);
622         if (shift == 63 && payload > 1)
623         {
624             throw BinaryNotationException.Malformed("LEB128 value overflows 64 bits");
625         }
626         value |= payload << shift;
627         if ((next & 0x80) == 0)
628         {
629             return value;
630         }
631     }
632     throw BinaryNotationException.Malformed("LEB128 value overflows 64 bits");
633 }
634 }

```

1.5 ./Link.Foundation.Links.Notation/Binary/LinoFormat.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.IO;
4 using System.Linq;
5 using Link.Foundation.Links.Notation;
6
7 using LinoLink = Link.Foundation.Links.Notation.Link<string>;
8
9 namespace Link.Foundation.Links.Notation.Binary;
10
11 /// <summary>
12 /// Canonical LiNo text for documents, shared by every protocol.
13 /// </summary>
14 /// <remarks>
15 /// A document is the list of top-level links of a message. A reference is a

```

```

16 /// <see cref="LinoLink"/> whose <c>Values</c> is null. links-notation keeps
17 /// one extra wrapper around single-reference groups (<c>a</c> parses as
18 /// <c>(a)</c>); <see cref="ParseDocument"/> removes it, exactly like the Rust
19 /// port, so documents decoded from the same bytes compare equal in both
20 /// ports. Formatting preserves the canonical model of parsed documents.
21 /// Manually constructed ids without values cannot round trip through text.
22 /// </remarks>
23 public static class LinoFormat
24 {
25
26     /// <summary>A reference: a link without values.</summary>
27     public static LinoLink Reference(string text) => new(text);
28
29     /// <summary>A link with an optional id and the given values.</summary>
30     public static LinoLink Link(string? id, IList<LinoLink> values) => new(id!, values);
31
32     /// <summary>Parses LiNo text into a canonical document. Blank input is the empty
33     ↪ document.</summary>
34     public static IReadOnlyList<LinoLink> ParseDocument(string text)
35     {
36         ArgumentNullException.ThrowIfNull(text);
37         if (text.All(character => character is ' ' or '\t' or '\r' or '\n'))
38         {
39             return Array.Empty<LinoLink>();
40         }
41         IList<LinoLink> parsed;
42         try
43         {
44             parsed = new Parser().Parse(text);
45         }
46         catch (Exception exception) when (exception is not OutOfMemoryException)
47         {
48             ↪ throw new BinaryNotationException(BinaryErrorKind.InvalidLino, exception.Message,
49             exception);
50         }
51         return parsed.Select(Canonical).ToList();
52     }
53
54     /// <summary>
55     /// Converts a link produced by the C# parser into the canonical model:
56     /// an unnamed group holding exactly one reference is that reference.
57     /// </summary>
58     public static LinoLink Canonical(LinoLink link)
59     {
60         if (link.Values is null)
61         {
62             return link;
63         }
64         if (link.Id is null && link.Values.Count == 1 && link.Values[0].Values is null)
65         {
66             return link.Values[0];
67         }
68         return Link(link.Id, link.Values.Select(Canonical).ToList());
69     }
70
71     /// <summary>
72     /// Formats a document as canonical LiNo text, one top-level link per line.
73     /// A top-level link without an id and with at least two values is written
74     /// without its outer parentheses: <c>() ((1 1))</c>.
75     /// </summary>
76     public static string FormatDocument(IEnumerable<LinoLink> document)
77     {
78         ArgumentNullException.ThrowIfNull(document);
79         return string.Join("\n", document.Select(FormatTopLevel));
80     }
81
82     private static string FormatTopLevel(LinoLink link) =>
83     ↪ link.Values is { Count: >= 2 } values && link.Id is null ? JoinValues(values) :
84     FormatLink(link);
85
86     /// <summary>Formats one link as it appears nested inside another link.</summary>
87     public static string FormatLink(LinoLink link)
88     {
89         if (link.Values is not { } values)
90         {
91             return FormatReference(link.Id ?? string.Empty);
92         }
93         if (link.Id is null)

```

```

91     {
92         // `(a)` parses back as the reference `a`, so a one-reference
93         // link needs a second pair of parentheses.
94         return values.Count == 1 && values[0].Values is null
95             ? $"({FormatReference(values[0].Id ?? string.Empty)})"
96             : $"({JoinValues(values)})";
97     }
98     return values.Count == 0
99         ? $"({FormatReference(link.Id)}:)"
100         : $"({FormatReference(link.Id)}: {JoinValues(values)})";
101 }
102
103 private static string JoinValues(IEnumerable<LinoLink> values) => string.Join(" ",
↪ values.Select(FormatLink));
104
105 /// <summary>Quotes a reference when it would not survive parsing as a bare word.</summary>
106 /// <remarks>
107 /// links-notation opens a quoted reference with a run of N equal quote
108 /// characters, closes it with the next run of exactly N, and reads 2N
109 /// quotes inside as N literal ones. The opening run is counted greedily,
110 /// so the chosen quote must differ from the first character; N is one
111 /// more than the longest run of that quote inside, and odd, because an
112 /// even delimiter run may be read as an empty reference.
113 /// </remarks>
114 public static string FormatReference(string reference) => ReferenceLiteral.Format(reference);
115 }

```

1.6 ./Link.Foundation.Links.Notation/Binary/LinoMapping.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.IO;
4  using System.Linq;
5  using System.Globalization;
6  using System.Text;
7
8  using LinoLink = Link.Foundation.Links.Notation.Link<string>;
9
10 namespace Link.Foundation.Links.Notation.Binary;
11
12 /// <summary>
13 /// Optional features of the binary LiNo protocol. Every feature is off by
14 /// default; each one can be switched on independently, like stacking a decorator.
15 /// </summary>
16 public readonly record struct BinaryLinoOptions
17 {
18     // Doublets are stored as null so that default(BinaryLinoOptions) means doublets.
19     private readonly ArityRange? arity;
20
21     /// <summary>
22     /// Send numbers and code points as Hybrid external references instead of
23     /// in-band unary links. Halves the internal address range of each width.
24     /// </summary>
25     public bool ExternalReferences { get; init; }
26
27     /// <summary>
28     /// The link lengths the encoder may use. The default, exactly 2, sends
29     /// only doublets; <c>2..3</c> adds triplets and <c>1..</c> any length.
30     /// The range must include 2.
31     /// </summary>
32     public ArityRange Arity
33     {
34         get => arity ?? ArityRange.Doublets;
35         init => arity = value == ArityRange.Doublets ? null : value;
36     }
37
38     /// <summary>
39     /// Give every section of the packet the narrowest width its links need
40     /// instead of one width for the whole packet.
41     /// </summary>
42     public bool PackedWidths { get; init; }
43
44     /// <summary>Enables or disables external references.</summary>
45     public BinaryLinoOptions WithExternalReferences(bool enabled = true) => this with {
↪ ExternalReferences = enabled };
46
47     /// <summary>Sets the link lengths the encoder may use.</summary>
48     public BinaryLinoOptions WithArity(ArityRange arity) => this with { Arity = arity };
49
50     /// <summary>Enables or disables packed widths.</summary>

```

```

51     public BinaryLinoOptions WithPackedWidths(bool enabled = true) => this with { PackedWidths =
↳ enabled };
52
53     /// <summary>The options a peer most likely used to write <paramref name="packet"/>, so a
↳ reply can be written in the same style.</summary>
54     public static BinaryLinoOptions OfPacket(LinksPacket packet)
55     {
56         ArgumentNullException.ThrowIfNull(packet);
57         ulong shortest = 2;
58         ulong longest = 2;
59         foreach (var ( , link) in packet.Links())
60         {
61             shortest = Math.Min(shortest, (ulong)link.Length);
62             longest = Math.Max(longest, (ulong)link.Length);
63         }
64         return new BinaryLinoOptions
65         {
66             ExternalReferences = packet.ExternalReferences,
67             Arity = ArityRange.Between(shortest, longest),
68             PackedWidths = packet.Sections.Select(section =>
↳ section.Width).Distinct().Skip(1).Any(),
69         };
70     }
71
72     /// <summary>Lists every option, for test and error messages.</summary>
73     public override string ToString() =>
74         $"BinaryLinoOptions {{ ExternalReferences = {ExternalReferences}, Arity = {Arity},
↳ PackedWidths = {PackedWidths} }}";
75 }
76
77 /// <summary>
78 /// Lossless mapping between LiNo documents and <see cref="LinksPacket"/>s,
79 /// byte-for-byte compatible with the Rust port.
80 /// </summary>
81 /// <remarks>
82 /// <list type="bullet">
83 /// <item><c>()</c> is the null link 0.</item>
84 /// <item>A numeric reference n is <c>(Number unary(n))</c>, or an external reference when
↳ enabled.</item>
85 /// <item>Any other reference is <c>(String code points...)</c>; code points are unary numbers
↳ or externals.</item>
86 /// <item>A link without an id and with exactly two values is a plain doublet.</item>
87 /// <item>A link without an id and any other number of values is a list.</item>
88 /// <item>A link with an id is <c>(Identified id values...)</c>.</item>
89 /// <item>The document is a list of its top-level links, stored last (the root).</item>
90 /// </list>
91 /// A list or typed value with any number of values other than two is a
92 /// single link of that many references when <see cref="BinaryLinoOptions.Arity"/>
93 /// allows it (<c>[marker, elements...]</c> for typed values, the bare elements
94 /// for lists), and otherwise the doublet <c>(marker chain)</c>, where chain is
95 /// the nil-terminated cons list <c>(e1 (e2 (... (en 0)))</c>. Identical
96 /// sub-links are emitted once and shared. Links are numbered so that every
97 /// link only refers to earlier ones: doublets of plain doublets first, then
98 /// the rest in creation order.
99 /// </remarks>
100 public static class LinoMapping
101 {
102     /// <summary>Converts a document into a packet.</summary>
103     public static LinksPacket EncodeDocument(IReadOnlyList<LinoLink> document, BinaryLinoOptions
↳ options = default, DecodeLimits? limits = null)
104     {
105         ArgumentNullException.ThrowIfNull(document);
106         if (options.Arity.Problem() is { } problem)
107         {
108             throw BinaryNotationException.Unencodable(problem);
109         }
110         if (!options.Arity.Contains(2))
111         {
112             throw BinaryNotationException.Unencodable($"arity {options.Arity} does not include
↳ doublets (2)");
113         }
114         // Check the model iteratively before entering the recursive encoder.
115         limits ??= DecodeLimits.Default;
116         limits.Validate();
117         var pending = new Stack<(LinoLink Link, int Depth)>(document.Select(link => (link, 0)));
118         var budget = limits.MaxNodes;
119         var stringsLeft = limits.MaxStringBytes;
120         while (pending.TryPop(out var item))

```

```

121     {
122         if (item.Depth >= limits.MaxDepth)
123         {
124             throw BinaryNotationException.Unencodable($"nesting deeper than
↪ {limits.MaxDepth}");
125         }
126         if (--budget < 0)
127         {
128             throw BinaryNotationException.Unencodable("too many LiNo nodes");
129         }
130         if (item.Link.Id is { } text)
131         {
132             if (item.Link.Values is not null && --budget < 0)
133                 throw BinaryNotationException.Unencodable("too many LiNo nodes");
134             var bytes = Encoding.UTF8.GetByteCount(text);
135             if (bytes > stringsLeft) throw BinaryNotationException.Unencodable("too many
↪ string bytes");
136             stringsLeft -= bytes;
137         }
138         if (item.Link.Values is { } children)
139         {
140             foreach (var child in children)
141             {
142                 pending.Push((child, item.Depth + 1));
143             }
144         }
145     }
146     var encoder = new Encoder(options);
147     if (document.Count > 0)
148     {
149         encoder.List(document.Select(encoder.Encode).ToList());
150     }
151     var packet = encoder.Finish();
152     if ((ulong)packet.Sections.Count > limits.MaxLinks || packet.LinkCount > limits.MaxLinks)
153         throw BinaryNotationException.Unencodable("too many packet links");
154     var referencesLeft = limits.MaxReferences;
155     foreach (var ( , link) in packet.Links())
156     {
157         if ((ulong)link.Length > referencesLeft) throw
↪ BinaryNotationException.Unencodable("too many packet references");
158         referencesLeft -= (ulong)link.Length;
159     }
160     return packet;
161 }
162
163 /// <summary>Converts a packet back into a document.</summary>
164 public static IReadOnlyList<LinoLink> DecodeDocument(LinksPacket packet, DecodeLimits?
↪ limits = null)
165 {
166     ArgumentNullException.ThrowIfNull(packet);
167     limits ??= DecodeLimits.Default;
168     limits.Validate();
169     var decoder = new Decoder(packet, limits);
170     if (decoder.LinkCount == 0)
171     {
172         return Array.Empty<LinoLink>();
173     }
174     var root = decoder.View(PacketReference.Internal(LinksPacket.FirstLinkAddress +
↪ (ulong)decoder.LinkCount - 1));
175     IReadOnlyList<PacketReference> items = root switch
176     {
177         { Kind: ViewKind.Link, Items: [{ IsExternal: false, Value: LinksPacket.List }, var
↪ chain] } => decoder.Chain(chain),
178         { Kind: ViewKind.Link } when !StartsWithMarker(root.Items!) => root.Items!,
179         => throw BinaryNotationException.Malformed("the root link is not a list"),
180     };
181     var budget = limits.MaxNodes;
182     return items.Select(item => decoder.Decode(item, 0, ref budget)).ToList();
183 }
184
185 /// <summary>Parses a canonical unsigned decimal number (no sign, no leading
↪ zeros).</summary>
186 public static bool TryParseCanonicalNumber(string text, out ulong value)
187 {
188     value = 0;
189     var canonical = !string.IsNullOrEmpty(text)
190         && text.All(character => character is >= '0' and <= '9')
191         && (text == "0" || text[0] != '0');

```

```

192         return canonical && ulong.TryParse(text, NumberStyles.None,
↪ CultureInfo.InvariantCulture, out value);
193     }
194
195     private static bool IsMarker(PacketReference reference) =>
196         !reference.IsExternal && reference.Value is >= LinksPacket.One and <
↪ LinksPacket.FirstLinkAddress;
197
198     private static bool StartsWithMarker(IReadOnlyList<PacketReference> items) => items.Count >
↪ 0 && IsMarker(items[0]);
199
200     private enum NodeKind
201     {
202         Internal,
203         External,
204         /// <summary>A doublet of plain doublets, markers and externals.</summary>
205         Doublet,
206         /// <summary>Any other link; numbered after every doublet.</summary>
207         Tuple,
208     }
209
210     private readonly record struct Node(NodeKind Kind, ulong Value)
211     {
212         public static Node Internal(ulong address) => new(NodeKind.Internal, address);
213
214         public static Node External(ulong value) => new(NodeKind.External, value);
215     }
216
217     private sealed class NodeListComparer : IEqualityComparer<List<Node>>
218     {
219         public static readonly NodeListComparer Instance = new();
220
221         public bool Equals(List<Node>? x, List<Node>? y) =>
222             ReferenceEquals(x, y) || (x is not null && y is not null && x.SequenceEqual(y));
223
224         public int GetHashCode(List<Node> obj)
225         {
226             var hash = new HashCode();
227             foreach (var node in obj)
228             {
229                 hash.Add(node);
230             }
231             return hash.ToHashCode();
232         }
233     }
234
235     private sealed class Encoder
236     {
237         private readonly BinaryLinoOptions options;
238         private readonly List<List<Node>> doublets = new();
239         private readonly List<List<Node>> tuples = new();
240         private readonly Dictionary<List<Node>, Node> created = new(NodeListComparer.Instance);
241         private readonly List<Node> powers = new() { Node.Internal(LinksPacket.One) };
242
243         public Encoder(BinaryLinoOptions options) => options = options;
244
245         private Node Link(List<Node> items)
246         {
247             if (created.TryGetValue(items, out var existing))
248             {
249                 return existing;
250             }
251             Node node;
252             if (items.Count == 2 && items.All(item => item.Kind != NodeKind.Tuple))
253             {
254                 doublets.Add(items);
255                 node = new Node(NodeKind.Doublet, (ulong) doublets.Count - 1);
256             }
257             else
258             {
259                 tuples.Add(items);
260                 node = new Node(NodeKind.Tuple, (ulong) tuples.Count - 1);
261             }
262             created[items] = node;
263             return node;
264         }
265
266         private Node Pair(Node first, Node second) => Link(new List<Node> { first, second });
267
268         private Node Chain(IReadOnlyList<Node> items)

```

```

269     {
270         var tail = Node.Internal(LinksPacket.NullAddress);
271         for (var index = items.Count - 1; index >= 0; index--)
272         {
273             tail = Pair(items[index], tail);
274         }
275         return tail;
276     }
277
278     // One link of `items` when the arity allows it, except that two items
279     // are always a doublet.
280     private bool FitsOneLink(int items) => items != 2 &&
↪ options.Arity.Contains((ulong)items);
281
282     private Node Typed(ulong marker, List<Node> elements)
283     {
284         if (FitsOneLink(elements.Count + 1))
285         {
286             elements.Insert(0, Node.Internal(marker));
287             return Link(elements);
288         }
289         return Pair(Node.Internal(marker), Chain(elements));
290     }
291
292     public Node List(List<Node> elements) => elements.Count switch
293     {
294         0 => Node.Internal(LinksPacket.NullAddress),
295         2 => Pair(elements[0], elements[1]),
296         when FitsOneLink(elements.Count) => Link(elements),
297         => Typed(LinksPacket.List, elements),
298     };
299
300     private Node Power(int exponent)
301     {
302         while (powers.Count <= exponent)
303         {
304             var previous = powers[^1];
305             powers.Add(Pair(previous, previous));
306         }
307         return powers[exponent];
308     }
309
310     private Node Unary(ulong value)
311     {
312         var powers = new List<Node>();
313         for (var bit = 63; bit >= 0; bit--)
314         {
315             if ((value & (1UL << bit)) != 0)
316             {
317                 powers.Add(Power(bit));
318             }
319         }
320         if (powers.Count == 0)
321         {
322             return Node.Internal(LinksPacket.NullAddress);
323         }
324         var sum = powers[^1];
325         for (var index = powers.Count - 2; index >= 0; index--)
326         {
327             sum = Pair(powers[index], sum);
328         }
329         return sum;
330     }
331
332     private bool IsExternal(ulong value) => options.ExternalReferences && value <=
↪ LinksPacket.ExternalCapacity(8);
333
334     private Node Scalar(ulong value) => IsExternal(value) ? Node.External(value) :
↪ Unary(value);
335
336     private Node Reference(string text)
337     {
338         var remaining = text.AsSpan();
339         while (!remaining.IsEmpty)
340         {
341             if (Rune.DecodeFromUtf16(remaining, out var rune, out var consumed) !=
↪ System.Buffers.OperationStatus.Done)
342             {

```

```

343         throw BinaryNotationException.Unencodable("reference contains invalid
↳ UTF-16");
344     }
345     remaining = remaining[consumed..];
346 }
347 if (TryParseCanonicalNumber(text, out var value))
348 {
349     return IsExternal(value) ? Node.External(value) :
↳ Pair(Node.Internal(LinksPacket.Number), Unary(value));
350 }
351 var codePoints = text.EnumerateRunes().Select(rune =>
↳ Scalar((ulong)rune.Value)).ToList();
352 return Typed(LinksPacket.String, codePoints);
353 }
354
355 public Node Encode(LinoLink link)
356 {
357     if (link.Values is not { } values)
358     {
359         return Reference(link.Id ?? string.Empty);
360     }
361     if (link.Id is { } id)
362     {
363         var elements = new List<Node> { Reference(id) };
364         elements.AddRange(values.Select(Encode));
365         return Typed(LinksPacket.Identified, elements);
366     }
367     return List(values.Select(Encode).ToList());
368 }
369
370 public LinksPacket Finish()
371 {
372     var doubletCount = (ulong) doublets.Count;
373     PacketReference Resolve(Node node) => node.Kind switch
374     {
375         NodeKind.Internal => PacketReference.Internal(node.Value),
376         NodeKind.External => PacketReference.External(node.Value),
377         NodeKind.Doublet => PacketReference.Internal(LinksPacket.FirstLinkAddress +
↳ node.Value),
378         => PacketReference.Internal(LinksPacket.FirstLinkAddress + doubletCount +
↳ node.Value),
379     };
380     var links = doublets
381         .Concat( tuples)
382         .Select((items, index) => (LinksPacket.FirstLinkAddress + (ulong)index,
↳ items.Select(Resolve).ToArray()))
383         .ToList();
384     return LinksPacket.Pack( options.ExternalReferences, links, options.PackedWidths);
385 }
386
387 private enum ViewKind
388 {
389     Null,
390     Marker,
391     External,
392     Link,
393 }
394
395 private readonly record struct View(ViewKind Kind, ulong Value = 0, PacketReference[]? Items
↳ = null);
396
397 private sealed class Decoder
398 {
399     private readonly DecodeLimits limits;
400     private long stringBytesLeft;
401     // links[i] is the link at address FirstLinkAddress + i.
402     private readonly List<PacketReference[]> links = new();
403     // unary[i] is the number link i denotes, if it is a unary number.
404     private readonly List<ulong?> unary = new();
405
406     public Decoder(LinksPacket packet, DecodeLimits limits)
407     {
408         limits = limits;
409         stringBytesLeft = limits.MaxStringBytes;
410         if ((ulong)packet.Sections.Count > limits.MaxLinks || packet.LinkCount >
↳ limits.MaxLinks)
411         {
412

```

```

413         throw BinaryNotationException.Limit($"packet declares more than
↪ {limits.MaxLinks} links");
414     }
415     var referencesLeft = limits.MaxReferences;
416     try
417     {
418         packet.Validate();
419     }
420     catch (BinaryNotationException exception)
421     {
422         throw BinaryNotationException.Malformed(exception.Detail);
423     }
424     foreach (var (address, link) in packet.Links())
425     {
426         if ((ulong)link.Length > referencesLeft)
427         {
428             throw BinaryNotationException.Limit($"references exceed the limit of
↪ {limits.MaxReferences}");
429         }
430         referencesLeft -= (ulong)link.Length;
431         var expected = LinksPacket.FirstLinkAddress + (ulong) links.Count;
432         if (address != expected)
433         {
434             throw BinaryNotationException.Malformed(
435                 $"a LiNo packet stores its links contiguously from address
↪ {LinksPacket.FirstLinkAddress}, " +
436                 $"found link {address} where {expected} belongs");
437         }
438         foreach (var reference in link)
439         {
440             if (!reference.IsExternal && reference.Value >= address)
441             {
442                 throw BinaryNotationException.Malformed(
443                     $"link {address} refers to {reference}, which is not an earlier
↪ link");
444             }
445         }
446         // Links only refer backwards, so one forward pass evaluates every
447         // unary number without recursion.
448         ulong? value = null;
449         if (link.Length == 2
450             && UnaryValue(link[0]) is { } source
451             && UnaryValue(link[1]) is { } target
452             && source <= ulong.MaxValue - target)
453         {
454             value = source + target;
455         }
456         unary.Add(value);
457         links.Add(link);
458     }
459 }
460
461 public int LinkCount => links.Count;
462
463 private ulong? UnaryValue(PacketReference reference) => reference switch
464 {
465     { IsExternal: true } => null,
466     { Value: LinksPacket.NullAddress } => 0,
467     { Value: LinksPacket.One } => 1,
468     { Value: >= LinksPacket.FirstLinkAddress } => unary[(int)(reference.Value -
↪ LinksPacket.FirstLinkAddress)],
469     => null,
470 };
471
472 public View View(PacketReference reference) => reference switch
473 {
474     { IsExternal: true } => new View(ViewKind.External, reference.Value),
475     { Value: LinksPacket.NullAddress } => new View(ViewKind.Null),
476     { Value: < LinksPacket.FirstLinkAddress } => new View(ViewKind.Marker,
↪ reference.Value),
477     => new View(ViewKind.Link, Items: links[(int)(reference.Value -
↪ LinksPacket.FirstLinkAddress)]),
478 };
479
480 private ulong Number(PacketReference reference) =>
481     reference.IsExternal
482         ? reference.Value

```

```

483         : UnaryValue(reference) ?? throw BinaryNotationException.Malformed("expected a
↪ unary number");
484
485     /// <summary>The elements of the cons list <c>(e1 (e2 (... (en 0))))</c>.</summary>
486     public List<PacketReference> Chain(PacketReference tail)
487     {
488         var elements = new List<PacketReference>();
489         while (true)
490         {
491             var view = View(tail);
492             switch (view)
493             {
494                 case { Kind: ViewKind.Null }:
495                     return elements;
496                 case { Kind: ViewKind.Link, Items: [var head, var next] }:
497                     if (elements.Count >= limits.MaxNodes)
498                     {
499                         throw BinaryNotationException.Limit("chain too long");
500                     }
501                     elements.Add(head);
502                     tail = next;
503                     break;
504                 default:
505                     throw BinaryNotationException.Malformed("broken element chain");
506             }
507         }
508     }
509
510     private LinoLink Typed(ulong marker, IReadOnlyList<PacketReference> elements, int depth,
↪ ref long budget)
511     {
512         switch (marker)
513         {
514             case LinksPacket.Number:
515                 if (elements.Count != 1)
516                 {
517                     throw BinaryNotationException.Malformed("a number needs exactly one
↪ value");
518                 }
519                 return
↪ ReferenceText(Number(elements[0]).ToString(CultureInfo.InvariantCulture));
520             case LinksPacket.String:
521                 var text = new StringBuilder((int)Math.Min(elements.Count,
↪ stringBytesLeft));
522                 long byteCount = 0;
523                 foreach (var element in elements)
524                 {
525                     var codePoint = Number(element);
526                     if (codePoint > int.MaxValue || !Rune.IsValid((int)codePoint))
527                     {
528                         throw BinaryNotationException.Malformed($"invalid code point
↪ {codePoint}");
529                     }
530                     var rune = new Rune((int)codePoint);
531                     if (rune.Utf8SequenceLength > stringBytesLeft - byteCount)
532                     {
533                         throw BinaryNotationException.Limit("too many expanded string
↪ bytes");
534                     }
535                     byteCount += rune.Utf8SequenceLength;
536                     text.Append(rune.ToString());
537                 }
538                 return ReferenceText(text.ToString());
539             case LinksPacket.List:
540                 return List(elements, 0, depth, ref budget);
541             case LinksPacket.Identified:
542                 if (elements.Count == 0)
543                 {
544                     throw BinaryNotationException.Malformed("an identified link needs an
↪ id");
545                 }
546                 var id = Decode(elements[0], depth, ref budget);
547                 if (id.Values is not null)
548                 {
549                     throw BinaryNotationException.Malformed("a link id must be a reference");
550                 }
551                 var values = List(elements, 1, depth, ref budget).Values!;
552                 return LinoFormat.Link(id.Id ?? string.Empty, values);

```

```

553         default:
554             throw BinaryNotationException.Malformed($"marker {marker} cannot start a
↪ typed value");
555     }
556 }
557
558 private LinoLink List(IReadOnlyList<PacketReference> elements, int skip, int depth, ref
↪ long budget)
559 {
560     var values = new List<LinoLink>(Math.Max(0, elements.Count - skip));
561     for (var index = skip; index < elements.Count; index++)
562     {
563         values.Add(Decode(elements[index], depth + 1, ref budget));
564     }
565     return LinoFormat.Link(null, values);
566 }
567
568 private LinoLink ReferenceText(string text)
569 {
570     var bytes = Encoding.UTF8.GetByteCount(text);
571     if (bytes > stringBytesLeft)
572     {
573         throw BinaryNotationException.Limit("too many expanded string bytes");
574     }
575     stringBytesLeft -= bytes;
576     return LinoFormat.Reference(text);
577 }
578
579 public LinoLink Decode(PacketReference reference, int depth, ref long budget)
580 {
581     if (depth >= limits.MaxDepth)
582     {
583         throw BinaryNotationException.Limit($"nesting deeper than { limits.MaxDepth}");
584     }
585     if (budget <= 0)
586     {
587         throw BinaryNotationException.Limit("too many LiNo nodes");
588     }
589     budget--;
590     var view = View(reference);
591     switch (view)
592     {
593         case { Kind: ViewKind.Null }:
594             return LinoFormat.Link(null, new List<LinoLink>());
595         case { Kind: ViewKind.External }:
596             return ReferenceText(view.Value.ToString(CultureInfo.InvariantCulture));
597         case { Kind: ViewKind.Marker }:
598             throw BinaryNotationException.Malformed($"marker {view.Value} used as a
↪ value");
599         case { Items: [{ IsExternal: false, Value: LinksPacket.Number }, var value] }:
600             return Typed(LinksPacket.Number, new[] { value }, depth, ref budget);
601         case { Items: [var marker, var chain] } when IsMarker(marker):
602             return Typed(marker.Value, Chain(chain), depth, ref budget);
603         default:
604             var items = view.Items!;
605             return StartsWithMarker(items)
606                 ? Typed(items[0].Value, items[1..], depth, ref budget)
607                 : List(items, 0, depth, ref budget);
608     }
609 }
610 }
611 }

```

1.7 ./Link.Foundation.Links.Notation/Binary/PacketReader.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.IO;
4 using System.Linq;
5 namespace Link.Foundation.Links.Notation.Binary;
6
7 /// <summary>
8 /// A buffered reader for consecutive binary packets. Reuse it across reads
9 /// so bytes buffered from the following packet remain available.
10 /// </summary>
11 public sealed class PacketReader
12 {
13     private readonly Stream stream;
14     private readonly byte[] buffer;
15     private int position;

```

```

16     private int length;
17     -
18     public PacketReader(Stream stream, int bufferSize = 8192)
19     {
20         ArgumentNullException.ThrowIfNull(stream);
21         ArgumentOutOfRangeException.ThrowIfNegativeOrZero(bufferSize);
22         stream = stream;
23         buffer = new byte[bufferSize];
24     }
25
26     /// <summary>Wraps a byte array, e.g. a whole encoded message.</summary>
27     public static PacketReader FromBytes(byte[] bytes) => new(new MemoryStream(bytes, writable:
↪ false));
28
29     /// <summary>The next byte without consuming it, or -1 at the end of the stream.</summary>
30     public int PeekByte() => Fill() ? buffer[ position] : -1;
31
32     /// <summary>The next byte, or -1 at the end of the stream.</summary>
33     public int ReadByte() => Fill() ? buffer[ position++] : -1;
34
35     /// <summary>True when every byte of the stream has been consumed.</summary>
36     public bool AtEnd => !Fill();
37
38     /// <summary>Fills <paramref name="destination"/>; a short stream is a malformed
↪ message.</summary>
39     public void ReadExactly(Span<byte> destination)
40     {
41         while (!destination.IsEmpty)
42         {
43             if (!Fill())
44             {
45                 throw BinaryNotationException.Malformed("unexpected end of packet");
46             }
47             var count = Math.Min(destination.Length, length - position);
48             buffer.AsSpan( position, count).CopyTo(destination);
49             position += count;
50             destination = destination[count..];
51         }
52     }
53
54     private bool Fill()
55     {
56         if ( position < length)
57         {
58             return true;
59         }
60         try
61         {
62             length = stream.Read( buffer, 0, buffer.Length);
63         }
64         catch (IOException exception)
65         {
66             throw new BinaryNotationException(BinaryErrorKind.Io, exception.Message, exception);
67         }
68         catch (ObjectDisposedException exception)
69         {
70             throw new BinaryNotationException(BinaryErrorKind.Io, exception.Message, exception);
71         }
72         position = 0;
73         return length > 0;
74     }
75 }

```

1.8 ./Link.Foundation.Links.Notation/Binary/SectionPlanner.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.IO;
4  using System.Linq;
5  namespace Link.Foundation.Links.Notation.Binary;
6
7  /// <summary>Splits links into <see cref="Section"/>s with a linear dynamic program.</summary>
8  /// <remarks>
9  /// After link k, <c>costs[s]</c> is the fewest estimated bytes for links
10 /// 0..=k with link k in a section of state s. A link either continues the
11 /// section of the previous link (same state, no hole between them and, for a
12 /// fixed arity, the same length) or opens a new section after the cheapest
13 /// previous state, paying for a section header. The tie-breaking matches the
14 /// Rust <c>SectionPlanner</c>, so both ports produce identical bytes.
15 /// </remarks>

```

```

16 internal sealed class SectionPlanner
17 {
18     /// <summary>Estimated bytes of a section header (shape and count), used to weigh a
    ↪ split.</summary>
19     private const ulong SectionHeaderEstimate = 2;
20     /// <summary>Estimated extra bytes of a variable-arity section: its extra arity field and
    ↪ each length prefix.</summary>
21     private const ulong VariableArityEstimate = 1;
22     private const ulong Unreachable = ulong.MaxValue / 4;
23     /// <summary>Planner states: a width code (0..4) times fixed (0) or variable (1)
    ↪ arity.</summary>
24     private static readonly int States = LinksPacket.Widths.Count * 2;
25     -
26     private readonly IReadOnlyList<ulong Address, PacketReference[] References> links;
27     private readonly byte[] needs;
28
29     public SectionPlanner(bool externalReferences, IReadOnlyList<ulong Address,
    ↪ PacketReference[] References> links)
30     {
31         links = links;
32         needs = new byte[links.Count];
33         var previousAddress = 0UL;
34         for (var index = 0; index < links.Count; index++)
35         {
36             var (address, references) = links[index];
37             if (address <= previousAddress)
38             {
39                 throw BinaryNotationException.Unencodable(
40                     $"link addresses must ascend from 1, got {address} after {previousAddress}");
41             }
42             if (address == ulong.MaxValue)
43             {
44                 throw BinaryNotationException.Unencodable("addresses overflow 64 bits");
45             }
46             if (references.Length == 0)
47             {
48                 throw BinaryNotationException.Unencodable($"link {address} has no references");
49             }
50             previousAddress = address;
51             byte need = 1;
52             foreach (var reference in references)
53             {
54                 need = Math.Max(need, LinksPacket.ReferenceWidth(reference, externalReferences));
55             }
56             needs[index] = need;
57         }
58     }
59
60     private static int FirstCheapest(ulong[] costs)
61     {
62         var best = 0;
63         for (var state = 1; state < costs.Length; state++)
64         {
65             if (costs[state] < costs[best])
66             {
67                 best = state;
68             }
69         }
70         return best;
71     }
72
73     /// <summary>
74     /// The sections as <c>(link count, width)</c> pairs; with
75     /// <paramref name="packedWidths"/> every width may be used, otherwise only
76     /// the widest one needed.
77     /// </summary>
78     public List<(int Count, byte Width)> Plan(bool packedWidths)
79     {
80         var sections = new List<(int Count, byte Width)>();
81         if (links.Count == 0)
82         {
83             return sections;
84         }
85         var widest = needs.Max();
86         var opensSection = new byte[links.Count];
87         var previousBest = new byte[links.Count];
88         var costs = Enumerable.Repeat(Unreachable, States).ToArray();
89         for (var index = 0; index < links.Count; index++)

```

```

90     {
91         var (address, link) = links[index];
92         var best = FirstCheapest(costs);
93         previousBest[index] = (byte)best;
94         var cheapestBefore = index == 0 ? 0 : costs[best];
95         var continues = index > 0 && links[index - 1].Address + 1 == address;
96         var sameLength = index > 0 && links[index - 1].References.Length == link.Length;
97         var next = Enumerable.Repeat(Unreachable, States).ToArray();
98         for (var state = 0; state < States; state++)
99         {
100             var width = LinksPacket.Widths[state / 2];
101             var variable = state % 2 == 1;
102             if (!(packedWidths || width == widest) || width < needs[index])
103             {
104                 continue;
105             }
106             var variableEstimate = variable ? VariableArityEstimate : 0;
107             var body = (ulong)link.Length * width + variableEstimate;
108             var openingCost = cheapestBefore + SectionHeaderEstimate + variableEstimate;
109             var continuingCost = continues && (variable || sameLength) ? costs[state] :
↪ Unreachable;
110             if (continuingCost <= openingCost)
111             {
112                 next[state] = continuingCost + body;
113             }
114             else
115             {
116                 next[state] = openingCost + body;
117                 opensSection[index] |= (byte)(1 << state);
118             }
119         }
120         costs = next;
121     }
122     var current = FirstCheapest(costs);
123     var end = links.Count;
124     for (var index = links.Count - 1; index >= 0; index--)
125     {
126         if ((opensSection[index] & (1 << current)) != 0)
127         {
128             sections.Add((end - index, LinksPacket.Widths[current / 2]));
129             end = index;
130             current = previousBest[index];
131         }
132     }
133     sections.Reverse();
134     return sections;
135 }
136 }

```

1.9 ./Link.Foundation.Links.Notation/Comments.cs

```

1 namespace Link.Foundation.Links.Notation
2 {
3     /// <summary>
4     /// Comments, and the rule that decides where one starts.
5     /// </summary>
6     /// <remarks>
7     /// <c>#</c> starts a comment when it opens a token: at the start of the document or
8     /// after a space, a tab or a line break. A <c>#</c> written inside a token
9     /// (<c>issue#1047</c>) or inside a delimited reference (<c>"# not a comment"</c>) is
10    /// an ordinary character.
11    /// </remarks>
12    public static class Comments
13    {
14        /// <summary>
15        /// The character that starts a comment.
16        /// </summary>
17        public const char Comment = '#';
18
19        /// <summary>
20        /// What a delimited reference may start after.
21        /// </summary>
22        private const string BeforeReference = " \t\n\r(";
23
24        /// <summary>
25        /// What a comment may start after.
26        /// </summary>
27        private const string BeforeComment = " \t\n\r";
28
29        /// <summary>

```

```

30     /// Blanks every comment of a document.
31     /// </summary>
32     /// <param name="document">The document to read.</param>
33     /// <returns>
34     /// The same document with the characters of every comment replaced by spaces.
35     /// </returns>
36     /// <remarks>
37     /// Every character of a comment is replaced rather than removed, so every later
38     /// character keeps the position it had in the document the caller wrote, and
39     /// anything the parser reports about a position still points at what the reader
40     /// can see.
41     /// </remarks>
42     public static string StripComments(string document)
43     {
44         char[]? blanked = null;
45         int position = 0;
46
47         while (position < document.Length)
48         {
49             char character = document[position];
50
51             if (Quotes.IsQuote(character) && Follows(document, position, BeforeReference))
52             {
53                 int end = Quotes.ReferenceEnd(document, position);
54                 position = end < 0 ? position + 1 : end;
55                 continue;
56             }
57
58             if (character == Comment && Follows(document, position, BeforeComment))
59             {
60                 blanked ??= document.ToCharArray();
61                 while (position < document.Length &&
62                     document[position] != '\n' &&
63                     document[position] != '\r')
64                 {
65                     blanked[position] = ' ';
66                     position++;
67                 }
68                 continue;
69             }
70
71             position++;
72         }
73
74         return blanked == null ? document : new string(blanked);
75     }
76
77     /// <summary>
78     /// Reports whether the character before the position is one of the allowed ones,
79     /// the start of the document counting as allowed.
80     /// </summary>
81     private static bool Follows(string document, int position, string allowed) =>
82         position == 0 || allowed.IndexOf(document[position - 1]) >= 0;
83
84 }
85

```

1.10 ./Link.Foundation.Links.Notation/FormatConfig.cs

```

1  namespace Link.Foundation.Links.Notation
2  {
3      /// <summary>
4      /// FormatConfig is an alias for FormatOptions to maintain API consistency
5      /// with the Python implementation.
6      ///
7      /// C# uses 'FormatOptions' internally, but provides 'FormatConfig'
8      /// as an alias to match Python's naming convention.
9      /// </summary>
10     public class FormatConfig : FormatOptions
11     {
12         // FormatConfig inherits all functionality from FormatOptions
13         // No additional implementation needed - this is purely for naming consistency
14     }
15 }

```

1.11 ./Link.Foundation.Links.Notation/FormatOptions.cs

```

1  using System;
2
3  namespace Link.Foundation.Links.Notation
4  {

```

```

5  /// <summary>
6  /// FormatOptions for Lino notation formatting.
7  /// Provides configuration options for controlling how Link objects are formatted.
8  /// </summary>
9  public class FormatOptions
10 {
11     /// <summary>
12     /// If true, omit parentheses where safe (default: false)
13     /// </summary>
14     public bool LessParentheses { get; set; } = false;
15
16     /// <summary>
17     /// Maximum line length before auto-indenting (default: 80)
18     /// </summary>
19     public int MaxLineLength { get; set; } = 80;
20
21     /// <summary>
22     /// If true, indent lines exceeding MaxLineLength (default: false)
23     /// </summary>
24     public bool IndentLongLines { get; set; } = false;
25
26     /// <summary>
27     /// Maximum number of references before auto-indenting (default: null = unlimited)
28     /// </summary>
29     public int? MaxInlineRefs { get; set; } = null;
30
31     /// <summary>
32     /// If true, group consecutive links with same ID (default: false)
33     /// </summary>
34     public bool GroupConsecutive { get; set; } = false;
35
36     /// <summary>
37     /// String to use for indentation (default: "  " = two spaces)
38     /// </summary>
39     public string IndentString { get; set; } = "  ";
40
41     /// <summary>
42     /// If true, prefer inline format when under thresholds (default: true)
43     /// </summary>
44     public bool PreferInline { get; set; } = true;
45
46     /// <summary>
47     /// Check if line should be indented based on length
48     /// </summary>
49     /// <param name="line">The line to check</param>
50     /// <returns>True if line should be indented based on length threshold</returns>
51     public bool ShouldIndentByLength(string line)
52     {
53         if (!IndentLongLines)
54         {
55             return false;
56         }
57         // Count printable unicode characters
58         return line.Length > MaxLineLength;
59     }
60
61     /// <summary>
62     /// Check if link should be indented based on reference count
63     /// </summary>
64     /// <param name="refCount">Number of references in the link</param>
65     /// <returns>True if link should be indented based on reference count threshold</returns>
66     public bool ShouldIndentByRefCount(int refCount)
67     {
68         if (MaxInlineRefs == null)
69         {
70             return false;
71         }
72         return refCount > MaxInlineRefs.Value;
73     }
74 }
75 }

```

1.12 ./Link.Foundation.Links.Notation/FurthestFailureTracer.cs

```

1  using Pegasus.Common;
2  using Pegasus.Common.Tracing;
3
4  namespace Link.Foundation.Links.Notation
5  {
6      /// <summary>

```

```

7     /// Remembers the furthest position a parse reached.
8     /// </summary>
9     /// <remarks>
10    /// A parsing expression grammar backtracks, so the cursor the generated parser raises
11    /// its error with is the start of the document rather than the place the document
12    /// stopped making sense. The furthest position any rule reached is that place, and the
13    /// tracer is the only hook the generated parser offers to observe it.
14    /// </remarks>
15    internal sealed class FurthestFailureTracer : ITracer
16    {
17        /// <summary>Offset of the furthest position the parse reached.</summary>
18        public int Furthest { get; private set; }
19
20        public void TraceRuleEnter(string ruleName, Cursor cursor) => Reach(cursor.Location);
21
22        public void TraceRuleExit<T>(string ruleName, Cursor cursor, IParseResult<T> result)
23        {
24            if (result != null) Reach(result.EndCursor.Location);
25        }
26
27        public void TraceCacheHit<T>(string ruleName, Cursor cursor, CacheKey key,
↪ IParseResult<T> result)
28        {
29            if (result != null) Reach(result.EndCursor.Location);
30        }
31
32        public void TraceCacheMiss(string ruleName, Cursor cursor, CacheKey key)
33        {
34        }
35
36        public void TraceInfo(string ruleName, Cursor cursor, string info)
37        {
38        }
39
40        private void Reach(int location)
41        {
42            if (location > Furthest) Furthest = location;
43        }
44    }
45 }

```

1.13 ./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs

```

1 using System.Collections.Generic;
2 using System.Runtime.CompilerServices;
3
4 namespace Link.Foundation.Links.Notation
5 {
6     /// <summary>
7     /// Provides extension methods for collections of <see cref="LinksGroup{TLinkAddress}"/>
↪ instances.
8     /// </summary>
9     public static class ILinksGroupListExtensions
10    {
11        /// <summary>
12        /// Converts a collection of links groups to a flat list of links by processing each
↪ group hierarchically.
13        /// </summary>
14        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This
↪ can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
15        /// <param name="groups">The collection of links groups to convert.</param>
16        /// <returns>A flat list of links representing all the links from the input
↪ groups.</returns>
17        [MethodImpl(MethodImplOptions.AggressiveInlining)]
18        public static List<Link<TLinkAddress>> ToLinksList<TLinkAddress>(this
↪ IList<LinksGroup<TLinkAddress>> groups)
19        {
20            var list = new List<Link<TLinkAddress>>();
21            for (var i = 0; i < groups.Count; i++)
22            {
23                CollectLinksWithIndentedIdSyntaxSupport(list, groups[i], null);
24            }
25            return list;
26        }
27
28        private static void
↪ CollectLinksWithIndentedIdSyntaxSupport<TLinkAddress>(List<Link<TLinkAddress>> list,
↪ LinksGroup<TLinkAddress> group, Link<TLinkAddress>? parentDependency)
29        {

```

```

30     var link = group.Link;
31     var groups = group.Groups;
32
33     if (groups != null && groups.Count > 0)
34     {
35         bool isIndentedIdSyntax = link.Id != null &&
36             (link.Values == null || link.Values.Count == 0);
37
38         if (isIndentedIdSyntax && !parentDependency.HasValue)
39         {
40             var childValues = new List<Link<TLinkAddress>>>();
41             for (int i = 0; i < groups.Count; i++)
42             {
43                 var transformedLink = TransformIndentedIdLink(groups[i]);
44                 childValues.Add(transformedLink);
45             }
46
47             var linkWithChildren = new Link<TLinkAddress>(link.Id, childValues);
48             list.Add(linkWithChildren);
49         }
50         else
51         {
52             var transformedGroups = new List<LinksGroup<TLinkAddress>>>();
53             for (int i = 0; i < groups.Count; i++)
54             {
55                 var childGroup = groups[i];
56                 var childLink = childGroup.Link;
57                 var childGroups = childGroup.Groups;
58
59                 if (childLink.Id != null &&
60                     (childLink.Values == null || childLink.Values.Count == 0) &&
61                     childGroups != null && childGroups.Count > 0)
62                 {
63                     var transformedLink = TransformIndentedIdLink(childGroup);
64                     transformedGroups.Add(new LinksGroup<TLinkAddress>(transformedLink));
65                 }
66                 else
67                 {
68                     transformedGroups.Add(childGroup);
69                 }
70             }
71
72             var currentDependency = parentDependency.HasValue ?
↪ parentDependency.Value.Combine(link) : link;
73             list.Add(currentDependency);
74
75             for (int i = 0; i < transformedGroups.Count; i++)
76             {
77                 CollectLinksWithIndentedIdSyntaxSupport(list, transformedGroups[i],
↪ currentDependency);
78             }
79         }
80     }
81     else
82     {
83         var currentLink = parentDependency.HasValue ?
↪ parentDependency.Value.Combine(link) : link;
84         list.Add(currentLink);
85     }
86 }
87
88 private static Link<TLinkAddress>
↪ TransformIndentedIdLink<TLinkAddress>(LinksGroup<TLinkAddress> group)
89 {
90     var link = group.Link;
91     var groups = group.Groups;
92
93     if (groups != null && groups.Count > 0 &&
94         link.Id != null &&
95         (link.Values == null || link.Values.Count == 0))
96     {
97         var childValues = new List<Link<TLinkAddress>>>();
98         for (int i = 0; i < groups.Count; i++)
99         {
100             var transformedChild = TransformIndentedIdLink(groups[i]);
101             childValues.Add(transformedChild);
102         }
103         return new Link<TLinkAddress>(link.Id, childValues);
104     }

```

```

105         else if (groups != null && groups.Count > 0)
106         {
107             var values = new List<Link<TLinkAddress>>();
108             if (link.Values != null)
109             {
110                 values.AddRange(link.Values);
111             }
112             for (int i = 0; i < groups.Count; i++)
113             {
114                 values.Add(TransformIndentedIdLink(groups[i]));
115             }
116             return new Link<TLinkAddress>(link.Id, values);
117         }
118         else if (!group.IsParenthesized && link.Id == null && link.Values != null &&
↪ link.Values.Count == 1)
119         {
120             return link.Values[0];
121         }
122         else
123         {
124             return link;
125         }
126     }
127 }
128 }

```

1.14 ./Link.Foundation.Links.Notation/IListExtensions.cs

```

1 using Platform.Collections;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Runtime.CompilerServices;
6
7 namespace Link.Foundation.Links.Notation
8 {
9     /// <summary>
10     /// Provides extension methods for formatting collections of <see
↪ cref="Link{TLinkAddress}" /> instances.
11     /// </summary>
12     public static class IListExtensions
13     {
14         /// <summary>
15         /// Formats a collection of links as a multi-line string with each link on a separate
↪ line.
16         /// </summary>
17         /// <typeparam name="TLinkAddress">The type used for link
↪ addresses/identifiers.</typeparam>
18         /// <param name="links">The collection of links to format.</param>
19         /// <returns>A multi-line string representation of the links.</returns>
20         [MethodImpl(MethodImplOptions.AggressiveInlining)]
21         public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links) =>
↪ string.Join(Environment.NewLine, links.Select(l => l.ToString()));
22
23         /// <summary>
24         /// Formats a collection of links as a multi-line string with optional parentheses
↪ trimming for cleaner output.
25         /// </summary>
26         /// <typeparam name="TLinkAddress">The type used for link
↪ addresses/identifiers.</typeparam>
27         /// <param name="links">The collection of links to format.</param>
28         /// <param name="lessParentheses">True to remove outer parentheses from each link for
↪ cleaner formatting; false to use standard formatting.</param>
29         /// <returns>A multi-line string representation of the links.</returns>
30         [MethodImpl(MethodImplOptions.AggressiveInlining)]
31         public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links, bool
↪ lessParentheses)
32         {
33             if (lessParentheses == false)
34             {
35                 return links.Format();
36             }
37             else
38             {
39                 return string.Join(Environment.NewLine, links.Select(l =>
↪ l.ToString().TrimSingle('(').TrimSingle(')')));
40             }
41         }
42     }

```

```

43     /// <summary>
44     /// Formats a collection of links using FormatOptions configuration.
45     /// </summary>
46     /// <typeparam name="TLinkAddress">The type used for link
↪ addresses/identifiers.</typeparam>
47     /// <param name="links">The collection of links to format.</param>
48     /// <param name="options">The FormatOptions to use for formatting.</param>
49     /// <returns>A multi-line string representation of the links.</returns>
50     [MethodImpl(MethodImplOptions.AggressiveInlining)]
51     public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links,
↪ FormatOptions options)
52     {
53         if (links == null || links.Count == 0)
54         {
55             return string.Empty;
56         }
57
58         // Apply consecutive link grouping if enabled
59         var linksToFormat = options.GroupConsecutive ? GroupConsecutiveLinks(links) : links;
60
61         // Format each link with options
62         var formattedLinks = linksToFormat.Select(l => l.FormatWithOptions(options));
63         return string.Join(Environment.NewLine, formattedLinks);
64     }
65
66     /// <summary>
67     /// Group consecutive links with the same ID.
68     /// </summary>
69     private static List<Link<TLinkAddress>>
↪ GroupConsecutiveLinks<TLinkAddress>(IList<Link<TLinkAddress>> links)
70     {
71         if (links == null || links.Count == 0)
72         {
73             return new List<Link<TLinkAddress>>();
74         }
75
76         var grouped = new List<Link<TLinkAddress>>();
77         int i = 0;
78
79         while (i < links.Count)
80         {
81             var current = links[i];
82
83             // Look ahead for consecutive links with same ID
84             if (current.Id != null && current.Values != null && current.Values.Count > 0)
85             {
86                 // Collect all values with same ID
87                 var sameIdValues = new List<Link<TLinkAddress>>(current.Values);
88                 int j = i + 1;
89
90                 while (j < links.Count)
91                 {
92                     var nextLink = links[j];
93                     if (EqualityComparer<TLinkAddress>.Default.Equals(nextLink.Id,
↪ current.Id) &&
94                         nextLink.Values != null && nextLink.Values.Count > 0)
95                     {
96                         sameIdValues.AddRange(nextLink.Values);
97                         j++;
98                     }
99                     else
100                     {
101                         break;
102                     }
103                 }
104
105                 // If we found consecutive links, create grouped link
106                 if (j > i + 1)
107                 {
108                     var groupedLink = new Link<TLinkAddress>(current.Id, sameIdValues);
109                     grouped.Add(groupedLink);
110                     i = j;
111                     continue;
112                 }
113             }
114
115             grouped.Add(current);
116             i++;
117         }

```

```

118         return grouped;
119     }
120 }
121 }
122 }

```

1.15 ./Link.Foundation.Links.Notation/Link.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Runtime.CompilerServices;
4  using System.Text;
5  using Platform.Collections;
6  using Platform.Collections.Lists;
7
8  namespace Link.Foundation.Links.Notation
9  {
10     /// <summary>
11     /// Represents a link in the Links Notation with an optional identifier and nested values.
12     /// Links can represent simple references, complex nested structures, or relationships
13     ↪ between references to links.
14     /// </summary>
15     /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
16     ↪ any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
17     public struct Link<TLinkAddress> : IEquatable<Link<TLinkAddress>>
18     {
19         private static readonly EqualityComparer<TLinkAddress> EqualityComparerInstance =
20         ↪ EqualityComparer<TLinkAddress>.Default;
21
22         /// <summary>
23         /// Gets the identifier or address of this link. Can be null for anonymous links.
24         /// </summary>
25         public readonly TLinkAddress? Id;
26
27         /// <summary>
28         /// Gets the collection of nested link values. Can be null or empty for simple reference
29     ↪ links.
30         /// </summary>
31         public readonly IList<Link<TLinkAddress>>? Values;
32
33         /// <summary>
34         /// Initializes a new link with the specified identifier and values.
35         /// </summary>
36         /// <param name="id">The link identifier or address.</param>
37         /// <param name="values">The nested link values.</param>
38         [MethodImpl(MethodImplOptions.AggressiveInlining)]
39         public Link(TLinkAddress? id, IList<Link<TLinkAddress>>? values) => (Id, Values) = (id,
40     ↪ values);
41
42         /// <summary>
43         /// Initializes a new anonymous link with the specified values.
44         /// </summary>
45         /// <param name="values">The nested link values.</param>
46         [MethodImpl(MethodImplOptions.AggressiveInlining)]
47         public Link(IList<Link<TLinkAddress>> values) : this(default!, values) { }
48
49         /// <summary>
50         /// Initializes a new anonymous link with the specified array of values.
51         /// </summary>
52         /// <param name="values">The nested link values as a parameter array.</param>
53         [MethodImpl(MethodImplOptions.AggressiveInlining)]
54         public Link(params Link<TLinkAddress>[] values) : this(default!, values) { }
55
56         /// <summary>
57         /// Initializes a new simple reference link with the specified identifier.
58         /// </summary>
59         /// <param name="id">The link identifier or address.</param>
60         [MethodImpl(MethodImplOptions.AggressiveInlining)]
61         public Link(TLinkAddress id) : this(id, default!) { }
62
63         /// <summary>
64         /// Returns the string representation of this link in Links Notation format.
65         /// </summary>
66         /// <returns>A string representation of the link with proper escaping and
67     ↪ formatting.</returns>
68         [MethodImpl(MethodImplOptions.AggressiveInlining)]
69         public override string ToString() => Id == null ?
70             $"({GetValuesString()})" :
71             (Values == null || Values.Count == 0) ?
72             $"({EscapeReference(Id.ToString())})" :

```

```

67         $"({EscapeReference(Id.ToString())}): {GetValuesString()})";
68
69     /// <summary>
70     /// Gets the string representation of the link's values.
71     /// </summary>
72     /// <returns>A space-separated string of the values, or empty string if no values
↪ exist.</returns>
73     [MethodImpl(MethodImplOptions.AggressiveInlining)]
74     public string GetValuesString()
75     {
76         if (Values == null || Values.Count == 0)
77         {
78             return "";
79         }
80         var sb = new StringBuilder();
81         for (int i = 0; i < Values.Count; i++)
82         {
83             if (i > 0)
84             {
85                 sb.Append(' ');
86             }
87             sb.Append(GetValueString(Values[i]));
88         }
89         return sb.ToString();
90     }
91
92     /// <summary>
93     /// Simplifies the link structure by unwrapping single-value containers and recursively
↪ simplifying nested values.
94     /// </summary>
95     /// <returns>A simplified version of this link.</returns>
96     [MethodImpl(MethodImplOptions.AggressiveInlining)]
97     public Link<TLinkAddress> Simplify()
98     {
99         if (Values == null || Values.Count == 0)
100         {
101             return this;
102         }
103         else if (Values.Count == 1)
104         {
105             return Values[0];
106         }
107         else
108         {
109             var newValues = new Link<TLinkAddress>[Values.Count];
110             for (int i = 0; i < Values.Count; i++)
111             {
112                 newValues[i] = Values[i].Simplify();
113             }
114             return new Link<TLinkAddress>(Id, newValues);
115         }
116     }
117
118     /// <summary>
119     /// Combines this link with another link to create a new link containing both as values.
120     /// </summary>
121     /// <param name="other">The other link to combine with.</param>
122     /// <returns>A new link containing both links as values.</returns>
123     [MethodImpl(MethodImplOptions.AggressiveInlining)]
124     public Link<TLinkAddress> Combine(Link<TLinkAddress> other) => new
↪ Link<TLinkAddress>(this, other);
125
126     /// <summary>
127     /// Gets the string representation of a link value, choosing the appropriate format.
128     /// </summary>
129     /// <param name="value">The link value to convert to string.</param>
130     /// <returns>The string representation of the link value.</returns>
131     [MethodImpl(MethodImplOptions.AggressiveInlining)]
132     public static string GetValueString(Link<TLinkAddress> value) =>
↪ value.ToLinkOrIdString();
133
134     /// <summary>
135     /// Escapes a reference string for safe use in Links Notation format by adding quotes if
↪ necessary.
136     /// </summary>
137     /// <param name="reference">The reference string to escape.</param>
138     /// <returns>The escaped reference string with appropriate quoting.</returns>
139     public static string EscapeReference(string? reference)

```

```

140     {
141         if (reference == null)
142         {
143             return "";
144         }
145         return ReferenceLiteral.Format(reference);
146     }
147
148     /// <summary>
149     /// Converts the link to its string representation, choosing between simple reference
150     format or full link format.
151     /// </summary>
152     /// <returns>A string representation optimized for the link's content.</returns>
153     [MethodImpl(MethodImplOptions.AggressiveInlining)]
154     public string ToLinkOrIdString() => (Values == null || Values.Count == 0) ? (Id == null
155     ? "" : EscapeReference(Id?.ToString())) : ToString();
156
157     /// <summary>
158     /// Implicitly converts a value to a simple reference link.
159     /// </summary>
160     /// <param name="value">The value to convert.</param>
161     [MethodImpl(MethodImplOptions.AggressiveInlining)]
162     public static implicit operator Link<TLinkAddress>(TLinkAddress value) => new
163     Link<TLinkAddress>(value);
164
165     /// <summary>
166     /// Implicitly converts a tuple of identifier and values to a link.
167     /// </summary>
168     /// <param name="value">The tuple containing identifier and values.</param>
169     [MethodImpl(MethodImplOptions.AggressiveInlining)]
170     public static implicit operator Link<TLinkAddress>((TLinkAddress,
171     IList<Link<TLinkAddress>>) value) => new(value.Item1, value.Item2);
172
173     /// <summary>
174     /// Implicitly converts a source-target pair to an anonymous link.
175     /// </summary>
176     /// <param name="value">The tuple containing source and target links.</param>
177     [MethodImpl(MethodImplOptions.AggressiveInlining)]
178     public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source,
179     Link<TLinkAddress> target) value) => new(value.source, value.target);
180
181     /// <summary>
182     /// Implicitly converts a tuple of identifier, source, and target to a link.
183     /// </summary>
184     /// <param name="value">The tuple containing id, source and target.</param>
185     [MethodImpl(MethodImplOptions.AggressiveInlining)]
186     public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id,
187     Link<TLinkAddress> source, Link<TLinkAddress> target) value) => new(value.id.Id, new[] {
188     value.source, value.target });
189
190     /// <summary>
191     /// Implicitly converts a source-linker-target triplet to an anonymous link.
192     /// </summary>
193     /// <param name="value">The tuple containing source, linker, and target links.</param>
194     [MethodImpl(MethodImplOptions.AggressiveInlining)]
195     public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source,
196     Link<TLinkAddress> linker, Link<TLinkAddress> target) value) => new(value.source,
197     value.linker, value.target);
198
199     /// <summary>
200     /// Implicitly converts a tuple of identifier, source, linker, and target to a link.
201     /// </summary>
202     /// <param name="value">The tuple containing id, source, linker, and target.</param>
203     [MethodImpl(MethodImplOptions.AggressiveInlining)]
204     public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id,
205     Link<TLinkAddress> source, Link<TLinkAddress> linker, Link<TLinkAddress> target) value) =>
206     new(value.id.Id, new[] { value.source, value.linker, value.target });
207
208     /// <summary>
209     /// Determines whether the specified object is equal to this link.
210     /// </summary>
211     /// <param name="obj">The object to compare with this link.</param>
212     /// <returns>True if the specified object is equal to this link; otherwise,
213     false.</returns>
214     [MethodImpl(MethodImplOptions.AggressiveInlining)]
215     public override bool Equals(object? obj) => obj is Link<TLinkAddress> link &&
216     Equals(link);

```

```

204
205     /// <summary>
206     /// Returns the hash code for this link.
207     /// </summary>
208     /// <returns>A hash code for this link.</returns>
209     [MethodImpl(MethodImplOptions.AggressiveInlining)]
210     public override int GetHashCode() => (Id, (Values ??
↪ Array.Empty<Link<TLinkAddress>>()).GenerateHashCode()).GetHashCode();
211
212     /// <summary>
213     /// Indicates whether the current link is equal to another link.
214     /// Two anonymous links (both with null Ids) are considered equal if their values are
↪ equal.
215     /// </summary>
216     /// <param name="other">The link to compare with this link.</param>
217     /// <returns>True if the current link is equal to the other parameter; otherwise,
↪ false.</returns>
218     [MethodImpl(MethodImplOptions.AggressiveInlining)]
219     public bool Equals(Link<TLinkAddress> other)
220     {
221         // Both have null IDs - compare only values
222         if (Id == null && other.Id == null)
223         {
224             return (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ??
↪ Array.Empty<Link<TLinkAddress>>());
225         }
226
227         // Only one has null ID - not equal
228         if (Id == null || other.Id == null)
229         {
230             return false;
231         }
232
233         // Both have IDs - compare IDs and values
234         return EqualityComparer.Instance.Equals(Id, other.Id) &&
235             (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ??
↪ Array.Empty<Link<TLinkAddress>>());
236     }
237
238     /// <summary>
239     /// Determines whether two link instances are equal.
240     /// </summary>
241     /// <param name="left">The first link to compare.</param>
242     /// <param name="right">The second link to compare.</param>
243     /// <returns>True if the links are equal; otherwise, false.</returns>
244     [MethodImpl(MethodImplOptions.AggressiveInlining)]
245     public static bool operator ==(Link<TLinkAddress> left, Link<TLinkAddress> right) =>
↪ left.Equals(right);
246
247     /// <summary>
248     /// Determines whether two link instances are not equal.
249     /// </summary>
250     /// <param name="left">The first link to compare.</param>
251     /// <param name="right">The second link to compare.</param>
252     /// <returns>True if the links are not equal; otherwise, false.</returns>
253     [MethodImpl(MethodImplOptions.AggressiveInlining)]
254     public static bool operator !=(Link<TLinkAddress> left, Link<TLinkAddress> right) =>
↪ !(left == right);
255 }
256

```

1.16 ./Link.Foundation.Links.Notation/LinkFormatExtensions.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Runtime.CompilerServices;
5 using System.Text;
6
7 namespace Link.Foundation.Links.Notation
8 {
9     /// <summary>
10    /// Provides extension methods for formatting <see cref="Link{TLinkAddress}"/> instances
↪ with FormatOptions.
11    /// </summary>
12    public static class LinkFormatExtensions
13    {
14        /// <summary>
15        /// Format a link using FormatOptions configuration.

```

```

16     /// </summary>
17     /// <typeparam name="TLinkAddress">The type used for link
↪ addresses/identifiers.</typeparam>
18     /// <param name="link">The link to format.</param>
19     /// <param name="options">The FormatOptions to use.</param>
20     /// <returns>Formatted string representation.</returns>
21     public static string FormatWithOptions<TLinkAddress>(this Link<TLinkAddress> link,
↪ FormatOptions options)
22     {
23         // Empty link
24         if (link.Id == null && (link.Values == null || link.Values.Count == 0))
25         {
26             return options.LessParentheses ? "" : "()";
27         }
28
29         // Link with only ID, no values
30         if (link.Values == null || link.Values.Count == 0)
31         {
32             var escapedId = Link<TLinkAddress>.EscapeReference(link.Id?.ToString());
33             return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
34                 escapedId :
35                 $"({escapedId})";
36         }
37
38         // Check if we should use indented format
39         bool shouldIndent = false;
40         if (options.ShouldIndentByRefCount(link.Values.Count))
41         {
42             shouldIndent = true;
43         }
44         else
45         {
46             // Try inline format first
47             var valuesStr = string.Join(" ", link.Values.Select(v => GetValueString(v)));
48             string testLine;
49             if (link.Id != null)
50             {
51                 var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
52                 testLine = options.LessParentheses ? $"{{idStr}}: {{valuesStr}}" : $"({idStr}:
↪ {{valuesStr}})";
53             }
54             else
55             {
56                 testLine = options.LessParentheses ? valuesStr : $"({valuesStr})";
57             }
58
59             if (options.ShouldIndentByLength(testLine))
60             {
61                 shouldIndent = true;
62             }
63         }
64
65         // Format with indentation if needed
66         if (shouldIndent && options.PreferInline == false)
67         {
68             return FormatIndented(link, options);
69         }
70
71         // Standard inline formatting
72         var values = string.Join(" ", link.Values.Select(v => GetValueString(v)));
73
74         // Link with values only (null id)
75         if (link.Id == null)
76         {
77             if (options.LessParentheses)
78             {
79                 var allSimple = link.Values.All(v => v.Values == null || v.Values.Count ==
↪ 0);
80                 if (allSimple)
81                 {
82                     return string.Join(" ", link.Values.Select(v =>
↪ Link<TLinkAddress>.EscapeReference(v.Id?.ToString())));
83                 }
84                 return values;
85             }
86             return $"({values})";
87         }
88     }

```

```

89     // Link with ID and values
90     var id = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
91     var withColon = $"{id}: {values}";
92     return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
93         withColon :
94         $"({withColon})";
95 }
96
97 /// <summary>
98 /// Format a link with indentation.
99 /// </summary>
100 private static string FormatIndented<TLinkAddress>(Link<TLinkAddress> link,
↪ FormatOptions options)
101 {
102     // FormatIndented is only reached with a non-empty Values collection,
103     // but the nullable annotation on Link<T>.Values does not know that.
104     var values = link.Values ?? Array.Empty<Link<TLinkAddress>>();
105
106     if (link.Id == null)
107     {
108         // Values only - format each on separate line
109         var lines = values.Select(v => options.IndentString + GetValueString(v));
110         return string.Join(Environment.NewLine, lines);
111     }
112
113     // Link with ID - format as id:\n value1\n value2
114     var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
115     var sb = new StringBuilder();
116     sb.Append($"{{idStr}}:");
117
118     foreach (var v in values)
119     {
120         sb.Append(Environment.NewLine);
121         sb.Append(options.IndentString);
122         sb.Append(GetValueString(v));
123     }
124
125     return sb.ToString();
126 }
127
128 /// <summary>
129 /// Get the string representation of a value.
130 /// </summary>
131 private static string GetValueString<TLinkAddress>(Link<TLinkAddress> value)
132 {
133     return value.ToLinkOrIdString();
134 }
135
136 /// <summary>
137 /// Check if a string needs to be wrapped in parentheses.
138 /// </summary>
139 private static bool NeedsParentheses(string? s)
140 {
141     return s != null && (s.Contains(" ") || s.Contains(":") || s.Contains("(") ||
↪ s.Contains(")"));
142 }
143 }
144 }

```

1.17 ./Link.Foundation.Links.Notation/LinksGroup.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Runtime.CompilerServices;
4  using Platform.Collections;
5  using Platform.Collections.Lists;
6
7  namespace Link.Foundation.Links.Notation
8  {
9      /// <summary>
10     /// Represents a group of links with hierarchical structure, where each group contains a
↪ primary link and optional nested groups.
11     /// This structure supports the indentation-based syntax of the Links Notation.
12     /// </summary>
13     /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
↪ any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
14     public struct LinksGroup<TLinkAddress> : IEquatable<LinksGroup<TLinkAddress>>
15     {
16         /// <summary>
17         /// Gets or sets the primary link of this group.

```

```

18     /// </summary>
19     public Link<TLinkAddress> Link
20     {
21         [MethodImpl(MethodImplOptions.AggressiveInlining)]
22         get;
23         [MethodImpl(MethodImplOptions.AggressiveInlining)]
24         set;
25     }
26
27     /// <summary>
28     /// Gets or sets the collection of nested groups within this group. Can be null if no
29     ↪ nested groups exist.
30     /// </summary>
31     public IList<LinksGroup<TLinkAddress>>? Groups
32     {
33         [MethodImpl(MethodImplOptions.AggressiveInlining)]
34         get;
35         [MethodImpl(MethodImplOptions.AggressiveInlining)]
36         set;
37     }
38
39     /// <summary>
40     ↪ /// Gets or sets a value indicating whether this group was written as a parenthesized
41     ↪ group.
42     ↪ /// A parenthesized group is a link in its own right, so it keeps its identifier and
43     ↪ values
44     ↪ instead of being unwrapped when it is used as a child of the indented identifier
45     ↪ syntax.
46     /// This is a parse-time hint and therefore takes no part in equality comparisons.
47     /// </summary>
48     internal bool IsParenthesized
49     {
50         [MethodImpl(MethodImplOptions.AggressiveInlining)]
51         get;
52         [MethodImpl(MethodImplOptions.AggressiveInlining)]
53         set;
54     }
55
56     /// <summary>
57     /// Initializes a new links group with the specified link and nested groups.
58     /// </summary>
59     /// <param name="link">The primary link of this group.</param>
60     /// <param name="groups">The nested groups within this group.</param>
61     [MethodImpl(MethodImplOptions.AggressiveInlining)]
62     public LinksGroup(Link<TLinkAddress> link, IList<LinksGroup<TLinkAddress>>? groups)
63     {
64         Link = link;
65         Groups = groups;
66     }
67
68     /// <summary>
69     /// Initializes a new links group with the specified link and no nested groups.
70     /// </summary>
71     /// <param name="link">The primary link of this group.</param>
72     [MethodImpl(MethodImplOptions.AggressiveInlining)]
73     public LinksGroup(Link<TLinkAddress> link) : this(link, null) { }
74
75     /// <summary>
76     ↪ /// Implicitly converts a links group to a list of links by flattening the hierarchical
77     ↪ structure.
78     /// </summary>
79     /// <param name="value">The links group to convert.</param>
80     [MethodImpl(MethodImplOptions.AggressiveInlining)]
81     public static implicit operator List<Link<TLinkAddress>>(LinksGroup<TLinkAddress> value)
82     ↪ => value.ToLinksList();
83
84     /// <summary>
85     ↪ /// Converts this links group to a flat list of links, preserving the hierarchical
86     ↪ relationships.
87     /// </summary>
88     /// <returns>A list of links representing the flattened structure of this
89     ↪ group.</returns>
90     [MethodImpl(MethodImplOptions.AggressiveInlining)]
91     public List<Link<TLinkAddress>> ToLinksList()
92     {
93         var list = new List<Link<TLinkAddress>>();
94         AppendToLinksList(list);
95         return list;
96     }

```

```

89
90     /// <summary>
91     /// Appends this links group to the specified list as flattened links.
92     /// </summary>
93     /// <param name="list">The list to append the links to.</param>
94     [MethodImpl(MethodImplOptions.AggressiveInlining)]
95     public void AppendToLinksList(List<Link<TLinkAddress>> list) => AppendToLinksList(list,
↵ Link, this);
96
97     /// <summary>
98     /// Recursively appends a links group to a list, combining dependencies with nested
↵ groups to create a flattened structure.
99     /// </summary>
100    /// <param name="list">The list to append the links to.</param>
101    /// <param name="dependency">The dependency link to combine with nested groups.</param>
102    /// <param name="group">The links group to process.</param>
103    [MethodImpl(MethodImplOptions.AggressiveInlining)]
104    public static void AppendToLinksList(List<Link<TLinkAddress>> list, Link<TLinkAddress>
↵ dependency, LinksGroup<TLinkAddress> group)
105    {
106        list.Add(dependency);
107        var groups = group.Groups;
108        if (groups != null && !groups.IsNullOrEmpty())
109        {
110            for (int i = 0; i < groups.Count; i++)
111            {
112                var innerGroup = groups[i];
113                AppendToLinksList(list, dependency.Combine(innerGroup.Link), innerGroup);
114            }
115        }
116    }
117
118    /// <summary>
119    /// Determines whether the specified object is equal to this links group.
120    /// </summary>
121    /// <param name="obj">The object to compare with this links group.</param>
122    /// <returns>True if the specified object is equal to this links group; otherwise,
↵ false.</returns>
123    [MethodImpl(MethodImplOptions.AggressiveInlining)]
124    public override bool Equals(object? obj) => obj is LinksGroup<TLinkAddress> linksGroup ?
↵ Equals(linksGroup) : false;
125
126    /// <summary>
127    /// Returns the hash code for this links group.
128    /// </summary>
129    /// <returns>A hash code for this links group.</returns>
130    [MethodImpl(MethodImplOptions.AggressiveInlining)]
131    public override int GetHashCode() => (Link, (Groups ??
↵ Array.Empty<LinksGroup<TLinkAddress>>()).GenerateHashCode()).GetHashCode();
132
133    /// <summary>
134    /// Indicates whether the current links group is equal to another links group.
135    /// </summary>
136    /// <param name="other">The links group to compare with this links group.</param>
137    /// <returns>True if the current links group is equal to the other parameter; otherwise,
↵ false.</returns>
138    [MethodImpl(MethodImplOptions.AggressiveInlining)]
139    public bool Equals(LinksGroup<TLinkAddress> other) => Link.Equals(other.Link) && (Groups
↵ ?? Array.Empty<LinksGroup<TLinkAddress>>()).EqualTo(other.Groups ??
↵ Array.Empty<LinksGroup<TLinkAddress>>());
140
141    /// <summary>
142    /// Determines whether two links group instances are equal.
143    /// </summary>
144    /// <param name="left">The first links group to compare.</param>
145    /// <param name="right">The second links group to compare.</param>
146    /// <returns>True if the links groups are equal; otherwise, false.</returns>
147    [MethodImpl(MethodImplOptions.AggressiveInlining)]
148    public static bool operator ==(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
↵ right) => left.Equals(right);
149
150    /// <summary>
151    /// Determines whether two links group instances are not equal.
152    /// </summary>
153    /// <param name="left">The first links group to compare.</param>
154    /// <param name="right">The second links group to compare.</param>
155    /// <returns>True if the links groups are not equal; otherwise, false.</returns>

```

```

156     [MethodImpl(MethodImplOptions.AggressiveInlining)]
157     public static bool operator !=(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
↪ right) => !(left == right);
158
159     /// <summary>
160     /// Returns a string representation of this links group.
161     /// </summary>
162     /// <returns>A string representation of this links group.</returns>
163     [MethodImpl(MethodImplOptions.AggressiveInlining)]
164     public override string ToString() => ToLinksList().Format();
165 }
166 }

```

1.18 ./Link.Foundation.Links.Notation/ParseException.cs

```

1  using System;
2  using System.Text;
3
4  namespace Link.Foundation.Links.Notation
5  {
6      /// <summary>
7      /// The error raised when a document does not parse, carrying the position the parser
8      /// stopped at.
9      /// </summary>
10     /// <remarks>
11     /// The position is the furthest one the parser reached, which is the character the
12     /// document stops making sense at rather than the point the last alternative gave up
13     /// on. It derives from <see cref="FormatException"/>, the exception the generated
14     /// parser raised on its own, so code that catches that keeps working.
15     /// A document nested deeper than <see cref="Parser.MaxDepth"/> is refused with this
16     /// exception too; then <see cref="MaxDepth"/> says how deep the nesting may go, and the
17     /// position is the group or the line that is one level too deep.
18     /// </remarks>
19     public class ParseException : FormatException
20     {
21         /// <summary>The number of characters a quoted line is cut down to.</summary>
22         private const int QuotedLineWidth = 80;
23
24         /// <summary>What a message writes in place of the part of a long line it left
↪ out.</summary>
25         private const string Ellipsis = "...";
26
27         /// <summary>
28         /// Creates an exception describing where <paramref name="subject"/> stopped parsing.
29         /// </summary>
30         /// <param name="subject">The document that failed to parse.</param>
31         /// <param name="offset">Offset of the position the parser stopped at.</param>
32         /// <param name="innerException">The error the generated parser raised, if any.</param>
33         public ParseException(string subject, int offset, Exception? innerException = null)
34             : this(Locate(subject ?? string.Empty, offset), null, innerException)
35         {
36         }
37
38         /// <summary>
39         /// Creates an exception saying that <paramref name="subject"/> nests deeper than
40         /// <paramref name="maxDepth"/> at <paramref name="offset"/>.
41         /// </summary>
42         /// <param name="subject">The document that nests too deeply.</param>
43         /// <param name="offset">Offset of the group or the line that is one level too
↪ deep.</param>
44         /// <param name="maxDepth">The deepest nesting allowed.</param>
45         /// <param name="innerException">The error the parser raised, if any.</param>
46         public ParseException(string subject, int offset, int maxDepth, Exception?
↪ innerException = null)
47             : this(Locate(subject ?? string.Empty, offset), maxDepth, innerException)
48         {
49         }
50
51         private ParseException(Position position, int? maxDepth, Exception? innerException)
52             : base(Describe(position, maxDepth), innerException)
53         {
54             Offset = position.Offset;
55             Line = position.Line;
56             Column = position.Column;
57             Found = position.Found;
58             LineText = position.LineText;
59             MaxDepth = maxDepth;
60         }
61
62         /// <summary>Offset of the offending position from the start of the document.</summary>

```

```

63     public int Offset { get; }
64
65     /// <summary>Line the offending position is on, counted from 1.</summary>
66     public int Line { get; }
67
68     /// <summary>Column the offending position is at, counted from 1.</summary>
69     public int Column { get; }
70
71     /// <summary>The character found instead, or null at the end of the document.</summary>
72     public char? Found { get; }
73
74     /// <summary>The offending line, as written, without its line ending.</summary>
75     public string LineText { get; }
76
77     /// <summary>
78     /// The deepest nesting allowed, when the document is nested deeper than that;
79     /// <see langword="null"/> for any other error.
80     /// </summary>
81     public int? MaxDepth { get; }
82     -
83     /// <summary>
84     /// The one-line summary: where the parser stopped and what stands there, or that the
85     /// nesting is too deep there.
86     /// </summary>
87     public string Summary => Summarize(new Position(Offset, Line, Column, Found, LineText),
↪ MaxDepth);
88
89     /// <summary>
90     /// The offending line with a caret under the offending column, quoted the way a
91     /// compiler quotes source. A long line is shown as a window around the caret, so the
92     /// message stays the same size whether the document has ten lines or fifteen hundred.
93     /// </summary>
94     public string Snippet => Quote(new Position(Offset, Line, Column, Found, LineText));
95
96     /// <summary>Where the parser stopped, and what the document holds there.</summary>
97     private readonly record struct Position(
98         int Offset,
99         int Line,
100        int Column,
101        char? Found,
102        string LineText);
103
104     /// <summary>The message the exception carries: the summary and the quoted
↪ line.</summary>
105     private static string Describe(Position position, int? maxDepth) =>
106         $"{(maxDepth.HasValue ? "Nesting too deep" : "Syntax error")}" at
↪ {Summarize(position, maxDepth)}\n{Quote(position)}";
107
108     private static string Summarize(Position position, int? maxDepth)
109     {
110         if (maxDepth.HasValue)
111         {
112             ↪ return $"line {position.Line}, column {position.Column}: nesting depth exceeds
the maximum of {maxDepth.Value}";
113         }
114         var found = position.Found.HasValue
115             ? $"{Escape(position.Found.Value)}\"
116             : "end of input";
117         return $"line {position.Line}, column {position.Column}: unexpected {found}";
118     }
119
120     private static string Quote(Position position)
121     {
122         var (quoted, column) = QuoteLine(position.LineText, position.Column);
123         var number = position.Line.ToString();
124         var gutter = new string(' ', number.Length);
125         return $"{number} | {quoted}\n{gutter} | {new string(' ', column - 1)}~";
126     }
127
128     private static string Escape(char character) => character switch
129     {
130         '\n' => "\\n",
131         '\r' => "\\r",
132         '\t' => "\\t",
133         '"' => "\\\"",
134         '\\' => "\\\\",
135         => character.ToString(),
136     };
137

```

```

138     /// <summary>
139     /// Turns the position the parser stopped at into a line, a column and the line
140     /// itself, so the message can point at the defect instead of quoting the rest of the
141     /// document.
142     /// </summary>
143     private static Position Locate(string document, int offset)
144     {
145         offset = Math.Max(0, Math.Min(offset, document.Length));
146
147         var line = 1;
148         var lineStart = 0;
149         for (var index = 0; index < offset; index++)
150         {
151             if (document[index] == '\n')
152             {
153                 line++;
154                 lineStart = index + 1;
155             }
156         }
157
158         var lineEnd = document.IndexOf('\n', lineStart);
159         if (lineEnd < 0) lineEnd = document.Length;
160
161         var lineText = document.Substring(lineStart, lineEnd - lineStart).TrimEnd('\r');
162         var found = offset < document.Length ? document[offset] : (char?)null;
163         return new Position(offset, line, offset - lineStart + 1, found, lineText);
164     }
165
166     /// <summary>
167     /// Cuts <paramref name="line"/> down to a window around <paramref name="column"/>, and
168     /// says which column the offending character sits at in that window. Both columns
169     /// count from 1.
170     /// </summary>
171     private static (string Quoted, int Column) QuoteLine(string line, int column)
172     {
173         if (line.Length <= QuotedLineWidth) return (line, column);
174
175         var target = column - 1;
176         var lastStart = line.Length - QuotedLineWidth;
177         var start = Math.Min(Math.Max(target - (QuotedLineWidth / 2), 0), lastStart);
178         var end = start + QuotedLineWidth;
179
180         var quoted = new StringBuilder();
181         if (start > 0) quoted.Append(Ellipsis);
182         quoted.Append(line, start, QuotedLineWidth);
183         if (end < line.Length) quoted.Append(Ellipsis);
184
185         var shift = start > 0 ? Ellipsis.Length : 0;
186         return (quoted.ToString(), target - start + shift + 1);
187     }
188 }
189 }

```

1.19 ./Link.Foundation.Links.Notation/Parser.cs

```

1  using System;
2  using System.Collections.Generic;
3
4  namespace Link.Foundation.Links.Notation
5  {
6      /// <summary>
7      /// Parses Links Notation documents into links.
8      /// </summary>
9      /// <remarks>
10     /// The rules live in <c>Parser.peg</c> and are compiled into
11     /// <see cref="GeneratedParser"/>; this class runs them and, when a document does not
12     /// parse, reports the position the parser stopped at instead of the bare
13     /// <see cref="FormatException"/> the generated parser raises.
14     /// </remarks>
15     public class Parser
16     {
17         /// <summary>
18         /// How deep links may nest when nothing says otherwise.
19         /// </summary>
20         /// <remarks>
21         /// The parser recurses once per level of nesting, and running out of stack ends the
22         /// process where no caller can catch it. The limit is set well below what fits in a
23         /// 1 MiB thread stack, and it is the same in every Links Notation implementation, so
24         /// a document one of them reads is not too deep for another.
25         /// </remarks>

```

```

26     public const int DefaultMaxDepth = 64;
27
28     /// <summary>
29     /// Creates a parser.
30     /// </summary>
31     /// <param name="comments">
32     /// When <see langword="false"/>, <c>#</c> is read as an ordinary character
33     /// instead of the start of a comment.
34     /// </param>
35     public Parser(bool comments = true) : this(comments, DefaultMaxDepth)
36     {
37     }
38
39     /// <summary>
40     /// Creates a parser that refuses links nested deeper than <paramref name="maxDepth"/>.
41     /// </summary>
42     /// <param name="comments">
43     /// When <see langword="false"/>, <c>#</c> is read as an ordinary character
44     /// instead of the start of a comment.
45     /// </param>
46     /// <param name="maxDepth">How deep links may nest; see <see cref="MaxDepth"/>.</param>
47     /// <exception cref="ArgumentOutOfRangeException"><paramref name="maxDepth"/> is
    ↪ negative.</exception>
48     public Parser(bool comments, int maxDepth)
49     {
50         Comments = comments;
51         MaxDepth = maxDepth;
52     }
53
54     /// <summary>
55     /// Whether <c>#</c> starts a comment that runs to the end of its line.
56     /// </summary>
57     public bool Comments { get; }
58
59     /// <summary>
60     /// How deep links may nest. Every parenthesized group and every indentation level
61     /// is one level, and the lines of a document are at level 0, so with a limit of 1
62     /// <c>(a)</c> parses while <c>((a))</c>, <c>(a (b))</c> and a group on an indented
63     /// line do not. A document nested deeper is refused with a
64     /// <see cref="ParseException"/> whose <see cref="ParseException.MaxDepth"/> is set,
65     /// rather than recursed into until the stack runs out.
66     /// </summary>
67     /// <exception cref="ArgumentOutOfRangeException">The value is negative.</exception>
68     public int MaxDepth
69     {
70         get => maxDepth;
71         init => maxDepth = value >= 0
72             ? value
73             : throw new ArgumentOutOfRangeException(nameof(MaxDepth), value, "Maximum
    ↪ nesting depth cannot be negative.");
74     }
75
76     private readonly int maxDepth;
77
78     /// <summary>
79     /// Parses a Links Notation document.
80     /// </summary>
81     /// <param name="subject">The document to parse.</param>
82     /// <param name="fileName">The optional file name to use in error messages.</param>
83     /// <returns>The links the document holds.</returns>
84     /// <exception cref="ParseException">
85     /// Thrown when the document does not parse, or nests deeper than
86     /// <see cref="MaxDepth"/>. The exception carries the line, the column and the
87     /// offending line of the position the parser stopped at.
88     /// </exception>
89     public IList<Link<string>> Parse(string subject, string? fileName = null)
90     {
91         // Comments are blanked rather than removed, so a position reported for the
92         // prepared document is the same position in the document the caller wrote.
93         var prepared = Comments ? Notation.Comments.StripComments(subject) : subject;
94         var tracer = new FurthestFailureTracer();
95         var parser = new GeneratedParser { Tracer = tracer, MaxDepth = MaxDepth };
96         try
97         {
98             return parser.Parse(prepared, fileName);
99         }
100         catch (GeneratedParser.NestingTooDeepException error)
101         {
102             throw new ParseException(subject, error.Offset, error.MaxDepth, error);

```

```

103     }
104     catch (FormatException error) when (error is not ParseException)
105     {
106         throw new ParseException(subject, tracer.Furthest, error);
107     }
108 }
109 }
110 }

```

1.20 ./Link.Foundation.Links.Notation/Quotes.cs

```

1  using System.Text;
2
3  namespace Link.Foundation.Links.Notation
4  {
5      /// <summary>
6      /// Reads a delimited reference, the shape shared by the grammar and by
7      /// <see cref="Comments"/>.
8      /// </summary>
9      /// <remarks>
10     /// Any number N of delimiters opens and closes a reference, and 2N delimiters stand
11     /// for N written as content. The grammar reads references through this class, so
12     /// blanking comments reaches a reference exactly as far as the parser does.
13     /// </remarks>
14     internal static class Quotes
15     {
16         /// <summary>
17         /// Reports whether the character is one of the delimiters a reference can be
18         /// written between.
19         /// </summary>
20         /// <param name="character">The character to test.</param>
21         /// <returns><see langword="true"/> when the character opens a reference.</returns>
22         internal static bool IsQuote(char character) =>
23             character == '"' || character == '\'' || character == '`';
24
25         /// <summary>
26         /// A body written between an even run of delimiters is substantive when it
27         /// holds at least one visible character and does not straddle a parenthesis.
28         /// An even run can always be read as delimiter pairs enclosing nothing, so the
29         /// n-quote reading is only taken when it carries something the pairs cannot.
30         /// </summary>
31         /// <param name="content">The body to weigh.</param>
32         /// <returns><see langword="true"/> when the body carries something a pair
33         ↪ cannot.</returns>
34         internal static bool IsSubstantiveBody(string content)
35         {
36             int depth = 0;
37             bool hasVisible = false;
38
39             foreach (var c in content)
40             {
41                 if (c == '(')
42                 {
43                     depth++;
44                 }
45                 else if (c == ')')
46                 {
47                     depth--;
48                     if (depth < 0) return false;
49                 }
50                 if (!char.IsWhiteSpace(c)) hasVisible = true;
51             }
52
53             return hasVisible && depth == 0;
54         }
55
56         /// <summary>
57         /// Reads the reference starting at the given position.
58         /// </summary>
59         /// <param name="subject">The whole document being read.</param>
60         /// <param name="start">The position of the opening delimiter.</param>
61         /// <param name="value">The content of the reference, without its delimiters.</param>
62         /// <param name="length">How many characters the reference spans.</param>
63         /// <returns><see langword="true"/> when a reference was read at that position.</returns>
64         internal static bool TryParse(string? subject, int start, out string? value, out int
65         ↪ length)
66         {
67             value = null;
68             length = 0;
69         }
70     }
71 }

```

```

68         if (subject == null || start >= subject.Length || !IsQuote(subject[start])) return
↪ false;
69
70         char quoteChar = subject[start];
71
72         // Count opening quotes
73         int quoteCount = 0;
74         int pos = start;
75         while (pos < subject.Length && subject[pos] == quoteChar)
76         {
77             quoteCount++;
78             pos++;
79         }
80
81         bool isEvenRun = quoteCount % 2 == 0;
82
83         string openClose = new string(quoteChar, quoteCount);
84         string escapeSeq = new string(quoteChar, quoteCount * 2);
85
86         var content = new StringBuilder();
87         while (pos < subject.Length)
88         {
89             // Check for escape sequence (2*N quotes)
90             if (pos + escapeSeq.Length <= subject.Length &&
91                 string.CompareOrdinal(subject, pos, escapeSeq, 0, escapeSeq.Length) == 0)
92             {
93                 content.Append(openClose); // 2*N quotes become N quotes
94                 pos += escapeSeq.Length;
95                 continue;
96             }
97
98             // Check for closing quotes (exactly N quotes, not more)
99             if (pos + quoteCount <= subject.Length &&
100                 string.CompareOrdinal(subject, pos, openClose, 0, quoteCount) == 0)
101             {
102                 int afterClose = pos + quoteCount;
103                 if (afterClose >= subject.Length || subject[afterClose] != quoteChar)
104                 {
105                     // Found valid closing
106                     if (isEvenRun && !IsSubstantiveBody(content.ToString()))
107                     {
108                         ↪ length);
109                         return SetEmptyReference(isEvenRun, quoteCount, out value, out
110
111                         value = content.ToString();
112                         length = afterClose - start;
113                         return true;
114                     }
115                 }
116
117                 // Take next character
118                 content.Append(subject[pos]);
119                 pos++;
120             }
121
122             // No valid closing found
123             return SetEmptyReference(isEvenRun, quoteCount, out value, out length);
124         }
125
126         /// <summary>
127         /// Finds where the reference starting at the given position ends.
128         /// </summary>
129         /// <param name="subject">The whole document being read.</param>
130         /// <param name="start">The position of the opening delimiter.</param>
131         /// <returns>
132         /// The position right after the closing delimiters, or -1 when no reference
133         /// starts there.
134         /// </returns>
135         internal static int ReferenceEnd(string subject, int start) =>
136             TryParse(subject, start, out var ignored, out var length) ? start + length : -1;
137
138         /// <summary>
139         /// A run of an even number of delimiters that does not open a reference with a
140         /// substantive body is the empty reference: the shortest reading, a bare
141         /// delimiter pair enclosing nothing, wins over a longer n-quote delimiter.
142         /// </summary>
143         private static bool SetEmptyReference(bool isEvenRun, int quoteCount, out string? value,
↪ out int length)

```

```

144     {
145         if (!isEvenRun)
146         {
147             value = null;
148             length = 0;
149             return false;
150         }
151
152         value = string.Empty;
153         length = quoteCount;
154         return true;
155     }
156 }
157 }

```

1.21 ./Link.Foundation.Links.Notation/ReferenceLiteral.cs

```

1  using System;
2  using System.Linq;
3  using System.Text;
4  using System.Text.RegularExpressions;
5
6  namespace Link.Foundation.Links.Notation;
7
8  /// <summary>Version 1 lossless Unicode reference literals.</summary>
9  public static class ReferenceLiteral
10 {
11     private static readonly UTF8Encoding Utf8 = new(false, true);
12     private static readonly Regex Literal = new(@"\\A~([0-9]+)\\{([0-9a-fA-F]*)\\}\\z");
13
14     /// <summary>Encodes exact Unicode as a version 1 UTF-8 hexadecimal literal.</summary>
15     public static string Encode(string text) =>
16     ↪ "$"~1{{{Convert.ToHexString(Utf8.GetBytes(text)).ToLowerInvariant()}}}}";
17
18     /// <summary>Decodes a complete literal; malformed UTF-8 and versions are refused.</summary>
19     public static string Decode(string literal)
20     {
21         var match = Literal.Match(literal);
22         if (!match.Success || match.Groups[1].Value != "1" || match.Groups[2].Length % 2 != 0)
23         {
24             throw new FormatException("Invalid or unsupported reference literal (expected
25     ↪ ~1{UTF-8 hex})");
26         }
27         try { return Utf8.GetString(Convert.FromHexString(match.Groups[2].Value)); }
28         catch (DecoderFallbackException error) { throw new FormatException("Reference literal
29     ↪ must contain valid UTF-8", error); }
30     }
31
32     internal static bool HasPrefix(string subject, int start)
33     {
34         if (start >= subject.Length || subject[start] != '~') return false;
35         var position = start + 1;
36         while (position < subject.Length && subject[position] is >= '0' and <= '9') position++;
37         return position > start + 1 && position < subject.Length && subject[position] == '{';
38     }
39
40     internal static bool TryRead(string subject, int start, out string value, out int length)
41     {
42         value = "";
43         length = 0;
44         if (!HasPrefix(subject, start)) return false;
45         var opening = subject.IndexOf('{', start);
46         var closing = subject.IndexOf('}', opening + 1);
47         if (closing < 0) throw new FormatException("Unclosed reference literal");
48         length = closing + 1 - start;
49         value = Decode(subject.Substring(start, length));
50         return true;
51     }
52
53     internal static string Format(string text)
54     {
55         Utf8.GetByteCount(text); // Refuse unpaired surrogates before formatting.
56         if (text.Length == 0 || text.Any(c => c < 32 || c == 127)) return Encode(text);
57         var needsQuotes = HasPrefix(text, 0) || text.StartsWith('#') || text.Any(c =>
58     ↪ char.IsWhiteSpace(c) || c is '\u0000' or '(' or ')' or ':' or '\\' or '"' or '`');
59         if (!needsQuotes) return text;
60         var (quote, count) = new[] { '\'', '"', '`' }
61             .Where(quote => text[0] != quote)
62             .Select(quote => (Quote: quote, Count: (LongestRun(text, quote) + 1) | 1))
63             .MinBy(candidate => candidate.Count);

```

```

60     var delimiter = new string(quote, count);
61     return $"{delimiter}{text}{delimiter}";
62 }
63
64 private static int LongestRun(string text, char quote)
65 {
66     var (longest, current) = (0, 0);
67     foreach (var character in text)
68     {
69         current = character == quote ? current + 1 : 0;
70         longest = Math.Max(longest, current);
71     }
72     return longest;
73 }
74 }

```

1.22 ./Link.Foundation.Links.Notation/StreamParseException.cs

```

1  using System;
2
3  namespace Link.Foundation.Links.Notation
4  {
5      /// <summary>A parser error located relative to the complete input stream.</summary>
6      public sealed class StreamParseException : FormatException
7      {
8          internal StreamParseException(ParseException error, int offset, int line, int column)
9              : base($"Stream parse error at line {line}, column {column}: {error.Message}", error)
10          {
11              Offset = offset;
12              Line = line;
13              Column = column;
14              ParseError = error;
15          }
16
17          /// <summary>The canonical parser error raised for the buffered record.</summary>
18          public ParseException ParseError { get; }
19
20          /// <summary>Absolute character offset from the start of the stream.</summary>
21          public int Offset { get; }
22
23          /// <summary>One-based line number in the complete stream.</summary>
24          public int Line { get; }
25
26          /// <summary>One-based column number in the complete stream.</summary>
27          public int Column { get; }
28      }
29 }

```

1.23 ./Link.Foundation.Links.Notation/StreamParser.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Runtime.CompilerServices;
4  using System.Text;
5  using System.Threading;
6  using System.Threading.Tasks;
7
8  namespace Link.Foundation.Links.Notation
9  {
10     /// <summary>Incrementally emits complete top-level Links Notation records.</summary>
11     public sealed class StreamParser
12     {
13         private const int DefaultMaxBufferSize = 10 * 1024 * 1024;
14         private const string BeforeReference = " \t\n\r(:";
15         private const string BeforeComment = " \t\n\r";
16
17         private readonly Parser parser;
18         private readonly StringBuilder buffer = new();
19         private readonly StringBuilder currentLine = new();
20         private readonly List<Link<string>> links = new();
21         private int? baseIndentation;
22         private bool lineClassified;
23         private int offset;
24         private int line;
25         private int column;
26         private int segmentOffset;
27         private int segmentLine;
28         private bool ended;
29         private int maxBufferSize = DefaultMaxBufferSize;
30
31         /// <summary>Creates a stream backed by a default canonical parser.</summary>
32         public StreamParser() : this(new Parser()) { }
33     }

```

```

34     /// <summary>Creates a stream backed by <paramref name="parser"/>.</summary>
35     public StreamParser(Parser parser)
36     {
37         parser = parser ?? throw new ArgumentNullException(nameof(parser));
38         Reset();
39     }
40
41     /// <summary>Raised once for every completed link.</summary>
42     public event Action<Link<string>>? LinkParsed;
43
44     /// <summary>Whether emitted links are retained for <see cref="Finish()"/> and <see
45     ↪ cref="Drain"/>.</summary>
46     public bool Collect { get; set; } = true;
47
48     /// <summary>The largest unresolved record accepted by the stream.</summary>
49     public int MaxBufferSize
50     {
51         get => maxBufferSize;
52         set => maxBufferSize = value > 0
53             ? value
54             : throw new ArgumentOutOfRangeException(nameof(value), "Maximum buffer size must
55             ↪ be positive.");
56     }
57
58     /// <summary>The point immediately after the last character written.</summary>
59     public StreamPosition Position => new(
60         offset,
61         line,
62         column,
63         buffer.Length + currentLine.Length);
64
65     /// <summary>Consumes a chunk and returns links made complete by it.</summary>
66     public IReadOnlyList<Link<string>> Write(string chunk)
67     {
68         ArgumentNullException.ThrowIfNull(chunk);
69         if (ended) throw new InvalidOperationException("Cannot write after Finish().");
70
71         var emitted = new List<Link<string>>();
72         foreach (var character in chunk)
73         {
74             currentLine.Append(character);
75             offset++;
76
77             if (character == '\n')
78             {
79                 buffer.Append( currentLine);
80                 currentLine.Clear();
81                 lineClassified = false;
82                 line++;
83                 column = 1;
84             }
85             else
86             {
87                 if (! lineClassified && character != ' ' && character != '\t' && character
88                 ↪ != '\r')
89                 {
90                     lineClassified = true;
91                     if (!( parser.Comments && character == Comments.Comment))
92                     {
93                         StartContentLine(LeadingSpaces( currentLine), emitted);
94                     }
95                 }
96                 column++;
97             }
98
99             if ( buffer.Length + currentLine.Length > MaxBufferSize)
100             {
101                 throw new ArgumentException(
102                     $"Buffered record exceeds maximum size of {MaxBufferSize} characters.",
103                     nameof(chunk));
104             }
105         }
106         return emitted;
107     }
108
109     /// <summary>Finishes the stream and returns all undrained links.</summary>
110     public IReadOnlyList<Link<string>> Finish() => Finish(string.Empty);
111
112     /// <summary>Consumes a final chunk and finishes the stream.</summary>
113     public IReadOnlyList<Link<string>> Finish(string chunk)

```

```

111     {
112         if ( ended)
113         {
114             if (chunk.Length > 0) throw new InvalidOperationException("Cannot write after
↪ Finish().");
115             return Collect ? links.ToArray() : Array.Empty<Link<string>>();
116         }
117
118         var emitted = chunk.Length == 0
119             ? new List<Link<string>>()
120             : new List<Link<string>>(Write(chunk));
121         var document = buffer.ToString() + currentLine;
122         if (document.Length > 0)
123         {
124             IList<Link<string>> parsed;
125             try
126             {
127                 parsed = parser.Parse(document);
128             }
129             catch (ParseException error)
130             {
131                 throw new StreamParseException(
132                     error,
133                     segmentOffset + error.Offset,
134                     segmentLine + error.Line - 1,
135                     error.Column);
136             }
137             Publish(parsed, emitted);
138             AdvanceSegment(document);
139         }
140
141         buffer.Clear();
142         currentLine.Clear();
143         baseIndentation = null;
144         ended = true;
145         return Collect ? links.ToArray() : emitted;
146     }
147
148     /// <summary>Returns and forgets links retained since the previous call.</summary>
149     public IReadOnlyList<Link<string>> Drain()
150     {
151         var links = links.ToArray();
152         links.Clear();
153         return links;
154     }
155
156     /// <summary>Reuses the parser while preserving its options and event handlers.</summary>
157     public StreamParser Reset()
158     {
159         buffer.Clear();
160         currentLine.Clear();
161         baseIndentation = null;
162         lineClassified = false;
163         links.Clear();
164         offset = 0;
165         line = 1;
166         column = 1;
167         segmentOffset = 0;
168         segmentLine = 1;
169         ended = false;
170         return this;
171     }
172
173     /// <summary>Lazily parses a sequence of chunks as a native enumerable.</summary>
174     public static IEnumerable<Link<string>> ParseChunks(
175         IEnumerable<string> chunks,
176         Parser? parser = null)
177     {
178         var stream = new StreamParser(parser ?? new Parser()) { Collect = false };
179         foreach (var chunk in chunks)
180         {
181             foreach (var link in stream.Write(chunk)) yield return link;
182         }
183         foreach (var link in stream.Finish()) yield return link;
184     }
185
186     /// <summary>Lazily parses an asynchronous sequence of chunks.</summary>
187     public static async IAsyncEnumerable<Link<string>> ParseChunksAsync(
188         IAsyncEnumerable<string> chunks,
189         Parser? parser = null,

```

```

190     [EnumeratorCancellation] CancellationToken cancellationToken = default)
191 {
192     var stream = new StreamParser(parser ?? new Parser()) { Collect = false };
193     await foreach (var chunk in chunks.WithCancellation(cancellationToken))
194     {
195         foreach (var link in stream.Write(chunk)) yield return link;
196     }
197     foreach (var link in stream.Finish()) yield return link;
198 }
199
200 private void StartContentLine(int indentation, List<Link<string>> emitted)
201 {
202     if ( buffer.Length > 0 &&
203         baseIndentation.HasValue &&
204         indentation <= baseIndentation.Value &&
205         StructurallyComplete( buffer.ToString(), parser.Comments))
206     {
207         try
208         {
209             var document = buffer.ToString();
210             var parsed = parser.Parse(document);
211             Publish(parsed, emitted);
212             AdvanceSegment(document);
213             buffer.Clear();
214             baseIndentation = null;
215         }
216         catch (ParseException)
217         {
218             // A rejected prefix can become valid when more input arrives.
219         }
220     }
221     baseIndentation ??= indentation;
222 }
223
224 private void Publish(IEnumerable<Link<string>> links, List<Link<string>> emitted)
225 {
226     foreach (var link in links)
227     {
228         emitted.Add(link);
229         if (Collect) links.Add(link);
230         LinkParsed?.Invoke(link);
231     }
232 }
233
234 private void AdvanceSegment(string document)
235 {
236     segmentOffset += document.Length;
237     foreach (var character in document)
238     {
239         if (character == '\n') segmentLine++;
240     }
241 }
242
243 private static int LeadingSpaces(StringBuilder line)
244 {
245     var indentation = 0;
246     while (indentation < line.Length && line[indentation] == ' ') indentation++;
247     return indentation;
248 }
249
250 private static bool StructurallyComplete(string document, bool comments)
251 {
252     var depth = 0;
253     for (var position = 0; position < document.Length; position++)
254     {
255         var character = document[position];
256         if (Quotes.IsQuote(character) && Follows(document, position, BeforeReference))
257         {
258             var end = Quotes.ReferenceEnd(document, position);
259             if (end < 0) return false;
260             position = end - 1;
261             continue;
262         }
263         if (comments && character == Comments.Comment && Follows(document, position,
264             ↪ BeforeComment))
265         {
266             var newline = document.IndexOf('\n', position);
267             if (newline < 0) break;

```

```

268         position = newline;
269         continue;
270     }
271     if (character == '(') depth++;
272     else if (character == ')') depth--;
273 }
274 return depth == 0;
275 }
276
277 private static bool Follows(string document, int position, string allowed) =>
278     position == 0 || allowed.IndexOf(document[position - 1]) >= 0;
279 }
280
281 /// <summary>An absolute stream position and the unresolved record size.</summary>
282 public readonly record struct StreamPosition(int Offset, int Line, int Column, int Buffered);
283 }

```

1.24 ./Link.Foundation.Links.Notation/ .cs

```

1 using System.Runtime.CompilerServices;
2
3 namespace Link.Foundation.Links.Notation
4 {
5     /// <summary>
6     /// A utility struct that provides convenient implicit conversions for creating links from
7     ↪ various tuple combinations.
8     /// The underscore name follows functional programming conventions for placeholder/utility
9     ↪ types.
10    /// </summary>
11    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
12    ↪ any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
13    public struct <TLinkAddress>
14    {
15        /// <summary>
16        /// Gets the underlying link represented by this utility struct.
17        /// </summary>
18        public readonly Link<TLinkAddress> Link;
19
20        /// <summary>
21        /// Initializes a new instance with the specified link.
22        /// </summary>
23        /// <param name="id">The link to wrap.</param>
24        [MethodImpl(MethodImplOptions.AggressiveInlining)]
25        public (Link<TLinkAddress> id) => Link = id;
26
27        /// <summary>
28        /// Implicitly converts a link to this utility struct.
29        /// </summary>
30        /// <param name="value">The link to convert.</param>
31        [MethodImpl(MethodImplOptions.AggressiveInlining)]
32        public static implicit operator <TLinkAddress>(Link<TLinkAddress> value) => new(value);
33
34        /// <summary>
35        /// Implicitly converts a link address to this utility struct.
36        /// </summary>
37        /// <param name="id">The link address to convert.</param>
38        [MethodImpl(MethodImplOptions.AggressiveInlining)]
39        public static implicit operator <TLinkAddress>(TLinkAddress id) => new(id);
40
41        /// <summary>
42        /// Implicitly converts a tuple of address and link to this utility struct.
43        /// </summary>
44        /// <param name="value">The tuple containing address and link.</param>
45        [MethodImpl(MethodImplOptions.AggressiveInlining)]
46        public static implicit operator <TLinkAddress>((TLinkAddress, Link<TLinkAddress>)
47        ↪ value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2 });
48
49        /// <summary>
50        /// Implicitly converts a tuple of address and two links to this utility struct.
51        /// </summary>
52        /// <param name="value">The tuple containing address and two links.</param>
53        [MethodImpl(MethodImplOptions.AggressiveInlining)]
54        public static implicit operator <TLinkAddress>((TLinkAddress, Link<TLinkAddress>,
55        ↪ Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2,
56        ↪ value.Item3 });
57
58        /// <summary>
59        /// Implicitly converts a tuple of address and three links to this utility struct.
60        /// </summary>
61        /// <param name="value">The tuple containing address and three links.</param>

```

```

56     [MethodImpl(MethodImplOptions.AggressiveInlining)]
57     public static implicit operator <TLinkAddress>((TLinkAddress, Link<TLinkAddress>,
↵ Link<TLinkAddress>, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[]
↵ { value.Item2, value.Item3, value.Item4 });
58
59     /// <summary>
60     /// Implicitly converts this utility struct back to a link.
61     /// </summary>
62     /// <param name="value">The utility struct to convert.</param>
63     [MethodImpl(MethodImplOptions.AggressiveInlining)]
64     public static implicit operator Link<TLinkAddress>( <TLinkAddress> value) => value.Link;
65 }
66 }

```

1.25 ./Link.Foundation.Links.Notation/id.cs

```

1  using System.Runtime.CompilerServices;
2
3  namespace Link.Foundation.Links.Notation
4  {
5      /// <summary>
6      /// A readonly struct that explicitly represents a link identifier/address.
7      /// This type is used to distinguish between regular values and explicit identifiers in link
↵ construction.
8      /// </summary>
9      /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
↵ any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
10     public readonly struct id<TLinkAddress>
11     {
12         /// <summary>
13         /// Gets the identifier value.
14         /// </summary>
15         public readonly TLinkAddress Id;
16
17         /// <summary>
18         /// Initializes a new identifier with the specified value.
19         /// </summary>
20         /// <param name="id">The identifier value.</param>
21         [MethodImpl(MethodImplOptions.AggressiveInlining)]
22         public id(TLinkAddress id) => Id = id;
23
24         /// <summary>
25         /// Explicitly converts a value to an identifier struct.
26         /// </summary>
27         /// <param name="id">The value to convert to an identifier.</param>
28         [MethodImpl(MethodImplOptions.AggressiveInlining)]
29         public static explicit operator id<TLinkAddress>(TLinkAddress id) => new(id);
30     }
31 }

```

1.26 ./Link.Foundation.Links.Notation.Tests/ApiTests.cs

```

1  using System;
2  using System.Collections.Generic;
3  using Xunit;
4
5  namespace Link.Foundation.Links.Notation.Tests
6  {
7      public static class ApiTests
8      {
9          [Fact]
10         public static void IsRefTest()
11         {
12             // C# doesn't have separate Ref/Link types, but we can test simple link behavior
13             var simpleLink = new Link<string>("some value", null);
14             Assert.Equal("some value", simpleLink.Id);
15             Assert.Null(simpleLink.Values);
16         }
17
18         [Fact]
19         public static void IsLinkTest()
20         {
21             // Test link with values
22             var values = new List<Link<string>> { new Link<string>("child", null) };
23             var link = new Link<string>("id", values);
24             Assert.Equal("id", link.Id);
25             Assert.Single(link.Values!);
26             Assert.Equal("child", link.Values?[0].Id);
27         }
28
29         [Fact]

```

```

30 public static void IsRefEquivalentTest()
31 {
32     // C# doesn't have separate Ref/Link types, but we can test simple link behavior
33     var simpleLink = new Link<string>("some value", null);
34     Assert.Equal("some value", simpleLink.Id);
35     Assert.Null(simpleLink.Values);
36 }
37
38 [Fact]
39 public static void IsLinkEquivalentTest()
40 {
41     // Test link with values
42     var values = new List<Link<string>> { new Link<string>("child", null) };
43     var link = new Link<string>("id", values);
44     Assert.Equal("id", link.Id);
45     Assert.Single(link.Values!);
46     Assert.Equal("child", link.Values?[0].Id);
47 }
48
49 [Fact]
50 public static void EmptyLinkTest()
51 {
52     var link = new Link<string>(null, new List<Link<string>>());
53     var output = link.ToString();
54     Assert.Equal("()", output);
55 }
56
57 [Fact]
58 public static void SimpleLinkTest()
59 {
60     var input = "(1: 1 1)";
61     var parser = new Parser();
62     var parsed = parser.Parse(input);
63
64     // Validate regular formatting
65     var output = parsed.Format();
66     Assert.Contains("1:", output);
67     Assert.Contains("1", output);
68 }
69
70 [Fact]
71 public static void LinkWithSourceTargetTest()
72 {
73     var input = "(index: source target)";
74     var parser = new Parser();
75     var parsed = parser.Parse(input);
76
77     // Validate regular formatting
78     var output = parsed.Format();
79     Assert.Equal(input, output);
80 }
81
82 [Fact]
83 public static void LinkWithSourceTypeTargetTest()
84 {
85     var input = "(index: source type target)";
86     var parser = new Parser();
87     var parsed = parser.Parse(input);
88
89     // Validate regular formatting
90     var output = parsed.Format();
91     Assert.Equal(input, output);
92 }
93
94 [Fact]
95 public static void SingleLineFormatTest()
96 {
97     var input = "id: value1 value2";
98     var parser = new Parser();
99     var parsed = parser.Parse(input);
100
101     // The parser should handle single-line format
102     var output = parsed.Format();
103     Assert.Contains("id", output);
104     Assert.Contains("value1", output);
105     Assert.Contains("value2", output);
106 }
107
108 [Fact]

```

```

109 public static void QuotedReferencesTest()
110 {
111     var input = @"("quoted id": "value with spaces")";
112     var parser = new Parser();
113     var parsed = parser.Parse(input);
114
115     var output = parsed.Format();
116     Assert.Contains("quoted id", output);
117     Assert.Contains("value with spaces", output);
118 }
119
120 [Fact]
121 public static void QuotedReferencesParsingTest()
122 {
123     // Test that quoted references are parsed correctly
124     var input = @"("quoted id": "value with spaces")";
125     var parser = new Parser();
126     var parsed = parser.Parse(input);
127
128     // Verify parsing worked correctly
129     var output = parsed.Format();
130     Assert.Contains("quoted id", output);
131     Assert.Contains("value with spaces", output);
132 }
133
134 [Fact]
135 public static void IndentedIdSyntaxParsingTest()
136 {
137     // Test that indented ID syntax is parsed correctly
138     var indented = "id:\n value1\n value2";
139     var inline = "(id: value1 value2)";
140
141     var parser = new Parser();
142     var indentedParsed = parser.Parse(indented);
143     var inlineParsed = parser.Parse(inline);
144
145     // Both should produce equivalent structures
146     var indentedOutput = indentedParsed.Format();
147     var inlineOutput = inlineParsed.Format();
148     Assert.Equal(indentedOutput, inlineOutput);
149     Assert.Equal("(id: value1 value2)", indentedOutput);
150 }
151
152 [Fact]
153 public static void IndentedIdSyntaxRoundtripTest()
154 {
155     var input = "id:\n value1\n value2";
156     var parser = new Parser();
157     var parsed = parser.Parse(input);
158
159     // Validate that we can format with indented syntax using FormatOptions
160     var options = new FormatOptions
161     {
162         MaxInlineRefs = 1, // Force indentation with more than 1 ref
163         PreferInline = false
164     };
165     var output = parsed.Format(options);
166     Assert.Equal(input, output);
167 }
168
169 [Fact]
170 public static void MultipleIndentedIdSyntaxParsingTest()
171 {
172     // Test that multiple indented ID links are parsed correctly
173     var indented = "id1:\n a\n b\nid2:\n c\n d";
174     var inline = "(id1: a b)\n(id2: c d)";
175
176     var parser = new Parser();
177     var indentedParsed = parser.Parse(indented);
178     var inlineParsed = parser.Parse(inline);
179
180     // Both should produce equivalent structures
181     var indentedOutput = indentedParsed.Format();
182     var inlineOutput = inlineParsed.Format();
183     Assert.Equal(indentedOutput, inlineOutput);
184     Assert.Equal("(id1: a b)\n(id2: c d)", indentedOutput);
185 }
186
187 [Fact]

```

```

188 public static void MultipleIndentedIdSyntaxRoundtripTest()
189 {
190     var input = "id1:\n a\n b\nid2:\n c\n d";
191     var parser = new Parser();
192     var parsed = parser.Parse(input);
193
194     // Validate that we can format with indented syntax using FormatOptions
195     var options = new FormatOptions
196     {
197         MaxInlineRefs = 1, // Force indentation with more than 1 ref
198         PreferInline = false
199     };
200     var output = parsed.Format(options);
201     Assert.Equal(input, output);
202 }
203 }
204 }

```

1.27 ./Link.Foundation.Links.Notation.Tests/BinaryLinksNotationTests.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.IO;
4 using System.Linq;
5 using Xunit;
6 using Link.Foundation.Links.Notation.Binary;
7
8 using LinoLink = Link.Foundation.Links.Notation.Link<string>;
9
10 namespace Link.Foundation.Links.Notation.Tests.Binary;
11
12 /// <summary>
13 /// The binary links notation: packets, sections, arities, widths and the
14 /// LiNo mapping on top of them. Mirrors <c>rust/tests/protocol packet tests.rs</c>.
15 /// </summary>
16 public sealed class BinaryLinksNotationTests
17 {
18     internal static string Hex(byte[] bytes) => string.Join(" ", bytes.Select(value =>
19     ↪ value.ToString("x2")));
20
21     internal static byte[] Unhex(string text) =>
22     ↪ text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Select(value =>
23     ↪ Convert.ToByte(value, 16)).ToArray();
24
25     private static IReadOnlyList<LinoLink> Parse(string text) => LinoFormat.ParseDocument(text);
26
27     private static bool ReferencesOption(string field) => field switch
28     {
29         "plain" => false,
30         "external" => true,
31         => throw new ArgumentException($"unknown references option {field}"),
32     };
33
34     private static bool WidthsOption(string field) => field switch
35     {
36         "uniform" => false,
37         "packed" => true,
38         => throw new ArgumentException($"unknown widths option {field}"),
39     };
40
41     /// <summary><c>address:reference reference...</c> separated by <c>;</c>, <c>#v</c> being
42     ↪ external.</summary>
43     private static List<(ulong Address, PacketReference[] References)> ParseLinks(string text) =>
44     ↪ text.Split(';').Select(link =>
45     {
46         var parts = link.Split(':', 2);
47         var references = parts[1].Split(' ').Select(reference => reference.StartsWith('#')
48         ? PacketReference.External(ulong.Parse(reference[1..]))
49         : PacketReference.Internal(ulong.Parse(reference))).ToArray();
50         return (ulong.Parse(parts[0]), references);
51     }).ToList();
52
53     /// <summary>The golden vectors shared with the Rust test suite: both ports must write and
54     ↪ read exactly these bytes.</summary>
55     private static IEnumerable<string> GoldenVectors() =>
56     ↪ File.ReadAllLines(Path.Combine(AppContext.BaseDirectory, "Protocol",
57     ↪ "binary-links-notation-vectors.txt"))
58     ↪ .Where(line => line.Length > 0 && !line.StartsWith('#'))
59     ↪ .Select(line => line.Split('\t'));
60
61 }

```

```

56     private static string Describe(IEnumerable<(ulong Address, PacketReference[] References)>
↪ links) =>
57         string.Join(";", links.Select(link => $"{link.Address}:{string.Join(" ",
↪ link.References)}"));
58
59     private static string Describe(LinksPacket packet) =>
60         $"{packet.ExternalReferences} " + string.Join(" | ", packet.Sections.Select(section =>
61             $"gap {section.Gap} arity {section.Arity} width {section.Width}: "
62             + string.Join(";", section.Links.Select(link => string.Join(" ", link)))));
63
64     [Fact]
65     public void GoldenDocumentsAreStable()
66     {
67         var checkedVectors = 0;
68         foreach (var fields in GoldenVectors().Where(fields => fields[0] == "document"))
69         {
70             Assert.Equal(6, fields.Length);
71             var text = fields[1].Replace("\\n", "\n");
72             var options = new BinaryLinoOptions
73             {
74                 ExternalReferences = ReferencesOption(fields[2]),
75                 Arity = ArityRange.Parse(fields[3]),
76                 PackedWidths = WidthsOption(fields[4]),
77             };
78             var document = Parse(text);
79             var binary = new BinaryLinoCodec(options);
80             Assert.True(fields[5] == Hex(binary.Encode(document)), $"{text} with {options}");
81             Assert.Equal(document, binary.Decode(Unhex(fields[5])));
82             checkedVectors++;
83         }
84         Assert.True(checkedVectors >= 80, $"only {checkedVectors} document vectors");
85     }
86
87     [Fact]
88     public void GoldenPacketsAreStable()
89     {
90         var checkedVectors = 0;
91         foreach (var fields in GoldenVectors().Where(fields => fields[0] == "links"))
92         {
93             Assert.Equal(5, fields.Length);
94             var links = ParseLinks(fields[1]);
95             var packet = LinksPacket.Pack(ReferencesOption(fields[2]), links,
↪ WidthsOption(fields[3]));
96             Assert.True(fields[4] == Hex(packet.ToBytes()), fields[1]);
97             var read = LinksPacket.FromBytes(Unhex(fields[4]));
98             Assert.Equal(Describe(packet), Describe(read));
99             Assert.Equal(Describe(links), Describe(read.Links()));
100             checkedVectors++;
101         }
102         Assert.True(checkedVectors >= 10, $"only {checkedVectors} links vectors");
103     }
104
105     [Fact]
106     public void WidthTiersFollowTheNumberOfLinks()
107     {
108         Assert.Equal(1, LinksPacket.AddressTier(0, false));
109         Assert.Equal(1, LinksPacket.AddressTier(255, false));
110         Assert.Equal(2, LinksPacket.AddressTier(256, false));
111         Assert.Equal(2, LinksPacket.AddressTier(65 535, false));
112         Assert.Equal(4, LinksPacket.AddressTier(65 536, false));
113         Assert.Equal(4, LinksPacket.AddressTier(uint.MaxValue, false));
114         Assert.Equal(8, LinksPacket.AddressTier((ulong)uint.MaxValue + 1, false));
115         // External references take the top bit, halving every range.
116         Assert.Equal(1, LinksPacket.AddressTier(127, true));
117         Assert.Equal(2, LinksPacket.AddressTier(128, true));
118         Assert.Equal(2, LinksPacket.AddressTier(32 767, true));
119         Assert.Equal(4, LinksPacket.AddressTier(32 768, true));
120         Assert.Equal(8, LinksPacket.AddressTier(ulong.MaxValue, false));
121         // No width holds an internal address in the external half.
122         Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
↪ LinksPacket.AddressTier(1UL << 63, true)).Kind);
123         Assert.Equal(ulong.MaxValue, LinksPacket.InternalCapacity(8, false));
124         Assert.Equal((ulong)long.MaxValue, LinksPacket.InternalCapacity(8, true));
125         Assert.Equal(127UL, LinksPacket.ExternalCapacity(1));
126     }
127
128     [Fact]
129     public void ExternalReferencesMatchPlatformDataHybrid()

```

```

130 {
131     foreach (var width in LinksPacket.Widths)
132     {
133         foreach (var value in new ulong[] { 0, 1, 2, 100,
↪ LinksPacket.ExternalCapacity(width) })
134         {
135             var raw = LinksPacket.EncodeExternal(value, width);
136             Assert.True(LinksPacket.TryDecodeExternal(raw, width, out var decoded));
137             Assert.Equal(value, decoded);
138         }
139         Assert.False(LinksPacket.TryDecodeExternal(LinksPacket.InternalCapacity(width,
↪ true), width, out var ignored));
140     }
141     Assert.Equal(0xFFUL, LinksPacket.EncodeExternal(1, 1));
142     Assert.Equal(0x80UL, LinksPacket.EncodeExternal(0, 1));
143 }
144
145 private static string ManyLinks(int count) =>
146     string.Join("\n", Enumerable.Range(0, count).Select(index => $"(n{index} m{index})"));
147
148 private static byte[] SectionWidths(LinksPacket packet) => packet.Sections.Select(section =>
↪ section.Width).ToArray();
149
150 [Fact]
151 public void UniformWidthsGrowPast256AddressesAndPackedWidthsStaySmall()
152 {
153     Assert.Equal(new byte[] { 1 },
↪ SectionWidths(LinoMapping.EncodeDocument(Parse(ManyLinks(3)))));
154
155     var document = Parse(ManyLinks(400));
156     var uniform = LinoMapping.EncodeDocument(document);
157     Assert.Equal(new byte[] { 2 }, SectionWidths(uniform));
158     var uniformBytes = uniform.ToBytes();
159     const int header = 1 + 2; // header byte + two-byte LEB128 count
160     Assert.Equal(header + (long)uniform.LinkCount * 2 * 2, uniformBytes.Length);
161
162     var packed = LinoMapping.EncodeDocument(document, new
↪ BinaryLinoOptions().WithPackedWidths());
163     Assert.Equal(1, SectionWidths(packed)[0]);
164     Assert.Contains((byte)2, SectionWidths(packed));
165     var packedBytes = packed.ToBytes();
166     Assert.True(packedBytes.Length < uniformBytes.Length);
167     foreach (var bytes in new[] { uniformBytes, packedBytes })
168     {
169         Assert.Equal(document, LinoMapping.DecodeDocument(LinksPacket.FromBytes(bytes)));
170     }
171 }
172
173 [Fact]
174 public void PackedWidthsNeverTakeMoreBytesThanUniformOnes()
175 {
176     var corpus = BinaryLinoCodecTests.Corpus.Append(ManyLinks(200)).Append("(1000 1)");
177     foreach (var text in corpus)
178     {
179         var document = Parse(text);
180         foreach (var options in BinaryLinoCodecTests.AllOptions().Where(options =>
↪ options.PackedWidths))
181         {
182             var packed = LinoMapping.EncodeDocument(document, options).ToBytes();
183             var uniform = LinoMapping.EncodeDocument(document,
↪ options.WithPackedWidths(false)).ToBytes();
184             Assert.True(packed.Length <= uniform.Length, $"{text} with {options}");
185         }
186     }
187 }
188
189 [Fact]
190 public void LargeExternalValuesWidenOnlyTheirSection()
191 {
192     var options = new BinaryLinoOptions().WithExternalReferences().WithPackedWidths();
193     Assert.Equal(new byte[] { 1 }, SectionWidths(LinoMapping.EncodeDocument(Parse("(100
↪ 1)"), options)));
194     Assert.Equal(new byte[] { 4, 1 }, SectionWidths(LinoMapping.EncodeDocument(Parse("(70000
↪ 1)"), options)));
195     // Beyond 63 bits the number falls back to in-band unary links.
196     Assert.True(LinoMapping.EncodeDocument(Parse("18446744073709551615"), options).LinkCount
↪ > 60);

```

```

198 }
199
200 private static int[] LinkLengths(IReadOnlyList<LinoLink> document, BinaryLinoOptions options)
201 {
202     var packet = LinoMapping.EncodeDocument(document, options);
203     Assert.Equal(document, new BinaryLinoCodec().Decode(packet.ToBytes()));
204     return packet.Links().Select(link => link.References.Length).ToArray();
205 }
206
207 [Fact]
208 public void ArityOptionsChooseDoubletsTripletsOrAnyLength()
209 {
210     var document = Parse("(a b c)\n(d e)");
211     var external = new BinaryLinoOptions().WithExternalReferences();
212     Assert.All(LinkLengths(document, external), length => Assert.Equal(2, length));
213     // `(a b c)` is one triplet; the strings `(String code point)` stay doublets.
214     var triplets = LinkLengths(document, external.WithArity(ArityRange.Between(2, 3)));
215     Assert.Single(triplets, length => length == 3);
216     Assert.All(triplets, length => Assert.InRange(length, 2, 3));
217     Assert.Equal(triplets, LinkLengths(document, external.WithArity(ArityRange.AtLeast(1))));
218     // Only any length turns a one-item list, here the root, into one link.
219     Assert.Equal(new[] { 2, 2, 2, 2 }, LinkLengths(Parse("a"), external));
220     Assert.Equal(new[] { 2, 2, 1 }, LinkLengths(Parse("a"),
    ↪ external.WithArity(ArityRange.AtLeast(1))));
221     // Long lists only become single links when the range allows their length.
222     var longList = Parse("(a b c d e f g h)");
223     foreach (var (arity, singleLink) in new[]
224     {
225         (ArityRange.Between(2, 3), false),
226         (ArityRange.Between(2, 8), true),
227         (ArityRange.AtLeast(2), true),
228     })
229     {
230         Assert.True(LinkLengths(longList, external.WithArity(arity)).Max() == 8 ==
    ↪ singleLink, arity.ToString());
231     }
232 }
233
234 [Fact]
235 public void AritiesWithoutDoubletsAreUnencodable()
236 {
237     foreach (var arity in new[] { ArityRange.Exactly(3), ArityRange.Exactly(1),
    ↪ ArityRange.AtLeast(3) })
238     {
239         var error = Assert.Throws<BinaryNotationException>(
240             () => LinoMapping.EncodeDocument(Parse("(a b)"), new
    ↪ BinaryLinoOptions().WithArity(arity)));
241         Assert.Equal(BinaryErrorKind.Unencodable, error.Kind);
242     }
243 }
244
245 [Fact]
246 public void ArityRangesParseAndDisplay()
247 {
248     foreach (var (text, range) in new[]
249     {
250         ("2", ArityRange.Doublets),
251         ("2..3", ArityRange.Between(2, 3)),
252         ("1..", ArityRange.AtLeast(1)),
253         ("3..3", ArityRange.Exactly(3)),
254         ("++2", ArityRange.Doublets),
255     })
256     {
257         Assert.Equal(range, ArityRange.Parse(text));
258     }
259     Assert.Equal("3", ArityRange.Between(3, 3).ToString());
260     Assert.Equal("2..3", ArityRange.Between(2, 3).ToString());
261     Assert.Equal("1..", ArityRange.AtLeast(1).ToString());
262     Assert.Equal(ArityRange.Doublets, new BinaryLinoOptions().Arity);
263     Assert.Equal(default, new BinaryLinoOptions().WithArity(ArityRange.Doublets));
264     foreach (var invalid in new[] { "", "0", "0..2", "3..2", "a", "1..b", "..3", "-1", " 2",
    ↪ "2 ", "++2", null })
265     {
266         Assert.False(ArityRange.TryParse(invalid, out var ignored), invalid);
267     }
268     Assert.False(ArityRange.TryParse("3..2", out var invalidRange, out var error));
269     Assert.Equal("arity range 3..2 is empty", error);

```

```

270     Assert.False(ArityRange.TryParse("x", out invalidRange, out error));
271     Assert.Equal("invalid arity 'x': expected n, min..max or min..", error);
272     Assert.Equal("arity must be at least 1", Assert.Throws<FormatException>(() =>
↪ ArityRange.Parse("0")).Message);
273     Assert.True(ArityRange.AtLeast(2).Contains(1 000));
274     Assert.False(ArityRange.Between(2, 3).Contains(4));
275     Assert.False(ArityRange.Between(2, 3).Contains(1));
276     Assert.True(ArityRange.Exactly(2).IsFixed && !ArityRange.AtLeast(2).IsFixed);
277 }
278
279 [Fact]
280 public void ReplyOptionsFollowTheReceivedPacket()
281 {
282     var document = Parse("(a b c) 70000");
283     var options = new BinaryLinoOptions()
284         .WithExternalReferences()
285         .WithArity(ArityRange.Between(2, 3))
286         .WithPackedWidths();
287     Assert.Equal(options, BinaryLinoOptions.OfPacket(LinoMapping.EncodeDocument(document,
↪ options)));
288     Assert.Equal(default, BinaryLinoOptions.OfPacket(LinoMapping.EncodeDocument(document)));
289     Assert.Equal(default, BinaryLinoOptions.OfPacket(new LinksPacket()));
290     Assert.Equal("BinaryLinoOptions { ExternalReferences = True, Arity = 2..3, PackedWidths
↪ = True }", options.ToString());
291 }
292
293 private static Section DoubletSection(ulong gap, ArityRange arity, byte width, params (ulong
↪ Source, ulong Target)[] links)
294 {
295     var section = new Section { Gap = gap, Arity = arity, Width = width };
296     section.Links.AddRange(links.Select(link =>
297         new[] { PacketReference.Internal(link.Source), PacketReference.Internal(link.Target)
↪ }));
298     return section;
299 }
300
301 [Fact]
302 public void HandBuiltPacketsDecode()
303 {
304     // One doublet (One One) is 2~1 in unary, so `(Number 7)` is the number 2.
305     var links = new (ulong, ulong)[] { (1, 1), (2, 6), (7, 0), (4, 8) };
306     var compact = new LinksPacket();
307     compact.Sections.Add(DoubletSection(5, ArityRange.Doublets, 1, links));
308     var bytes = compact.ToBytes();
309     Assert.Equal("10 04 01 01 02 06 07 00 04 08", Hex(bytes));
310     Assert.Equal("2", LinoFormat.FormatDocument(new BinaryLinoCodec().Decode(bytes)));
311
312     // The same links as a variable section of width 2 use the explicit layout.
313     var explicitLayout = new LinksPacket();
314     explicitLayout.Sections.Add(DoubletSection(5, ArityRange.AtLeast(1), 2, links));
315     bytes = explicitLayout.ToBytes();
316     Assert.Equal("12 01 1d 05 00 04 01 01 00 01 00 01 02 00 06 00 01 07 00 00 00 01 04 00 08
↪ 00", Hex(bytes));
317     Assert.Equal("2", LinoFormat.FormatDocument(new BinaryLinoCodec().Decode(bytes)));
318     Assert.Equal("10 00", Hex(new LinksPacket().ToBytes()));
319 }
320
321 private static PacketReference[] Internals(params ulong[] addresses) =>
322     addresses.Select(PacketReference.Internal).ToArray();
323
324 [Fact]
325 public void PackingRejectsLinksItCannotLayOut()
326 {
327     void Unencodable(bool externalReferences, params (ulong, PacketReference[])[] links)
328     {
329         var error = Assert.Throws<BinaryNotationException>(() =>
↪ LinksPacket.Pack(externalReferences, links, true));
330         Assert.Equal(BinaryErrorKind.Unencodable, error.Kind);
331     }
332     Unencodable(false, (0, Internals(1))); // address 0 is null
333     Unencodable(false, (2, Internals(1)), (1, Internals(1))); // descending
334     Unencodable(false, (2, Internals(1)), (2, Internals(1))); // repeated
335     Unencodable(false, (1, Array.Empty<PacketReference>())); // no references
336     Unencodable(false, (1, new[] { PacketReference.External(1) }));
337     Unencodable(true, (1, new[] { PacketReference.External(1UL << 63) }));
338     Unencodable(true, (1, Internals(1UL << 63))); // the top bit marks externals
339     Assert.NotNull(LinksPacket.Pack(false, new[] { (1UL, Internals(ulong.MaxValue)) },
↪ true));

```

```

340 }
341
342 [Fact]
343 public void WritingRejectsSectionsThatDoNotHoldTheirLinks()
344 {
345     static void Unencodable(Section section)
346     {
347         var packet = new LinksPacket();
348         packet.Sections.Add(section);
349         var error = Assert.Throws<BinaryNotationException>(() => packet.ToBytes());
350         Assert.Equal(BinaryErrorKind.Unencodable, error.Kind);
351     }
352     Unencodable(DoubletSection(0, ArityRange.Exactly(3), 1, (1, 1)));
353     Unencodable(DoubletSection(0, ArityRange.Exactly(0), 1));
354     Unencodable(DoubletSection(0, ArityRange.Between(3, 2), 1));
355     Unencodable(DoubletSection(0, ArityRange.Exactly(1UL << 62), 1));
356     Unencodable(DoubletSection(0, ArityRange.Doublets, 3, (1, 1)));
357     Unencodable(DoubletSection(0, ArityRange.Doublets, 1, (256, 0)));
358     Unencodable(DoubletSection(ulong.MaxValue, ArityRange.Doublets, 1, (1, 1)));
359 }
360
361 private static BinaryNotationException DecodeError(byte[] bytes, DecodeLimits? limits = null)
362 {
363     var protocol = new BinaryLiNoCodec { Limits = limits ?? DecodeLimits.Default };
364     return Assert.Throws<BinaryNotationException>(() => protocol.Decode(bytes));
365 }
366
367 /// <summary>Each packet breaks exactly one rule, the one its detail names.</summary>
368 [Theory]
369 [InlineData("", "empty input")]
370 [InlineData("20 00", "unsupported binary header byte 0x20")]
371 [InlineData("10 01 06 00", "link 6 refers to Internal(6), which is not an earlier link")]
372 [InlineData("10 01 07 00", "link 6 refers to Internal(7), which is not an earlier link")]
373 [InlineData("10 02 00", "unexpected end of packet")]
374 [InlineData("10 00 00", "trailing bytes after the packet")]
375 [InlineData("10 ff ff ff ff ff ff ff ff 7f", "LEB128 value overflows 64 bits")] // a
↪ tenth byte above 1
376 [InlineData("10 80 80 80 80 80 80 80 80 81", "LEB128 value overflows 64 bits")] // an
↪ eleventh byte
377 [InlineData("10 01 01 01", "the root link is not a list")]
378 [InlineData("10 02 00 02 04 06", "broken element chain")] // the chain ends in marker 2, not
↪ null
379 [InlineData("10 01 04 02", "broken element chain")]
380 [InlineData("10 02 02 00 04 06", "marker 2 used as a value")]
381 [InlineData("10 03 01 00 06 00 04 07", "marker 1 cannot start a typed value")]
382 [InlineData("12 02 14 05 01 20 02 02 06 00 04 07", "a number needs exactly one value")] //
↪ (Number) alone
383 [InlineData("10 03 05 00 06 00 04 07", "an identified link needs an id")]
384 [InlineData("10 03 02 03 06 00 04 07", "expected a unary number")] // (Number 3): marker 3
↪ is no number
385 [InlineData("10 05 05 05 06 00 02 07 08 00 04 09", "expected a unary number")] // (Number 6)
↪ where link 6 is (5 5)
386 [InlineData("10 04 00 00 05 06 07 00 04 08", "a link id must be a reference")] // the id is
↪ ()
387 [InlineData("16 00", "the explicit layout keeps the header width bits clear")]
388 [InlineData("12 01 04 05 00", "arity must be at least 1")]
389 [InlineData("12 01 18 01 01 05 00", "link length outside the section arity 1..2")] // a link
↪ of 6
390 [InlineData("12 01 f8 ff ff ff ff ff ff ff 01 ff ff ff ff ff ff ff ff 01 00", "arity
↪ range overflows 64 bits")]
391 [InlineData("12 01 24 ff ff ff ff ff ff ff ff 01 01", "addresses overflow 64 bits")] //
↪ the gap
392 [InlineData("12 01 20 ff ff ff ff ff ff ff ff 01", "addresses overflow 64 bits")] // the
↪ count
393 [InlineData("12 01 24 06 01 01 01", "a LiNo packet stores its links contiguously from
↪ address 6, found link 7 where 6 belongs")]
394 [InlineData("12 02 24 05 01 24 01 01 01 04 06", "a LiNo packet stores its links
↪ contiguously from address 6, found link 8 where 7 belongs")] // a hole
395 public void MalformedPacketsAreRejected(string hex, string detail)
396 {
397     var error = DecodeError(Unhex(hex));
398     Assert.Equal(BinaryErrorKind.Malformed, error.Kind);
399     Assert.Equal(detail, error.Detail);
400 }
401
402 private static List<ulong Address, PacketReference[] References> FromAddress6(params
↪ PacketReference[] links) =>

```

```

403         links.Select((references, index) => (6UL + (ulong)index, references)).ToList();
404
405     [Fact]
406     public void InvalidCodePointsAreRejected()
407     {
408         // (String (0x10000)) is not a valid code point.
409         var links = FromAddress6(
410             new[] { PacketReference.External(0x11 0000), PacketReference.Null },
411             Internals(3, 6),
412             new[] { PacketReference.Internal(7), PacketReference.Null },
413             Internals(4, 8));
414         Assert.Equal("invalid code point 1114112", DecodeError(LinksPacket.Pack(true, links,
415     ↪ false).ToBytes()).Detail);
416     }
417
418     [Fact]
419     public void HostilePacketsHitLimits()
420     {
421         void Limited(string hex, DecodeLimits limits) =>
422             Assert.True(BinaryErrorKind.LimitExceeded == DecodeError(Unhex(hex), limits).Kind,
423     ↪ hex);
424         var fewLinks = new DecodeLimits { MaxLinks = 8 };
425         Limited("10 09", fewLinks);
426         Limited("12 09", fewLinks); // more sections than links allowed
427         Limited("12 02 20 05 20 05", fewLinks); // 10 in all
428         var fewReferences = new DecodeLimits { MaxReferences = 5 };
429         Limited("10 03 01 01 01 01 01", fewReferences);
430         Limited("12 01 18 00 01 05", fewReferences); // one link of 6
431
432         // A doubling chain `d(k) = (d(k-1) d(k-1))` expands to 2^k nodes.
433         var chain = new List<PacketReference> { new[] { PacketReference.Null,
434     ↪ PacketReference.Null } };
435         for (ulong address = 6; address < 60; address++)
436         {
437             chain.Add(Internals(address, address));
438         }
439         var last = 5 + (ulong)chain.Count;
440         chain.Add(new[] { PacketReference.Internal(last), PacketReference.Null });
441         chain.Add(Internals(4, last + 1));
442         var bomb = LinksPacket.Pack(false, FromAddress6(chain.ToArray(), false);
443         Assert.Equal("limit exceeded: too many LiNo nodes", DecodeError(bomb.ToBytes(), new
444     ↪ DecodeLimits { MaxNodes = 1 << 12 }).Message);
445
446         var deep = string.Concat(Enumerable.Repeat("(y ", 40)) + "x" + new string(')', 40);
447         var bytes = new BinaryLinoCodec().Encode(Parse(deep));
448         Assert.Equal("limit exceeded: nesting deeper than 10", DecodeError(bytes, new
449     ↪ DecodeLimits { MaxDepth = 10 }).Message);
450
451         var threeLinks = new BinaryLinoCodec().Encode(Parse("a b c"));
452         Assert.Equal("limit exceeded: chain too long", DecodeError(threeLinks, new DecodeLimits
453     ↪ { MaxNodes = 2 }).Message);
454         Assert.Equal(ulong.MaxValue, DecodeLimits.Unlimited.MaxLinks);
455     }
456 }

```

1.28 ./Link.Foundation.Links.Notation.Tests/BinaryLinoCodecTests.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.IO;
4  using System.Linq;
5  using Xunit;
6  using Link.Foundation.Links.Notation.Binary;
7
8  using LinoLink = Link.Foundation.Links.Notation.Link<string>;
9
10 namespace Link.Foundation.Links.Notation.Tests.Binary;
11
12 /// <summary>Binary and text LiNo protocol codecs (issue #105).</summary>
13 public sealed class BinaryLinoCodecTests
14 {
15     [Fact]
16     public void EncoderUsesConfiguredLimits()
17     {
18         var document = new[] { new LinoLink("abcdef", new[] { new LinoLink("value") }) };
19         foreach (var limits in new[]
20         {
21             new DecodeLimits { MaxNodes = 1 },
22             new DecodeLimits { MaxStringBytes = 2 },
23             new DecodeLimits { MaxDepth = 1 },

```

```

24         new DecodeLimits { MaxLinks = 1 },
25         new DecodeLimits { MaxReferences = 1 },
26     })
27     {
28         Assert.Throws<BinaryNotationException>(() => new BinaryLinoCodec { Limits = limits
↪     }.Encode(document));
29     }
30     var deep = new LinoLink("a");
31     for (var index = 0; index < 70; index++) deep = new LinoLink(null, new[] { deep });
32     var model = new[] { deep };
33     var codec = new BinaryLinoCodec { Limits = new DecodeLimits { MaxDepth = 80 } };
34     Assert.Equal(model, codec.Decode(codec.Encode(model)));
35 }
36
37 internal static readonly string[] Corpus =
38 {
39     "()",
40     "((1: 1 1)) ((1: 1 2))",
41     "((1 1)) ()",
42     "((1: 1 1)) ()",
43     "((($i: $s $t)) (($i: $s $t)))",
44     "((($index: $source $target)) (($index: $target $source)))",
45     "(a b c d)",
46     "(name: 'with space' \"it's\")",
47     "((a))",
48     "(((a)))",
49     "(a (b c) ((d)))",
50     "1\n2\n3",
51     "hello",
52     "\U0001F600 привет \u4E16\u754C",
53     "0",
54     "007",
55     "9223372036854775807",
56     "9223372036854775808",
57     "18446744073709551615",
58     "18446744073709551616",
59     "'multi\nline'",
60     ".dot",
61     "'a''b''c''d'",
62     "(1: (2: 3 4) 5)",
63     "()",
64     "(* *)",
65     "(type: type type)",
66 };
67
68 internal static IEnumerable<BinaryLinoOptions> AllOptions()
69 {
70     foreach (var externalReferences in new[] { false, true })
71     {
72         foreach (var arity in new[] { ArityRange.Doublets, ArityRange.Between(2, 3),
↪     ArityRange.AtLeast(1) })
73         {
74             foreach (var packedWidths in new[] { false, true })
75             {
76                 yield return new BinaryLinoOptions
77                 {
78                     ExternalReferences = externalReferences,
79                     Arity = arity,
80                     PackedWidths = packedWidths,
81                 };
82             }
83         }
84     }
85 }
86
87 private static IReadOnlyList<LinoLink> Parse(string text) => LinoFormat.ParseDocument(text);
88
89 [Fact]
90 public void EveryOptionSetRoundTripsTheCorpus()
91 {
92     foreach (var text in Corpus)
93     {
94         var document = Parse(text);
95         foreach (var options in AllOptions())
96         {
97             var binary = new BinaryLinoCodec(options);
98             var bytes = binary.Encode(document);
99             Assert.InRange(bytes[0], 0x10, 0x1F);

```

```

100     Assert.True(document.SequenceEqual(binary.Decode(bytes)), $"{text} with
→   {options}")};
101         }
102     }
103 }
104
105 [Fact]
106 public void CanonicalTextRoundTripsTheCorpus()
107 {
108     foreach (var text in Corpus)
109     {
110         var document = Parse(text);
111         var canonical = LinoFormat.FormatDocument(document);
112         Assert.True(document.SequenceEqual(Parse(canonical)), $"{text} -> {canonical}");
113     }
114     Assert.Equal("() ((1 1))", LinoFormat.FormatDocument(Parse("()((1 1)))"));
115     Assert.Equal("((1: 1 1)) ((1: 1 2))", LinoFormat.FormatDocument(Parse("((1: 1 1)) ((1: 1
→ 2))")));
116     // Only a binary message carries an id without values; links-notation reads its text
→ back as the reference.
117     Assert.Equal("(a:", LinoFormat.FormatDocument(new[] { LinoFormat.Link("a", new
→ List<LinoLink>()) }));
118     Assert.True(Parse("(a:)").SequenceEqual(new[] { LinoFormat.Reference("a") }));
119 }
120
121 [Fact]
122 public void ParsedDocumentsMatchTheRustModel()
123 {
124     // The C# parser wraps single references in an extra group; the
125     // canonical model drops it so both ports decode to equal documents.
126     // These structures match the canonical model used by the shared vectors.
127     var a = LinoFormat.Reference("a");
128     LinoLink Group(LinoLink value) => LinoFormat.Link(null, new List<LinoLink> { value });
129     Assert.Equal(new[] { a }, Parse("a"));
130     Assert.Equal(new[] { a }, Parse("(a)"));
131     Assert.Equal(new[] { Group(a) }, Parse("((a))"));
132     Assert.Equal(new[] { Group(Group(a)) }, Parse("(((a)))"));
133     Assert.Equal(new[] { LinoFormat.Link("a", new List<LinoLink> { LinoFormat.Reference("b")
→ }) }, Parse("(a: (b))"));
134     Assert.Equal(new[] { LinoFormat.Reference("1"), LinoFormat.Reference("2"),
→ LinoFormat.Reference("3") }, Parse("1\n2\n3"));
135 }
136
137 [Fact]
138 public void ReferencesAreQuotedOnlyWhenNeeded()
139 {
140     Assert.Equal("plain", LinoFormat.FormatReference("plain"));
141     Assert.Equal("$x", LinoFormat.FormatReference("$x"));
142     Assert.Equal("~1{}", LinoFormat.FormatReference("{}"));
143     Assert.Equal("'a b'", LinoFormat.FormatReference("a b"));
144     Assert.Equal("\"it's\"", LinoFormat.FormatReference("it's"));
145     Assert.Equal("`\"`", LinoFormat.FormatReference("\"\""));
146     Assert.Equal("\\'\\'\\'\\'\\'\\'\\'\\'\\'", LinoFormat.FormatReference("'\"'"));
147     foreach (var reference in new[]
→ { "", "a b", "a:b", "(x)", "it's", "'\\'", "\"\\\"", "tab\there", "a\nb", "'x'", "x'",
→ "\"\", \"'\", \"'\", \"'\", \"'\", \"'\", \"'\", \"'\", \"'\", \"'\", \"'\", \"'\", \"'\",
→ "\"'x'\"", "a'b\"", "\"'\"", "(a)" })
148     {
149         var formatted = LinoFormat.FormatReference(reference);
150         var document = Parse(formatted);
151         Assert.Equal(formatted, LinoFormat.FormatDocument(document));
152         Assert.Equal(new[] { LinoFormat.Reference(reference) }, document);
153     }
154 }
155
156 [Fact]
157 public void EveryShortReferenceOverDelimitersRoundTrips()
158 {
159     var alphabet = new[] { '\'', '"', '`', 'a', ' ', '(', ')', ':' };
160     var stack = new Stack<string>();
161     stack.Push(string.Empty);
162     while (stack.TryPop(out var reference))
163     {
164         if (reference.Length > 0)
165         {

```

```

170         var formatted = LinoFormat.FormatReference(reference);
171         var document = Parse(formatted);
172         Assert.True(
173             document.Count == 1 && document[0].Equals(LinoFormat.Reference(reference)),
174             $"{reference} as {formatted}");
175     }
176     if (reference.Length < 5)
177     {
178         foreach (var character in alphabet)
179         {
180             stack.Push(reference + character);
181         }
182     }
183 }
184 }
185
186 [Fact]
187 public void CommentReferencesSurviveBinaryAndTextRoundTrips()
188 {
189     foreach (var reference in new[] { "#", "#tag", "# with space", "issue#1047" })
190     {
191         var document = new[] { LinoFormat.Reference(reference) };
192         var codec = new BinaryLinoCodec();
193         var decoded = codec.Decode(codec.Encode(document));
194         Assert.Equal(document, decoded);
195         Assert.Equal(document, Parse(LinoFormat.FormatDocument(decoded)));
196     }
197 }
198
199 [Fact]
200 public void TextHelpersPreserveUnicodeWhitespaceReferences()
201 {
202     foreach (var reference in new[] { "\u00a0", "\u2003", "\u2028" })
203     {
204         var document = new[] { LinoFormat.Reference(reference) };
205         Assert.Equal(document, Parse(reference));
206         Assert.Equal(document, Parse(LinoFormat.FormatDocument(document)));
207     }
208 }
209 [Fact]
210 public void NativeParserGroupsRoundTripWithoutCanonicalization()
211 {
212     foreach (var text in Corpus)
213     {
214         var document = new Parser().Parse(text).ToList();
215         foreach (var options in AllOptions())
216         {
217             var codec = new BinaryLinoCodec(options);
218             Assert.Equal(document, codec.Decode(codec.Encode(document)));
219         }
220     }
221 }
222
223 [Fact]
224 public void InvalidEncoderModelsAreErrors()
225 {
226     var link = LinoFormat.Reference("a");
227     for (var depth = 0; depth < DecodeLimits.Default.MaxDepth; depth++)
228     {
229         link = LinoFormat.Link(null, new List<LinoLink> { link });
230     }
231     Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
232     ↪ new BinaryLinoCodec().Encode(new[] { link })).Kind);
233     var invalid = new BinaryLinoOptions().WithArity(ArityRange.Between(0, 2));
234     Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
235     ↪ new BinaryLinoCodec(invalid).Encode(Array.Empty<LinoLink>())).Kind);
236     Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
237     ↪ new BinaryLinoCodec().Encode(new[] { LinoFormat.Reference("\ud800" )}).Kind);
238 }
239
240 [Fact]
241 public void IdsDoNotAddAnExtraModelNestingLevel()
242 {
243     var link = LinoFormat.Link("id", new List<LinoLink>());
244     for (var depth = 1; depth < DecodeLimits.Default.MaxDepth; depth++)
245     {
246         link = LinoFormat.Link(null, new List<LinoLink> { link });
247     }

```

```

245     var document = new[] { link };
246     foreach (var options in AllOptions())
247     {
248         var codec = new BinaryLinoCodec(options);
249         Assert.Equal(document, codec.Decode(codec.Encode(document)));
250     }
251 }
252
253 [Fact]
254 public void InMemoryPacketsObeyLimits()
255 {
256     var packet = LinoMapping.EncodeDocument(Parse("(a b)"));
257     foreach (var limits in new[] { new DecodeLimits { MaxLinks = 0 }, new DecodeLimits {
↪ MaxReferences = 0 } })
258     {
259         Assert.Equal(BinaryErrorKind.LimitExceeded,
↪ Assert.Throws<BinaryNotationException>(() => LinoMapping.DecodeDocument(packet,
↪ limits)).Kind);
260     }
261     var links = new List<ulong, PacketReference[]> { (6, new[] { PacketReference.Null,
↪ PacketReference.Null }) };
262     for (ulong address = 7; address < 80; address++)
263     {
264         links.Add((address, new[] { PacketReference.Internal(address - 1),
↪ PacketReference.Null }));
265     }
266     var deep = LinksPacket.Pack(false, links, false);
267     Assert.Equal(BinaryErrorKind.LimitExceeded, Assert.Throws<BinaryNotationException>(() =>
↪ LinoMapping.DecodeDocument(deep)).Kind);
268     Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
↪ LinksPacket.Pack(false, new[] { (ulong.MaxValue, new[] { PacketReference.Null }) },
↪ false)).Kind);
269 }
270
271 [Fact]
272 public void InMemoryPacketsRejectInvalidWireShapes()
273 {
274     foreach (var kind in new[] { "external", "width", "arity" })
275     {
276         var packet = LinksPacket.Pack(true, new[] { (6UL, new[] {
↪ PacketReference.External(3) }) }, false);
277         if (kind == "external") packet.ExternalReferences = false;
278         if (kind == "width") packet.Sections[0].Width = 3;
279         if (kind == "arity") packet.Sections[0].Arity = ArityRange.Doublets;
280         Assert.Equal(BinaryErrorKind.Malformed, Assert.Throws<BinaryNotationException>(() =>
↪ LinoMapping.DecodeDocument(packet)).Kind);
281     }
282 }
283
284 [Fact]
285 public void CompactAddressOverflowIsRejectedBeforeReadingLinks()
286 {
287     var bytes = new List<byte> { 0x10 };
288     LinksPacket.WriteLeb128(bytes, ulong.MaxValue);
289     var exception = Assert.Throws<BinaryNotationException>(() =>
↪ LinksPacket.FromBytes(bytes.ToArray(), DecodeLimits.Unlimited));
290     Assert.Equal(BinaryErrorKind.Malformed, exception.Kind);
291     Assert.Contains("addresses overflow", exception.Detail);
292 }
293
294 [Fact]
295 public void RepeatedReferencesObeyATotalUtf8ExpansionBudget()
296 {
297     foreach (var reference in new[] { "\U0001F600", "12" })
298     {
299         var document = new[] { LinoFormat.Reference(reference),
↪ LinoFormat.Reference(reference) };
300         foreach (var options in AllOptions())
301         {
302             var packet = LinoMapping.EncodeDocument(document, options);
303             var bytes = System.Text.Encoding.UTF8.GetByteCount(reference) * 2;
304             Assert.Equal(document, LinoMapping.DecodeDocument(packet, new DecodeLimits {
↪ MaxStringBytes = bytes }));
305             Assert.Equal(BinaryErrorKind.LimitExceeded,
↪ Assert.Throws<BinaryNotationException>(() => LinoMapping.DecodeDocument(packet, new
↪ DecodeLimits { MaxStringBytes = bytes - 1 })).Kind);
306         }

```

```

307     }
308 }
309
310 [Fact]
311 public void StringBudgetStopsBeforeLaterScalars()
312 {
313     var bytes = Convert.FromHexString("13024605011001030000009fffffff9fffffff0028ffff06");
314     var codec = new BinaryLinoCodec { Limits = new DecodeLimits { MaxStringBytes = 1 } };
315     Assert.Equal(BinaryErrorKind.LimitExceeded, Assert.Throws<BinaryNotationException>(() =>
↵ codec.Decode(bytes)).Kind);
316 }
317
318 [Fact]
319 public void StreamPacketsAreReadOneAtATimeAndTruncationIsRejected()
320 {
321     var bytes = new BinaryLinoCodec().Encode(Parse("(a b)"));
322     for (var end = 0; end < bytes.Length; end++)
323     {
324         Assert.Throws<BinaryNotationException>(() => LinksPacket.FromBytes(bytes[..end]));
325     }
326     var reader = PacketReader.FromBytes(bytes.Concat(bytes).ToArray());
327     Assert.NotNull(LinksPacket.ReadFrom(reader));
328     Assert.NotNull(LinksPacket.ReadFrom(reader));
329     Assert.Null(LinksPacket.ReadFrom(reader));
330 }
331 }

```

1.29 ./Link.Foundation.Links.Notation.Tests/CommentsTests.cs

```

1 using System.Linq;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     /// <summary>
7     /// Conformance tests for line comments.
8     ///
9     /// https://github.com/link-foundation/links-notation/issues/301
10    ///
11    /// <c>#</c> starts a comment that runs to the end of the line, unless it sits
12    /// inside a token or inside a delimited reference. Parsers read comments by
13    /// default and can be told to treat <c>#</c> as an ordinary character again.
14    /// The table below is shared with the Rust, JavaScript, Python, Go, Java and
15    /// PHP suites, so a document written by one implementation reads the same in
16    /// all of them.
17    /// </summary>
18    public static class CommentsTests
19    {
20        /// <summary>
21        /// Renders a parsed node unambiguously: every reference is wrapped in
22        /// angle brackets.
23        /// </summary>
24        private static string Render(Link<string> node)
25        {
26            if (node.Values == null || node.Values.Count == 0)
27            {
28                return "<" + (node.Id ?? "") + ">";
29            }
30            var head = node.Id == null ? "" : "<" + node.Id + ">: ";
31            return "(" + head + string.Join(" ", node.Values.Select(Render)) + ">";
32        }
33
34        private static string RenderedWith(Parser parser, string source) =>
35            string.Join("\n", parser.Parse(source).Select(Render));
36
37        private static string Rendered(string source) => RenderedWith(new Parser(), source);
38
39        private static void AssertParsesAs(string source, string expected)
40        {
41            Assert.Equal(expected, Rendered(source));
42        }
43
44        [Fact]
45        public static void ALineThatStartsWithAHashIsAComment()
46        {
47            AssertParsesAs("# a b\n", "");
48        }
49
50        [Fact]
51        public static void ACommentMayHoldAColon()

```

```

52 {
53     // The document from #301: prose with a colon used to be rejected.
54     AssertParsesAs("# a: b\n", "");
55 }
56
57 [Fact]
58 public static void ACommentMayHoldAnythingAtAll()
59 {
60     AssertParsesAs("# ) : ( \" ' ` #\n", "");
61 }
62
63 [Fact]
64 public static void ACommentEndsAtTheEndOfItsLine()
65 {
66     AssertParsesAs("# note\na: b\n", "<a>: <b>");
67 }
68
69 [Fact]
70 public static void ACommentMayFollowALink()
71 {
72     AssertParsesAs("a: b # why\n", "<a>: <b>");
73 }
74
75 [Fact]
76 public static void ACommentMayFollowAGroup()
77 {
78     AssertParsesAs("(a b) # why\n", "<a> <b>");
79 }
80
81 [Fact]
82 public static void ACommentNeedsNoClosingNewline()
83 {
84     AssertParsesAs("a: b # why", "<a>: <b>");
85 }
86
87 [Fact]
88 public static void ACommentLineInsideAnIndentedBlockIsSkipped()
89 {
90     Assert.Equal(
91         Rendered("parent\n  child\n"),
92         Rendered("parent\n  # what the child is for\n  child\n"));
93 }
94
95 [Fact]
96 public static void ACommentLineInsideAGroupIsSkipped()
97 {
98     Assert.Equal(
99         Rendered("(\\n  a\\n  b\\n)\\n"),
100         Rendered("(\\n  a\\n  # why\\n  b\\n)\\n"));
101 }
102
103 [Fact]
104 public static void ALineOfSpacesSeparatesLinksTheWayAnEmptyLineDoes()
105 {
106     // Blanking a comment leaves a line of spaces behind, so such a line
107     // has to read the way an empty line does.
108     Assert.Equal(Rendered("a\\n\\nb\\n"), Rendered("a\\n  \\nb\\n"));
109 }
110
111 [Fact]
112 public static void ADocumentOfCommentsAloneHoldsNoLinks()
113 {
114     AssertParsesAs("# one\\n# two\\n", "");
115 }
116
117 [Fact]
118 public static void AHashInsideATokenIsAnOrdinaryCharacter()
119 {
120     AssertParsesAs("issue#1047\\n", "<issue#1047>");
121 }
122
123 [Fact]
124 public static void AHashThatOpensATokenIsAnOrdinaryCharacter()
125 {
126     AssertParsesAs("a: b#c\\n", "<a>: <b#c>");
127 }
128
129 [Fact]
130 public static void AHashInsideADelimitedReferenceIsContent()

```

```

131 {
132     AssertParsesAs("\"# not a comment\" a\n", "(<# not a comment> <a>)");
133 }
134
135 [Fact]
136 public static void ACommentMayFollowADelimitedReference()
137 {
138     AssertParsesAs("\"a\" # why\n", "(<a>)");
139 }
140
141 [Fact]
142 public static void AHashInsideAMultiLineDelimitedReferenceIsContent()
143 {
144     AssertParsesAs("\"a # b\nc\" d\n", "(<a # b\nc> <d>)");
145 }
146
147 [Fact]
148 public static void CommentsAreOnByDefault()
149 {
150     Assert.True(new Parser().Comments);
151     AssertParsesAs("# a b\n", "");
152 }
153
154 [Fact]
155 public static void AParserWithoutCommentsKeepsTheHash()
156 {
157     Assert.Equal("(<#> <a> <b>)", RenderedWith(new Parser(comments: false), "# a b\n"));
158 }
159
160 [Fact]
161 public static void BlankingACommentKeepsTheLengthOfTheDocument()
162 {
163     Assert.Equal("a: b      \n", Comments.StripComments("a: b # why\n"));
164     Assert.Equal("\"# kept\" \n", Comments.StripComments("\"# kept\" \n"));
165     Assert.Equal("issue#1047\n", Comments.StripComments("issue#1047\n"));
166     Assert.Equal("      \n", Comments.StripComments("# why\n"));
167 }
168
169 [Fact]
170 public static void ACommentDoesNotMoveThePositionALaterErrorIsReportedAt()
171 {
172     var error = Assert.Throws<ParseException>(
173         () => new Parser().Parse("# a comment\nstage: rust: nextest\n"));
174
175     Assert.Equal(2, error.Line);
176     Assert.Equal(12, error.Column);
177 }
178
179 [Fact]
180 public static void AReferenceThatBeginsWithAHashIsWrittenQuoted()
181 {
182     // Without the quotes the document would read as `a` followed by a comment.
183     var document = new Link<string>(new Link<string>("a"), new
↪ Link<string>("#tag")).ToString();
184
185     Assert.Equal("(a '#tag')", document);
186     Assert.Equal("<a> <#tag>", Rendered(document));
187 }
188
189 [Fact]
190 public static void AHashThatCannotOpenACommentIsLeftUnquoted()
191 {
192     Assert.Equal("issue#1047", Link<string>.EscapeReference("issue#1047"));
193     Assert.Equal("#", Link<string>.EscapeReference("#"));
194     Assert.Equal("#ff0000", Link<string>.EscapeReference("#ff0000"));
195 }
196 }
197 }

```

1.30 ./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs

```

1 using System;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     public static class EdgeCaseParserTests
7     {
8         [Fact]
9         public static void EmptyLinkTest()

```

```

10 {
11     var source = @": ";
12     var parser = new Parser();
13     // Standalone ':' is now forbidden and should throw an exception
14     Assert.Throws<ParseException>(() => parser.Parse(source));
15 }
16
17 [Fact]
18 public static void EmptyLinkWithParenthesesTest()
19 {
20     var source = @"()";
21     var target = @"()";
22     var parser = new Parser();
23     var links = parser.Parse(source);
24     var formattedLinks = links.Format();
25     Assert.Equal(target, formattedLinks);
26 }
27
28 [Fact]
29 public static void EmptyLinkWithEmptySelfReferenceTest()
30 {
31     var source = @"(:)";
32     var parser = new Parser();
33     // '(:)' is now forbidden and should throw an exception
34     Assert.Throws<ParseException>(() => parser.Parse(source));
35 }
36
37 [Fact]
38 public static void AllFeaturesTest()
39 {
40     // Test single-line link with id
41     var input = "id: value1 value2";
42     var result = new Parser().Parse(input);
43     Assert.NotEmpty(result);
44
45     // Test multi-line link with id
46     input = "(id: value1 value2)";
47     result = new Parser().Parse(input);
48     Assert.NotEmpty(result);
49
50     // Test link without id (single-line) - now forbidden
51     input = ": value1 value2";
52     Assert.Throws<ParseException>(() => new Parser().Parse(input));
53
54     // Test link without id (multi-line) - now forbidden
55     input = "(: value1 value2)";
56     Assert.Throws<ParseException>(() => new Parser().Parse(input));
57
58     // Test singlet link
59     input = "(singlet)";
60     result = new Parser().Parse(input);
61     Assert.Single(result);
62     Assert.Null(result[0].Id);
63     Assert.NotNull(result[0].Values);
64     Assert.Single(result[0].Values!);
65     Assert.Equal("singlet", result[0].Values?[0].Id);
66     Assert.Null(result[0].Values?[0].Values);
67
68     // Test value link
69     input = "(value1 value2 value3)";
70     result = new Parser().Parse(input);
71     Assert.NotEmpty(result);
72
73     // Test quoted references
74     input = @"("id with spaces": "value with spaces")";
75     result = new Parser().Parse(input);
76     Assert.NotEmpty(result);
77
78     // Test single-quoted references
79     input = "('id': 'value')";
80     result = new Parser().Parse(input);
81     Assert.NotEmpty(result);
82
83     // Test nested links
84     input = "(outer: (inner: value))";
85     result = new Parser().Parse(input);
86     Assert.NotEmpty(result);
87 }
88

```

```

89 [Fact]
90 public static void TestSingletLinks()
91 {
92     // Test singlet (1)
93     var input = "(1)";
94     var result = new Parser().Parse(input);
95     Assert.Single(result);
96     Assert.Null(result[0].Id);
97     Assert.NotNull(result[0].Values);
98     Assert.Single(result[0].Values!);
99     Assert.Equal("1", result[0].Values?[0].Id);
100    Assert.Null(result[0].Values?[0].Values);
101
102    // Test (1 2)
103    input = "(1 2)";
104    result = new Parser().Parse(input);
105    Assert.Single(result);
106    Assert.Null(result[0].Id);
107    Assert.NotNull(result[0].Values);
108    Assert.Equal(2, result[0].Values?.Count);
109    Assert.Equal("1", result[0].Values?[0].Id);
110    Assert.Null(result[0].Values?[0].Values);
111    Assert.Equal("2", result[0].Values?[1].Id);
112    Assert.Null(result[0].Values?[1].Values);
113
114    // Test (1 2 3)
115    input = "(1 2 3)";
116    result = new Parser().Parse(input);
117    Assert.Single(result);
118    Assert.Null(result[0].Id);
119    Assert.NotNull(result[0].Values);
120    Assert.Equal(3, result[0].Values?.Count);
121    Assert.Equal("1", result[0].Values?[0].Id);
122    Assert.Null(result[0].Values?[0].Values);
123    Assert.Equal("2", result[0].Values?[1].Id);
124    Assert.Null(result[0].Values?[1].Values);
125    Assert.Equal("3", result[0].Values?[2].Id);
126    Assert.Null(result[0].Values?[2].Values);
127
128    // Test (1 2 3 4)
129    input = "(1 2 3 4)";
130    result = new Parser().Parse(input);
131    Assert.Single(result);
132    Assert.Null(result[0].Id);
133    Assert.NotNull(result[0].Values);
134    Assert.Equal(4, result[0].Values?.Count);
135    Assert.Equal("1", result[0].Values?[0].Id);
136    Assert.Null(result[0].Values?[0].Values);
137    Assert.Equal("2", result[0].Values?[1].Id);
138    Assert.Null(result[0].Values?[1].Values);
139    Assert.Equal("3", result[0].Values?[2].Id);
140    Assert.Null(result[0].Values?[2].Values);
141    Assert.Equal("4", result[0].Values?[3].Id);
142    Assert.Null(result[0].Values?[3].Values);
143 }
144
145 [Fact]
146 public static void EmptyDocumentTest()
147 {
148     var input = "";
149     // Empty document should return empty list
150     var result = new Parser().Parse(input);
151     Assert.Empty(result);
152 }
153
154 [Fact]
155 public static void WhitespaceOnlyTest()
156 {
157     var input = " \n \n ";
158     // Whitespace-only document should return empty list (similar to empty document)
159     var result = new Parser().Parse(input);
160     Assert.Empty(result);
161 }
162
163 [Fact]
164 public static void EmptyLinksTest()
165 {
166     var input = "()";

```

```

167     var result = new Parser().Parse(input);
168     Assert.NotEmpty(result);
169
170     // '(:)' is now forbidden
171     input = "(:)";
172     Assert.Throws<ParseException>(() => new Parser().Parse(input));
173
174     input = "(id:)";
175     result = new Parser().Parse(input);
176     Assert.NotEmpty(result);
177 }
178
179 [Fact]
180 public static void InvalidInputTest()
181 {
182     var input = "(invalid";
183     // Unclosed parentheses should throw an exception
184     Assert.Throws<ParseException>(() => new Parser().Parse(input));
185 }
186 }
187 }

```

1.31 ./Link.Foundation.Links.Notation.Tests/EmptyReferenceTests.cs

```

1 using System.Collections.Generic;
2 using System.Linq;
3 using Xunit;
4
5 namespace Link.Foundation.Links.Notation.Tests
6 {
7     /// <summary>
8     /// Conformance tests for the empty reference.
9     ///
10    /// https://github.com/link-foundation/links-notation/issues/288
11    ///
12    /// A bare delimiter pair is the empty reference. The three delimiters
13    /// <c>"</c>, <c>'</c> and <c>`</c> behave identically, and every longer
14    /// n-quote run keeps the meaning it already had. The table below is shared
15    /// with the Rust, JavaScript, Python, Go, Java and PHP suites, so a document
16    /// written by one implementation reads the same in all of them.
17    /// </summary>
18    public static class EmptyReferenceTests
19    {
20        /// <summary>
21        /// Renders a parsed node unambiguously: every reference is wrapped in
22        /// angle brackets so an empty one is visible as < >.
23        /// </summary>
24        private static string Render(Link<string> node)
25        {
26            if (node.Values == null || node.Values.Count == 0)
27            {
28                return "<" + (node.Id ?? "") + ">";
29            }
30            var head = node.Id == null ? "" : "<" + node.Id + "> ";
31            return "(" + head + string.Join(" ", node.Values.Select(Render)) + ")";
32        }
33
34        private static string Rendered(string source)
35        {
36            var links = new Parser().Parse(source);
37            return string.Join("\n", links.Select(Render));
38        }
39
40        private static void AssertParsesAs(string source, string expected)
41        {
42            Assert.Equal(expected, Rendered(source));
43        }
44
45        [Fact]
46        public static void BareDelimiterPairIsTheEmptyReference()
47        {
48            AssertParsesAs("(a \" b)", "<a> <> <b>");
49        }
50
51        [Fact]
52        public static void EveryDelimiterStyleYieldsTheSameEmptyReference()
53        {
54            AssertParsesAs("(a \" b)", "<a> <> <b>");
55            AssertParsesAs("(a ' b)", "<a> <> <b>");
56            AssertParsesAs("(a ` b)", "<a> <> <b>");
57        }
58    }
59 }

```

```

57     }
58
59     [Fact]
60     public static void AdjacentEmptyReferencesStaySeparate()
61     {
62         AssertParsesAs("(a \"\" \"\" b)", "<a> <> <b>");
63         AssertParsesAs("(a ' ' b)", "<a> <> <b>");
64         AssertParsesAs("(a ` ` b)", "<a> <> <b>");
65         AssertParsesAs("(a \"\" \"\" b)", "<a> <> <b>");
66     }
67
68     [Fact]
69     public static void NestedEmptyReferencesParse()
70     {
71         AssertParsesAs("(\"\" (\"\" 1))", "<> (<> <1>)");
72         AssertParsesAs("(\"\" (' 1))", "<> (<> <1>)");
73         AssertParsesAs("(\"\" (\"x\" 1))", "<x> (<> <1>)");
74         AssertParsesAs("(\"\" (\"x\" 1))", "<> (<x> <1>)");
75         AssertParsesAs("(\"\" x (\"\" 1))", "<> <x> (<> <1>)");
76         AssertParsesAs("(\"\" 1 (\"\" 1))", "<> <1> (<> <1>)");
77     }
78
79     [Fact]
80     public static void EmptyReferenceIsValidAsAnId()
81     {
82         AssertParsesAs("(\"\": 1)", "<>: <1>");
83         AssertParsesAs("(o: (\"\" (o: (\"\" 1))))", "<o>: (<> (<o>: (<> <1>)))");
84     }
85
86     [Fact]
87     public static void NQuoteDelimitedBodiesAreUnchanged()
88     {
89         // A run that encloses a substantive body keeps its n-quote meaning.
90         AssertParsesAs("(a \"\"x\"\" b)", "<a> <x> <b>");
91         AssertParsesAs("(x \"\" \"\" \"\")", "<x> < \"\" >");
92         AssertParsesAs("(x ' \" ')", "<x> < \"\" >");
93         // An n-quote-delimited empty is still empty.
94         AssertParsesAs("(a \"\" \"\" b)", "<a> <> <b>");
95     }
96
97     [Fact]
98     public static void ASingleSpaceStillReadsAsASpace()
99     {
100         AssertParsesAs("(a \" \" b)", "<a> < > <b>");
101     }
102
103     [Fact]
104     public static void EmptyReferenceSurvivesARoundTrip()
105     {
106         var sources = new[]
107         {
108             "(a \"\" b)",
109             "(a \"\" \"\" b)",
110             "(\"\" (\"\" 1))",
111             "(\"\": 1)",
112             "(o: (\"\" (o: (\"\" 1))))",
113         };
114         foreach (var source in sources)
115         {
116             var formatted = ((IList<Link<string>>)new Parser().Parse(source)).Format();
117             var reparsed = ((IList<Link<string>>)new Parser().Parse(formatted)).Format();
118             Assert.Equal(formatted, reparsed);
119         }
120     }
121
122     [Fact]
123     public static void EmptyReferenceIsWrittenAsAVersionedLiteral()
124     {
125         var links = (IList<Link<string>>)new Parser().Parse("(a \"\" b)");
126         Assert.Equal("(a ~1{ } b)", links.Format());
127     }
128 }
129 }

```

1.32 ./Link.Foundation.Links.Notation.Tests/FormatConfigTests.cs

```

1 using Xunit;
2
3 namespace Link.Foundation.Links.Notation.Tests
4 {

```

```

5 public class FormatConfigTests
6 {
7     [Fact]
8     public void FormatConfigBasicTest()
9     {
10         var config = new FormatConfig();
11         Assert.False(config.LessParentheses);
12         Assert.Equal(80, config.MaxLineLength);
13         Assert.False(config.IndentLongLines);
14     }
15
16     [Fact]
17     public void FormatWithLineLengthLimitTest()
18     {
19         // Create a config with line length limit
20         // The line would be 32+ chars, so use threshold of 30
21         var config = new FormatConfig
22         {
23             IndentLongLines = true,
24             MaxLineLength = 30,
25             PreferInline = false
26         };
27
28         // Verify config is set correctly
29         Assert.True(config.IndentLongLines);
30         Assert.Equal(30, config.MaxLineLength);
31         // Note: Full formatting integration would test actual output here
32     }
33
34     [Fact]
35     public void FormatWithMaxInlineRefsTest()
36     {
37         // Create a config with MaxInlineRefs=3 (should trigger indentation with 4 refs)
38         var config = new FormatConfig
39         {
40             MaxInlineRefs = 3,
41             PreferInline = false
42         };
43
44         // Verify config is set correctly
45         Assert.Equal(3, config.MaxInlineRefs);
46         Assert.True(config.ShouldIndentByRefCount(4));
47         // Note: Full formatting integration would test actual output here
48     }
49
50     [Fact]
51     public void FormatWithConsecutiveGroupingTest()
52     {
53         // Create a config with consecutive grouping enabled
54         var config = new FormatConfig
55         {
56             GroupConsecutive = true
57         };
58
59         // Verify config is set correctly
60         Assert.True(config.GroupConsecutive);
61         // Note: Full formatting integration would test grouping behavior here
62     }
63
64     [Fact]
65     public void FormatConfigCustomIndentTest()
66     {
67         var config = new FormatConfig
68         {
69             MaxInlineRefs = 3,
70             PreferInline = false,
71             IndentString = "    " // 4 spaces
72         };
73
74         Assert.Equal("    ", config.IndentString);
75     }
76
77     [Fact]
78     public void FormatConfigLessParenthesesTest()
79     {
80         var config = new FormatConfig
81         {
82             LessParentheses = true
83         };
84

```

```

85     - Assert.True(config.LessParentheses);
86 }
87
88 [Fact] -
89 public void RoundtripWithLineLengthFormattingTest()
90 {
91     - //-This test would require full format integration
92     // For now, just verify config creation
93     var config = new FormatConfig
94     {
95         MaxInlineRefs = 2,
96         PreferInline = false
97     };
98
99     Assert.Equal(2, config.MaxInlineRefs);
100 }
101
102 [Fact]
103 public void ShouldIndentByLengthTest()
104 {
105     var config = new FormatConfig
106     {
107         IndentLongLines = true,
108         MaxLineLength = 80
109     };
110
111     Assert.False(config.ShouldIndentByLength("short"));
112     Assert.True(config.ShouldIndentByLength(new string('a', 100)));
113 }
114
115 [Fact]
116 public void ShouldIndentByRefCountTest()
117 {
118     var config = new FormatConfig
119     {
120         MaxInlineRefs = 3
121     };
122
123     Assert.False(config.ShouldIndentByRefCount(2));
124     Assert.False(config.ShouldIndentByRefCount(3));
125     Assert.True(config.ShouldIndentByRefCount(4));
126 }
127 }
128 }

```

1.33 ./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs

```

1  using Xunit;
2  using System.Collections.Generic;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public class IndentationConsistencyTests
7      {
8          [Fact]
9          public void LeadingSpacesVsNoLeadingSpaces()
10         {
11             // Example with 2 leading spaces (from issue #135)
12             var withLeading = @" TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
13 TELEGRAM ALLOWED CHATS:
14     -1002975819706
15     -1002861722681
16 TELEGRAM HIVE OVERRIDES:
17     --all-issues
18     --once
19 TELEGRAM BOT VERBOSE: true";
20
21             // Example without leading spaces (from issue #135)
22             var withoutLeading = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
23 TELEGRAM ALLOWED CHATS:
24     -1002975819706
25     -1002861722681
26 TELEGRAM HIVE OVERRIDES:
27     --all-issues
28     --once
29 TELEGRAM BOT VERBOSE: true";
30
31             var resultWith = new Parser().Parse(withLeading);
32             var resultWithout = new Parser().Parse(withoutLeading);
33
34             // Compare the entire formatted output (complete round trip test)
35             Assert.Equal(resultWithout.Format(), resultWith.Format());

```

```

36     }
37
38     [Fact]
39     public void TwoSpacesVsFourSpacesIndentation()
40     {
41         // Example with 2 spaces per level
42         var twoSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
43 TELEGRAM ALLOWED CHATS:
44     -1002975819706
45     -1002861722681
46 TELEGRAM HIVE OVERRIDES:
47     --all-issues
48     --once
49     --auto-fork
50     --skip-issues-with-prs
51     --attach-logs
52     --verbose
53     --no-tool-check
54 TELEGRAM SOLVE OVERRIDES:
55     --auto-fork
56     --auto-continue
57     --attach-logs
58     --verbose
59     --no-tool-check
60 TELEGRAM BOT VERBOSE: true";
61
62         // Example with 4 spaces per level
63         var fourSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
64 TELEGRAM ALLOWED CHATS:
65     -1002975819706
66     -1002861722681
67 TELEGRAM HIVE OVERRIDES:
68     --all-issues
69     --once
70     --auto-fork
71     --skip-issues-with-prs
72     --attach-logs
73     --verbose
74     --no-tool-check
75 TELEGRAM SOLVE OVERRIDES:
76     --auto-fork
77     --auto-continue
78     --attach-logs
79     --verbose
80     --no-tool-check
81 TELEGRAM BOT VERBOSE: true";
82
83         var resultTwo = new Parser().Parse(twoSpaces);
84         var resultFour = new Parser().Parse(fourSpaces);
85
86         // Compare the entire formatted output (complete round trip test)
87         Assert.Equal(resultFour.Format(), resultTwo.Format());
88     }
89
90     [Fact]
91     public void SimpleTwoVsFourSpacesIndentation()
92     {
93         // Simple example with 2 spaces
94         var twoSpaces = @"parent:
95 child1
96 child2";
97
98         // Simple example with 4 spaces
99         var fourSpaces = @"parent:
100 child1
101 child2";
102
103         var resultTwo = new Parser().Parse(twoSpaces);
104         var resultFour = new Parser().Parse(fourSpaces);
105
106         // Compare the entire formatted output (complete round trip test)
107         Assert.Equal(resultFour.Format(), resultTwo.Format());
108     }
109
110     [Fact]
111     public void ThreeLevelNestingWithDifferentIndentation()
112     {
113         // Three levels with 2 spaces
114         var twoSpaces = @"level1:
115     level2:
116         level3a
117         level3b
118     level2b";

```

```

119
120         // Three levels with 4 spaces
121         var fourSpaces = @"level1:
122     level2:
123         level3a
124         level3b
125     level2b";
126
127         var resultTwo = new Parser().Parse(twoSpaces);
128         var resultFour = new Parser().Parse(fourSpaces);
129
130         // Compare the entire formatted output (complete round trip test)
131         Assert.Equal(resultFour.Format(), resultTwo.Format());
132     }
133 }
134 }

```

1.34 ./Link.Foundation.Links.Notation.Tests/IndentedIdNestedValuesTests.cs

```

1  using Xunit;
2
3  namespace Link.Foundation.Links.Notation.Tests
4  {
5      public class IndentedIdNestedValuesTests
6      {
7          [Fact]
8          public void IndentedIdRetainsNestedValues()
9          {
10             var cases = new[]
11             {
12                 new[] { "outer:\n inner:\n value1\n value2\n value3", "(outer: (inner:
↵ value1 value2) value3)" },
13                 new[] { "statement:\n subject:\n I\n verb:\n love\n object:\n you:\n
↵ very\n much", "(statement: (subject: I) (verb: love) (object: (you: very much)))"
↵ },
14                 new[] { "document:\n (metadata: title author date)\n content:\n
↵ paragraph1\n (paragraph2: text (with: nested structure))\n paragraph3", "(document:
↵ (metadata: title author date) (content: paragraph1 (paragraph2: text (with: nested
↵ structure)) paragraph3))" },
15                 new[] { "image:\n file: .gitpod.Dockerfile\n context: ./docker-content",
↵ "(image: (file: .gitpod.Dockerfile) (context: ./docker-content))" },
16                 new[] { "level1:\n level2:\n level3a\n level3b\n level2b", "(level1:
↵ (level2: level3a level3b) level2b)" },
17                 new[] { "root:\n child1\n child2\n grandchild", "(root: child1 (child2
↵ grandchild))" },
18             };
19
20             var parser = new Parser();
21             foreach (var testCase in cases)
22             {
23                 Assert.Equal(testCase[1], parser.Parse(testCase[0]).Format());
24             }
25         }
26     }
27 }

```

1.35 ./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class IndentedIdSyntaxTests
7      {
8          [Fact]
9          public static void BasicIndentedIdSyntaxTest()
10          {
11             var indentedSyntax = @"3:
12     papa
13     loves
14     mama";
15             var inlineSyntax = "(3: papa loves mama)";
16
17             var parser = new Parser();
18             var indentedResult = parser.Parse(indentedSyntax);
19             var inlineResult = parser.Parse(inlineSyntax);
20
21             var indentedFormatted = indentedResult.Format();
22             var inlineFormatted = inlineResult.Format();
23

```

```

24         Assert.Equal(inlineFormatted, indentedFormatted);
25         Assert.Equal("(3: papa loves mama)", indentedFormatted);
26     }
27
28     [Fact]
29     public static void IndentedIdSyntaxWithSingleValueTest()
30     {
31         var input = @"greeting:
32 hello";
33
34         var parser = new Parser();
35         var result = parser.Parse(input);
36         var formatted = result.Format();
37
38         Assert.Equal("(greeting: hello)", formatted);
39         Assert.Single(result);
40         Assert.Equal("greeting", result[0].Id);
41         Assert.Single(result[0].Values!);
42         Assert.Equal("hello", result[0].Values![0].Id);
43     }
44
45     [Fact]
46     public static void IndentedIdSyntaxWithMultipleValuesTest()
47     {
48         var input = @"action:
49 run
50 fast
51 now";
52
53         var parser = new Parser();
54         var result = parser.Parse(input);
55         var formatted = result.Format();
56
57         Assert.Equal("(action: run fast now)", formatted);
58         Assert.Single(result);
59         Assert.Equal("action", result[0].Id);
60         Assert.Equal(3, result[0].Values!.Count);
61     }
62
63     [Fact]
64     public static void IndentedIdSyntaxWithNumericIdTest()
65     {
66         var input = @"42:
67 answer
68 to
69 everything";
70
71         var parser = new Parser();
72         var result = parser.Parse(input);
73         var formatted = result.Format();
74
75         Assert.Equal("(42: answer to everything)", formatted);
76     }
77
78     [Fact]
79     public static void IndentedIdSyntaxWithQuotedIdTest()
80     {
81         var input = @"""complex id":
82 value1
83 value2";
84
85         var parser = new Parser();
86         var result = parser.Parse(input);
87         var formatted = result.Format();
88
89         Assert.Equal("('complex id': value1 value2)", formatted);
90     }
91
92     [Fact]
93     public static void MultipleIndentedIdLinksTest()
94     {
95         var input = @"first:
96 a
97 b
98 second:
99 c
100 d";
101
102         var parser = new Parser();
103         var result = parser.Parse(input);

```

```

104     var formatted = result.Format();
105
106     Assert.Equal(2, result.Count);
107     Assert.Contains("(first: a b)", formatted);
108     Assert.Contains("(second: c d)", formatted);
109 }
110
111 [Fact]
112 public static void MixedIndentedAndRegularSyntaxTest()
113 {
114     var input = @"first:
115         a
116         b
117     (second: c d)
118     third value";
119
120     var parser = new Parser();
121     var result = parser.Parse(input);
122
123     Assert.Equal(3, result.Count);
124
125     var formatted = result.Format();
126     Assert.Contains("(first: a b)", formatted);
127     Assert.Contains("(second: c d)", formatted);
128     Assert.Contains("third value", formatted);
129 }
130
131 [Fact]
132 public static void UnsupportedColonOnlySyntaxShouldFailTest()
133 {
134     var input = @"
135     papa
136     loves
137     mama";
138
139     var parser = new Parser();
140     Assert.Throws<ParseException>(() => parser.Parse(input));
141 }
142
143 [Fact]
144 public static void EmptyIndentedIdShouldWorkTest()
145 {
146     var input = "empty:";
147
148     var parser = new Parser();
149     var result = parser.Parse(input);
150
151     Assert.Single(result);
152     Assert.Equal("empty", result[0].Id);
153     var emptyValues = result[0].Values;
154     Assert.True(emptyValues == null || emptyValues.Count == 0);
155
156     var formatted = result.Format();
157     Assert.Equal("(empty)", formatted);
158 }
159
160 [Fact]
161 public static void EquivalenceTestComprehensiveTest()
162 {
163     var testCases = new[]
164     {
165         new { Indented = "test:\n one", Inline = "(test: one)" },
166         new { Indented = "x:\n a\n b\n c", Inline = "(x: a b c)" },
167         new { Indented = "\"quoted\":\n value", Inline = "\"quoted\": value" }
168     };
169
170     var parser = new Parser();
171
172     foreach (var testCase in testCases)
173     {
174         var indentedResult = parser.Parse(testCase.Indented);
175         var inlineResult = parser.Parse(testCase.Inline);
176
177         var indentedFormatted = indentedResult.Format();
178         var inlineFormatted = inlineResult.Format();
179
180         Assert.Equal(inlineFormatted, indentedFormatted);
181     }
182 }
183

```

```

184     [Fact]
185     public static void IndentedIdWithDeeperNestingTest()
186     {
187         var input = @"root:
188 child1
189 child2
190   grandchild";
191
192         var parser = new Parser();
193         var result = parser.Parse(input);
194
195         Assert.NotEmpty(result);
196
197         var rootLink = result[0];
198         Assert.Equal("root", rootLink.Id);
199         Assert.Equal(2, rootLink.Values!.Count);
200         Assert.Equal("(root: child1 (child2 grandchild))", result.Format());
201     }
202 }
203

```

1.36 ./Link.Foundation.Links.Notation.Tests/LinkTests.cs

```

1  using System;
2  using Xunit;
3  using System.Collections.Generic;
4
5  namespace Link.Foundation.Links.Notation.Tests
6  {
7      public static class LinkTests
8      {
9          [Fact]
10         public static void LinkConstructorWithIdOnlyTest()
11         {
12             var link = new Link<string>("test", null);
13             Assert.Equal("test", link.Id);
14             Assert.Null(link.Values);
15         }
16
17         [Fact]
18         public static void LinkConstructorWithIdAndValuesTest()
19         {
20             var values = new List<Link<string>> { new Link<string>("value1"), new
↪ Link<string>("value2") };
21             var link = new Link<string>("parent", values);
22             Assert.Equal("parent", link.Id);
23             Assert.Equal(2, link.Values?.Count);
24         }
25
26         [Fact]
27         public static void LinkToStringWithIdOnlyTest()
28         {
29             var link = new Link<string>("test");
30             Assert.Equal("(test)", link.ToString());
31         }
32
33         [Fact]
34         public static void LinkToStringWithValuesOnlyTest()
35         {
36             var values = new List<Link<string>> { new Link<string>("value1"), new
↪ Link<string>("value2") };
37             var link = new Link<string>(null, values);
38             Assert.Equal("(value1 value2)", link.ToString());
39         }
40
41         [Fact]
42         public static void LinkToStringWithIdAndValuesTest()
43         {
44             var values = new List<Link<string>> { new Link<string>("child1"), new
↪ Link<string>("child2") };
45             var link = new Link<string>("parent", values);
46             Assert.Equal("(parent: child1 child2)", link.ToString());
47         }
48
49         [Fact]
50         public static void LinkEqualsTest()
51         {
52             var link1 = new Link<string>("test");
53             var link2 = new Link<string>("test");
54             var link3 = new Link<string>("other");

```

```

55     Assert.Equal(link1, link2);
56     Assert.NotEqual(link1, link3);
57 }
58
59 [Fact]
60 public static void LinkCombineTest()
61 {
62     var link1 = new Link<string>("first");
63     var link2 = new Link<string>("second");
64     var combined = link1.Combine(link2);
65
66     Assert.Null(combined.Id);
67     Assert.Equal(2, combined.Values?.Count);
68     Assert.Equal("first", combined.Values?[0].Id);
69     Assert.Equal("second", combined.Values?[1].Id);
70 }
71
72 [Fact]
73 public static void LinkEscapeReferenceSimpleTest()
74 {
75     Assert.Equal("test", Link<string>.EscapeReference("test"));
76     Assert.Equal("test123", Link<string>.EscapeReference("test123"));
77 }
78
79 [Fact]
80 public static void LinkEscapeReferenceWithSpecialCharactersTest()
81 {
82     Assert.Equal("'has space'", Link<string>.EscapeReference("has space"));
83     Assert.Equal("'has:colon'", Link<string>.EscapeReference("has:colon"));
84     Assert.Equal("'has(paren)'", Link<string>.EscapeReference("has(paren)"));
85     Assert.Equal("'has)paren'", Link<string>.EscapeReference("has)paren"));
86     Assert.Equal("\"has'quote\"", Link<string>.EscapeReference("has'quote"));
87     Assert.Equal("'has\\quote'", Link<string>.EscapeReference("has\\quote"));
88 }
89
90 [Fact]
91 public static void LinkSimplifyTest()
92 {
93     // Test simplify with empty values
94     var link1 = new Link<string>("test", null);
95     var simplified1 = link1.Simplify();
96     Assert.Equal(link1, simplified1);
97
98     // Test simplify with single value
99     var link2 = new Link<string>(null, new List<Link<string>> { new
100     ↪ Link<string>("single", null) });
101     var simplified2 = link2.Simplify();
102     Assert.Equal("single", simplified2.Id);
103     Assert.Null(simplified2.Values);
104 }
105 }
106 }

```

1.37 ./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs

```

1  using System;
2  using System.Collections.Generic;
3  using Xunit;
4
5  namespace Link.Foundation.Links.Notation.Tests
6  {
7      public static class LinksGroupTests
8      {
9          [Fact]
10         public static void LinksGroupConstructorTest()
11         {
12             var element = new Link<string>("root", null);
13             var groups = new List<LinksGroup<string>>
14             {
15                 new LinksGroup<string>(new Link<string>("child1", null)),
16                 new LinksGroup<string>(new Link<string>("child2", null))
17             };
18             var group = new LinksGroup<string>(element, groups);
19
20             Assert.Equal(element, group.Link);
21             Assert.Equal(groups, group.Groups);
22         }
23
24         [Fact]

```

```

25 public static void LinksGroupToListFlattensStructureTest()
26 {
27     var root = new Link<string>("root", null);
28     var child1 = new Link<string>("child1", null);
29     var child2 = new Link<string>("child2", null);
30     var grandchild = new Link<string>("grandchild", null);
31
32     var childGroup = new LinksGroup<string>(child2, new List<LinksGroup<string>>
33     {
34         new LinksGroup<string>(grandchild)
35     });
36
37     var group = new LinksGroup<string>(root, new List<LinksGroup<string>>
38     {
39         new LinksGroup<string>(child1),
40         childGroup
41     });
42
43     var list = group.ToLinksList();
44     // The C# implementation creates a hierarchical structure
45     // root, root.Combine(child1), root.Combine(child2),
    ↪ root.Combine(child2).Combine(grandchild)
46     Assert.Equal(4, list.Count);
47     Assert.Equal(root, list[0]);
48 }
49
50 [Fact]
51 public static void LinksGroupAppendToLinksListTest()
52 {
53     var element = new Link<string>("root", null);
54     var children = new List<LinksGroup<string>>
55     {
56         new LinksGroup<string>(new Link<string>("child1", null)),
57         new LinksGroup<string>(new Link<string>("child2", null))
58     };
59     var group = new LinksGroup<string>(element, children);
60
61     var list = new List<Link<string>>();
62     group.AppendToLinksList(list);
63
64     Assert.Equal(3, list.Count);
65     Assert.Equal(element, list[0]);
66 }
67
68 [Fact]
69 public static void LinksGroupToStringTest()
70 {
71     var element = new Link<string>("root", null);
72     var children = new List<LinksGroup<string>>
73     {
74         new LinksGroup<string>(new Link<string>("child1", null)),
75         new LinksGroup<string>(new Link<string>("child2", null))
76     };
77     var group = new LinksGroup<string>(element, children);
78
79     var str = group.ToString();
80     Assert.Contains("root", str);
81     Assert.Contains("child1", str);
82     Assert.Contains("child2", str);
83 }
84
85 [Fact]
86 public static void LinksGroupConstructorEquivalentTest()
87 {
88     // Test creating a LinksGroup structure in an equivalent way
89     var root = new Link<string>("root", null);
90     var children = new List<Link<string>>
91     {
92         new Link<string>("child1", null),
93         new Link<string>("child2", null)
94     };
95
96     // Create a group with an id
97     var groupElement = new Link<string>("group", null);
98     var childGroups = new List<LinksGroup<string>>
99     {
100         new LinksGroup<string>(root),
101         new LinksGroup<string>(children[0]),
102         new LinksGroup<string>(children[1])

```

```

103         };
104         var group = new LinksGroup<string>(groupElement, childGroups);
105
106         Assert.Equal("group", group.Link.Id);
107         Assert.Equal(3, group.Groups!.Count);
108         Assert.Equal(root, group.Groups[0].Link);
109     }
110 }
111 }

```

1.38 ./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class MixedIndentationModesTests
7      {
8          [Fact]
9          public static void HeroExampleMixedModesTest()
10         {
11             var input = @"empInfo
12 employees:
13 (
14     name (James Kirk)
15     age 40
16 )
17 (
18     name (Jean-Luc Picard)
19     age 45
20 )
21 (
22     name (Wesley Crusher)
23     age 27
24 )";
25
26             var parser = new Parser();
27             var result = parser.Parse(input);
28
29             Assert.NotEmpty(result);
30             var formatted = result.Format();
31             Assert.Contains("empInfo", formatted);
32             Assert.Contains("employees", formatted);
33             Assert.Contains("James Kirk", formatted);
34             Assert.Contains("Jean-Luc Picard", formatted);
35             Assert.Contains("Wesley Crusher", formatted);
36         }
37
38         [Fact]
39         public static void HeroExampleAlternativeFormatTest()
40         {
41             var input = @"empInfo
42 (
43 employees:
44 (
45     name (James Kirk)
46     age 40
47 )
48 (
49     name (Jean-Luc Picard)
50     age 45
51 )
52 (
53     name (Wesley Crusher)
54     age 27
55 )
56 )";
57
58             var parser = new Parser();
59             var result = parser.Parse(input);
60
61             Assert.NotEmpty(result);
62             var formatted = result.Format();
63             Assert.Contains("empInfo", formatted);
64             Assert.Contains("employees:", formatted);
65             Assert.Contains("James Kirk", formatted);
66             Assert.Contains("Jean-Luc Picard", formatted);
67             Assert.Contains("Wesley Crusher", formatted);
68         }
69     }

```

```

70     [Fact]
71     public static void HeroExampleEquivalenceTest()
72     {
73         var version1 = @"empInfo
74 employees:
75 (
76     name (James Kirk)
77     age 40
78 )
79 (
80     name (Jean-Luc Picard)
81     age 45
82 )
83 (
84     name (Wesley Crusher)
85     age 27
86 )";
87
88         var version2 = @"empInfo
89 (
90     employees:
91     (
92         name (James Kirk)
93         age 40
94     )
95     (
96         name (Jean-Luc Picard)
97         age 45
98     )
99     (
100        name (Wesley Crusher)
101        age 27
102    )
103 )";
104
105         var parser = new Parser();
106         var result1 = parser.Parse(version1);
107         var result2 = parser.Parse(version2);
108
109         Assert.NotEmpty(result1);
110         Assert.NotEmpty(result2);
111
112         var formatted1 = result1.Format();
113         var formatted2 = result2.Format();
114
115         Assert.Equal(formatted1, formatted2);
116     }
117
118     [Fact]
119     public static void SetContextWithoutColonTest()
120     {
121         var input = @"empInfo
122 employees";
123
124         var parser = new Parser();
125         var result = parser.Parse(input);
126
127         Assert.NotEmpty(result);
128         var formatted = result.Format();
129         Assert.Contains("empInfo", formatted);
130         Assert.Contains("employees", formatted);
131     }
132
133     [Fact]
134     public static void SequenceContextWithColonTest()
135     {
136         var input = @"employees:
137 James Kirk
138 Jean-Luc Picard
139 Wesley Crusher";
140
141         var parser = new Parser();
142         var result = parser.Parse(input);
143
144         Assert.NotEmpty(result);
145         Assert.Single(result);
146         var formatted = result.Format();
147         Assert.Contains("employees:", formatted);
148         Assert.Contains("James Kirk", formatted);
149         Assert.Contains("Jean-Luc Picard", formatted);

```

```

150     Assert.Contains("Wesley Crusher", formatted);
151 }
152
153 [Fact]
154 public static void SequenceContextWithComplexValuesTest()
155 {
156     var input = @"employees:
157 (
158     name (James Kirk)
159     age 40
160 )
161 (
162     name (Jean-Luc Picard)
163     age 45
164 )";
165
166     var parser = new Parser();
167     var result = parser.Parse(input);
168
169     Assert.NotEmpty(result);
170     Assert.Single(result);
171     var formatted = result.Format();
172     Assert.Contains("employees:", formatted);
173     Assert.Contains("James Kirk", formatted);
174     Assert.Contains("Jean-Luc Picard", formatted);
175 }
176
177 [Fact]
178 public static void NestedSetAndSequenceContextsTest()
179 {
180     var input = @"company
181 departments:
182     engineering
183     sales
184 employees:
185     (name John)
186     (name Jane)";
187
188     var parser = new Parser();
189     var result = parser.Parse(input);
190
191     Assert.NotEmpty(result);
192     var formatted = result.Format();
193     Assert.Contains("company", formatted);
194     Assert.Contains("departments:", formatted);
195     Assert.Contains("employees:", formatted);
196 }
197
198 [Fact]
199 public static void DeeplyNestedMixedModesTest()
200 {
201     var input = @"root
202 level1
203     level2:
204         value1
205         value2
206     level2b
207     level3";
208
209     var parser = new Parser();
210     var result = parser.Parse(input);
211
212     Assert.NotEmpty(result);
213     var formatted = result.Format();
214     Assert.Contains("root", formatted);
215     Assert.Contains("level2:", formatted);
216 }
217 }
218 }

```

1.39 ./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs

```

1 using System;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     public static class MultiQuoteParserTests
7     {
8         // Helper to extract single reference ID

```

```

9         private static string? GetSingleRefId(System.Collections.Generic.IList<Link<string>>
↪ result)
10     {
11         if (result.Count == 1 && result[0].Id == null && result[0].Values?.Count == 1)
12         {
13             return result[0].Values![0].Id;
14         }
15         return result.Count == 1 ? result[0].Id : null;
16     }
17
18     // =====
19     // Backtick Quote Tests (Single Backtick)
20     // =====
21
22     [Fact]
23     public static void TestBacktickQuotedReference()
24     {
25         var parser = new Parser();
26         var result = parser.Parse("`backtick quoted`");
27         Assert.Equal("backtick quoted", GetSingleRefId(result));
28     }
29
30     [Fact]
31     public static void TestBacktickQuotedWithSpaces()
32     {
33         var parser = new Parser();
34         var result = parser.Parse("`text with spaces`");
35         Assert.Equal("text with spaces", GetSingleRefId(result));
36     }
37
38     [Fact]
39     public static void TestBacktickQuotedMultiline()
40     {
41         var parser = new Parser();
42         var result = parser.Parse("`line1\nline2`");
43         Assert.Single(result);
44         Assert.NotNull(result[0].Values);
45         Assert.Single(result[0].Values!);
46         Assert.Equal("line1\nline2", result[0].Values![0].Id);
47     }
48
49     [Fact]
50     public static void TestBacktickQuotedWithEscapedBacktick()
51     {
52         var parser = new Parser();
53         var result = parser.Parse("`text with `` escaped backtick`");
54         Assert.Equal("text with ` escaped backtick", GetSingleRefId(result));
55     }
56
57     // =====
58     // Single Quote Tests (with escaping)
59     // =====
60
61     [Fact]
62     public static void TestSingleQuoteWithEscapedSingleQuote()
63     {
64         var parser = new Parser();
65         var result = parser.Parse("'text with ' escaped quote'");
66         Assert.Equal("text with ' escaped quote", GetSingleRefId(result));
67     }
68
69     // =====
70     // Double Quote Tests (with escaping)
71     // =====
72
73     [Fact]
74     public static void TestDoubleQuoteWithEscapedDoubleQuote()
75     {
76         var parser = new Parser();
77         var result = parser.Parse("\"text with \" escaped quote\"");
78         Assert.Equal("text with \" escaped quote", GetSingleRefId(result));
79     }
80
81     // =====
82     // Double Quotes (2 quote chars) Tests
83     // =====
84
85     [Fact]

```

```

86 public static void TestDoubleDoubleQuotes()
87 {
88     var parser = new Parser();
89     var result = parser.Parse("\"\"double double quotes\"\"");
90     Assert.Equal("double double quotes", GetSingleRefId(result));
91 }
92
93 [Fact]
94 public static void TestDoubleDoubleQuotesWithSingleQuoteInside()
95 {
96     var parser = new Parser();
97     var result = parser.Parse("\"\"text with \" inside\"");
98     Assert.Equal("text with \" inside", GetSingleRefId(result));
99 }
100
101 [Fact]
102 public static void TestDoubleDoubleQuotesWithEscape()
103 {
104     var parser = new Parser();
105     var result = parser.Parse("\"\"text with \"\"\" escaped double\"");
106     Assert.Equal("text with \"\"\" escaped double", GetSingleRefId(result));
107 }
108
109 [Fact]
110 public static void TestDoubleSingleQuotes()
111 {
112     var parser = new Parser();
113     var result = parser.Parse("'double single quotes'");
114     Assert.Equal("double single quotes", GetSingleRefId(result));
115 }
116
117 [Fact]
118 public static void TestDoubleSingleQuotesWithSingleQuoteInside()
119 {
120     var parser = new Parser();
121     var result = parser.Parse("'text with ' inside'");
122     Assert.Equal("text with ' inside", GetSingleRefId(result));
123 }
124
125 [Fact]
126 public static void TestDoubleSingleQuotesWithEscape()
127 {
128     var parser = new Parser();
129     var result = parser.Parse("'text with ''' escaped single'");
130     Assert.Equal("text with ''' escaped single", GetSingleRefId(result));
131 }
132
133 [Fact]
134 public static void TestDoubleBacktickQuotes()
135 {
136     var parser = new Parser();
137     var result = parser.Parse("`double backtick quotes`");
138     Assert.Equal("double backtick quotes", GetSingleRefId(result));
139 }
140
141 [Fact]
142 public static void TestDoubleBacktickQuotesWithBacktickInside()
143 {
144     var parser = new Parser();
145     var result = parser.Parse("`text with ` inside`");
146     Assert.Equal("text with ` inside", GetSingleRefId(result));
147 }
148
149 [Fact]
150 public static void TestDoubleBacktickQuotesWithEscape()
151 {
152     var parser = new Parser();
153     var result = parser.Parse("`text with ``` escaped backtick`");
154     Assert.Equal("text with ``` escaped backtick", GetSingleRefId(result));
155 }
156
157 // =====
158 // Triple Quotes (3 quote chars) Tests
159 // =====
160
161 [Fact]
162 public static void TestTripleDoubleQuotes()
163 {

```

```

164     var parser = new Parser();
165     var result = parser.Parse("\"\"\"triple double quotes\"\"\"");
166     Assert.Equal("triple double quotes", GetSingleRefId(result));
167 }
168
169 [Fact]
170 public static void TestTripleDoubleQuotesWithDoubleQuoteInside()
171 {
172     var parser = new Parser();
173     var result = parser.Parse("\"\"\"text with \"\" inside\"\"\"");
174     Assert.Equal("text with \"\" inside", GetSingleRefId(result));
175 }
176
177 [Fact]
178 public static void TestTripleDoubleQuotesWithEscape()
179 {
180     var parser = new Parser();
181     var result = parser.Parse("\"\"\"text with \"\"\"\"\" escaped triple\"\"\"");
182     Assert.Equal("text with \"\"\"\" escaped triple", GetSingleRefId(result));
183 }
184
185 [Fact]
186 public static void TestTripleSingleQuotes()
187 {
188     var parser = new Parser();
189     var result = parser.Parse("'''triple single quotes'");
190     Assert.Equal("triple single quotes", GetSingleRefId(result));
191 }
192
193 [Fact]
194 public static void TestTripleBacktickQuotes()
195 {
196     var parser = new Parser();
197     var result = parser.Parse("`triple backtick quotes`");
198     Assert.Equal("triple backtick quotes", GetSingleRefId(result));
199 }
200
201 // =====
202 // Quadruple Quotes (4 quote chars) Tests
203 // =====
204
205 [Fact]
206 public static void TestQuadrupleDoubleQuotes()
207 {
208     var parser = new Parser();
209     var result = parser.Parse("\"\"\"\"quadruple double quotes\"\"\"\"");
210     Assert.Equal("quadruple double quotes", GetSingleRefId(result));
211 }
212
213 [Fact]
214 public static void TestQuadrupleSingleQuotes()
215 {
216     var parser = new Parser();
217     var result = parser.Parse("''''quadruple single quotes''");
218     Assert.Equal("quadruple single quotes", GetSingleRefId(result));
219 }
220
221 [Fact]
222 public static void TestQuadrupleBacktickQuotes()
223 {
224     var parser = new Parser();
225     var result = parser.Parse("`quadruple backtick quotes`");
226     Assert.Equal("quadruple backtick quotes", GetSingleRefId(result));
227 }
228
229 // =====
230 // Quintuple Quotes (5 quote chars) Tests
231 // =====
232
233 [Fact]
234 public static void TestQuintupleDoubleQuotes()
235 {
236     var parser = new Parser();
237     var result = parser.Parse("\"\"\"\"\"quintuple double quotes\"\"\"\"\"");
238     Assert.Equal("quintuple double quotes", GetSingleRefId(result));
239 }
240
241 [Fact]
242 public static void TestQuintupleSingleQuotes()

```

```

243 {
244     var parser = new Parser();
245     var result = parser.Parse("''''quintuple single quotes''''");
246     Assert.Equal("quintuple single quotes", GetSingleRefId(result));
247 }
248
249 [Fact]
250 public static void TestQuintupleBacktickQuotes()
251 {
252     var parser = new Parser();
253     var result = parser.Parse("````quintuple backtick quotes````");
254     Assert.Equal("quintuple backtick quotes", GetSingleRefId(result));
255 }
256
257 // =====
258 // Complex Scenarios Tests
259 // =====
260
261 [Fact]
262 public static void TestMixedQuotesInLink()
263 {
264     var parser = new Parser();
265     var result = parser.Parse("(\"double\" 'single' `backtick`)");
266     Assert.Single(result);
267     Assert.NotNull(result[0].Values);
268     Assert.Equal(3, result[0].Values!.Count);
269     Assert.Equal("double", result[0].Values![0].Id);
270     Assert.Equal("single", result[0].Values![1].Id);
271     Assert.Equal("backtick", result[0].Values![2].Id);
272 }
273
274 [Fact]
275 public static void TestBacktickAsIdInLink()
276 {
277     var parser = new Parser();
278     var result = parser.Parse("`myId` : value1 value2");
279     Assert.Single(result);
280     Assert.Equal("myId", result[0].Id);
281     Assert.NotNull(result[0].Values);
282     Assert.Equal(2, result[0].Values!.Count);
283 }
284
285 [Fact]
286 public static void TestCodeBlockLikeContent()
287 {
288     var parser = new Parser();
289     var result = parser.Parse("```const x = 1;```");
290     Assert.Equal("const x = 1;", GetSingleRefId(result));
291 }
292
293 [Fact]
294 public static void TestNestedQuotesInMarkdown()
295 {
296     var parser = new Parser();
297     var result = parser.Parse("`Use `code` in markdown`");
298     Assert.Equal("Use `code` in markdown", GetSingleRefId(result));
299 }
300
301 [Fact]
302 public static void TestJsonStringWithQuotes()
303 {
304     var parser = new Parser();
305     var result = parser.Parse("{\"key\": \"value\"}\"");
306     Assert.Equal("{ \"key\": \"value\"}", GetSingleRefId(result));
307 }
308
309 // =====
310 // Edge Cases
311 // =====
312
313 [Fact]
314 public static void TestWhitespacePreservedInQuotes()
315 {
316     var parser = new Parser();
317     var result = parser.Parse("  spaces  ");
318     Assert.Equal("  spaces  ", GetSingleRefId(result));
319 }
320

```

```

321 [Fact]
322 public static void TestMultilineInDoubleDoubleQuotes()
323 {
324     var parser = new Parser();
325     var result = parser.Parse(@"\"line1\nline2\"");
326     Assert.Single(result);
327     Assert.NotNull(result[0].Values);
328     Assert.Single(result[0].Values!);
329     Assert.Equal("line1\nline2", result[0].Values![0].Id);
330 }
331
332 // =====
333 // Unlimited Quotes (6+ quote chars) Tests
334 // =====
335
336 [Fact]
337 public static void TestUnlimitedQuotes6()
338 {
339     // Test 6-quote strings
340     var parser = new Parser();
341     var result = parser.Parse("\"\"\"\"hello\"\"\"");
342     Assert.Equal("hello", GetSingleRefId(result));
343 }
344
345 [Fact]
346 public static void TestUnlimitedQuotes10()
347 {
348     // Test 10-quote strings
349     var parser = new Parser();
350     var result = parser.Parse("\"\"\"\"\"\"\"\"very deeply
↪ quoted\"\"\"\"\"\"");
351     Assert.Equal("very deeply quoted", GetSingleRefId(result));
352 }
353
354 [Fact]
355 public static void TestUnlimitedQuotes6WithInnerQuotes()
356 {
357     // Test 6-quote strings with inner 5-quote sequences
358     var parser = new Parser();
359     var result = parser.Parse("\"\"\"\"\"\"hello with \"\"\"\"\" five quotes
↪ inside\"\"\"\"\"");
360     Assert.Equal("hello with \"\"\"\"\" five quotes inside", GetSingleRefId(result));
361 }
362
363 [Fact]
364 public static void TestUnlimitedSingleQuotes7()
365 {
366     // Test 7-quote single quote strings
367     var parser = new Parser();
368     var result = parser.Parse('''seven single quotes''');
369     Assert.Equal("seven single quotes", GetSingleRefId(result));
370 }
371
372 [Fact]
373 public static void TestUnlimitedBackticks8()
374 {
375     // Test 8-quote backtick strings
376     var parser = new Parser();
377     var result = parser.Parse("`eight backticks`");
378     Assert.Equal("eight backticks", GetSingleRefId(result));
379 }
380 }
381 }

```

1.40 ./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class MultilineParserTests
7      {
8          [Fact]
9          public static void TwoLinksTest()
10         {
11             var source = @"(first: x y)
12 (second: a b)";
13             var parser = new Parser();

```

```

14         var links = parser.Parse(source);
15         var target = links.Format();
16         Assert.Equal(source, target);
17     }
18
19     [Fact]
20     public static void ParseAndStringifyTest()
21     {
22         var source = @"(papa (lovesMama: loves mama))
23 (son lovesMama)
24 (daughter lovesMama)
25 (all (love mama))";
26         var parser = new Parser();
27         var links = parser.Parse(source);
28         var target = links.Format();
29         Assert.Equal(source, target);
30     }
31
32     [Fact]
33     public static void ParseAndStringifyTest2()
34     - {
35         var source = @"father (lovesMom: loves mom)
36 son lovesMom
37 daughter lovesMom
38 all (love mom)";
39         var links = new Parser().Parse(source);
40         var target = links.Format(lessParentheses: true);
41         Assert.Equal(source, target);
42     }
43
44     [Fact]
45     public static void ParseAndStringifyWithLessParenthesesTest()
46     {
47         var source = @"lovesMama: loves mama
48 papa lovesMama
49 son lovesMama
50 daughter lovesMama
51 all (love mama)";
52         var parser = new Parser();
53         var links = parser.Parse(source);
54         var target = links.Format(lessParentheses: true);
55         Assert.Equal(source, target);
56     }
57
58     [Fact]
59     public static void DuplicateIdentifiersTest()
60     {
61         var source = @"(a: a b)
62 (a: b c)";
63         var target = @"(a: a b)
64 (a: b c)";
65         var parser = new Parser();
66         var links = parser.Parse(source);
67         var formattedLinks = links.Format();
68         Assert.Equal(target, formattedLinks);
69     }
70
71     [Fact]
72     public static void ComplexStructureTest()
73     {
74         var input = @"(Type: Type Type)
75 Number
76 String
77 Array
78 Value
79 (property: name type)
80 (method: name params return)";
81
82         var result = new Parser().Parse(input);
83         Assert.NotEmpty(result);
84     }
85
86     [Fact]
87     public static void MixedFormatsTest()
88     {
89         // Mix of single-line and multi-line formats
90         var input = @"id1: value1
91 (id2: value2 value3)
92 simple ref

```

```

93     (complex:
94         nested1
95         nested2
96     );
97
98     var result = new Parser().Parse(input);
99     Assert.NotEmpty(result);
100 }
101
102 [Fact]
103 public static void MultilineWithIdTest()
104 {
105     // Test multi-line link with id
106     var input = "(id: value1 value2)";
107     var result = new Parser().Parse(input);
108     Assert.NotEmpty(result);
109 }
110
111 [Fact]
112 public static void MultipleTopLevelElementsTest()
113 {
114     // Test multiple top-level elements
115     var input = "(elem1: val1)\n(elem2: val2)";
116     var result = new Parser().Parse(input);
117     Assert.NotEmpty(result);
118 }
119
120 [Fact]
121 public static void MultilineSimpleLinksTest()
122 {
123     var input = "(1: 1 1)\n(2: 2 2)";
124     var parser = new Parser();
125     var parsed = parser.Parse(input);
126     Assert.NotEmpty(parsed);
127
128     // Validate regular formatting
129     var output = parsed.Format();
130     Assert.Contains("(1: 1 1)", output);
131     Assert.Contains("(2: 2 2)", output);
132
133     // Validate alternate formatting matches input
134     var outputAlternate = parsed.Format();
135     Assert.Equal(input, outputAlternate);
136 }
137
138 [Fact]
139 public static void IndentedChildrenTest()
140 {
141     var input = "parent\n  child1\n  child2";
142     var parser = new Parser();
143     var parsed = parser.Parse(input);
144
145     // The parsed structure should have parent with children
146     Assert.NotEmpty(parsed);
147 }
148 }
149 }

```

1.41 ./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class MultilineQuotedStringTests
7      {
8          [Fact]
9          public static void TestMultilineDoubleQuotedReference()
10         {
11             var input = @"(
12                 ""long
13                 string literal representing
14                 the reference""
15
16                 'another
17                 long string literal
18                 as another reference'
19             )";
20
21             var parser = new Parser();
22             var result = parser.Parse(input);

```

```

22         Assert.NotEmpty(result);
23         Assert.Single(result);
24
25         var link = result[0];
26         Assert.Null(link.Id);
27         Assert.NotNull(link.Values);
28         Assert.Equal(2, link.Values.Count);
29
30         // The two references sit on separate lines inside the parentheses, so the
31         // nested context turns each of them into its own link.
32         Assert.Equal(@"long
33 string literal representing
34 the reference", link.Values![0].Values![0].Id);
35
36         Assert.Equal(@"another
37 long string literal
38 as another reference", link.Values![1].Values![0].Id);
39     }
40
41     [Fact]
42     public static void TestSimpleMultilineDoubleQuoted()
43     {
44         var input = @"("line1
45 line2")";
46         var parser = new Parser();
47         var result = parser.Parse(input);
48
49         Assert.NotEmpty(result);
50         Assert.Single(result);
51
52         var link = result[0];
53         Assert.Null(link.Id);
54         Assert.NotNull(link.Values);
55         Assert.Single(link.Values);
56         Assert.Equal("line1\nline2", link.Values[0].Id);
57     }
58
59     [Fact]
60     public static void TestSimpleMultilineSingleQuoted()
61     {
62         var input = @"('line1
63 line2')";
64         var parser = new Parser();
65         var result = parser.Parse(input);
66
67         Assert.NotEmpty(result);
68         Assert.Single(result);
69
70         var link = result[0];
71         Assert.Null(link.Id);
72         Assert.NotNull(link.Values);
73         Assert.Single(link.Values);
74         Assert.Equal("line1\nline2", link.Values[0].Id);
75     }
76
77     [Fact]
78     public static void TestMultilineQuotedAsId()
79     {
80         var input = @"("multi
81 line
82 id": value1 value2)";
83         var parser = new Parser();
84         var result = parser.Parse(input);
85
86         Assert.NotEmpty(result);
87         Assert.Single(result);
88
89         var link = result[0];
90         Assert.Equal("multi\nline\nid", link.Id);
91         Assert.NotNull(link.Values);
92         Assert.Equal(2, link.Values.Count);
93     }
94 }
95
96 }

```

1.42 ./Link.Foundation.Links.Notation.Tests/NestedIndentationTests.cs

```

1 using Xunit;
2

```

```

3 namespace Link.Foundation.Links.Notation.Tests
4 {
5     /// <summary>
6     /// Tests for indentation inside parentheses.
7     ///
8     /// See https://github.com/link-foundation/links-notation/issues/282
9     ///
10    /// Indentation is structural at the root, so it must be structural inside parentheses too:
11    /// a parenthesized group opens a nested context that starts fresh at indentation level zero
12    /// and follows exactly the root's rules.
13    /// </summary>
14    public class NestedIndentationTests
15    {
16        private static string FormatSource(string source) => new Parser().Parse(source).Format();
17
18        private static void AssertFormat(string source, string expected) =>
19        ↪ Assert.Equal(expected, FormatSource(source));
20
21        [Fact]
22        public void ParenthesesReproduceRootIndentation()
23        {
24            AssertFormat("a\n b\nc\n d", "(a)\n((a) (b))\n(c)\n((c) (d))");
25
26            // The same lines inside parentheses keep the same structure, nested under
27            // the link the group belongs to.
28            ↪ AssertFormat("array (\n a\n b\n c\n d\n)", "(array ((a) ((a) (b)) (c) ((c) (d))))");
29
30            [Fact]
31            public void ParenthesesKeepRecordBoundaries()
32            {
33                var source = "value (\n id \"1\"\n label \"one\"\n)";
34                AssertFormat(source, "(value ((id 1) (label one)))");
35
36                var links = new Parser().Parse(source);
37                Assert.Single(links);
38
39                var group = links[0].Values![1];
40                Assert.Null(group.Id);
41                Assert.Equal(2, group.Values!.Count);
42                Assert.Equal("id", group.Values[0].Values![0].Id);
43                Assert.Equal("1", group.Values[0].Values![1].Id);
44                Assert.Equal("label", group.Values[1].Values![0].Id);
45                Assert.Equal("one", group.Values[1].Values![1].Id);
46            }
47
48            [Fact]
49            public void ParenthesesKeepSeveralRecordsSeparate()
50            {
51                AssertFormat(
52                    "value (\n (id \"1\" label \"one\")\n (id \"2\" label \"two\")\n)",
53                    "(value ((id 1 label one) (id 2 label two)))");
54            }
55
56            [Fact]
57            public void ParenthesesNestDeeply()
58            {
59                ↪ AssertFormat("outer (\n inner (\n x 1\n y 2\n )\n z 3\n)", "(outer ((inner ((x 1) (y 2))) (z 3)))");
60            }
61
62            [Fact]
63            public void SingleLineParenthesesAreUnchanged()
64            {
65                AssertFormat("(a b c)", "(a b c)");
66                AssertFormat("(1: 2 3)", "(1: 2 3)");
67                AssertFormat("(a: b c)", "(a: b c)");
68                AssertFormat("((a b))", "((a b))");
69                AssertFormat("(a)", "(a)");
70                AssertFormat("()", "()");
71            }
72
73            [Fact]
74            public void ParenthesesWithIndentedIdSyntax()
75            {
76                AssertFormat("( \n a: \n b \n c \n)", "(a: b c)");
77            }

```

```

78     [Fact]
79     public void EmployeeRecordsKeepTheirFields()
80     {
81         var source = "empInfo\n  employees:\n    (\n      name (James Kirk)\n      age 40\n
82     ↪    )\n"
83         + "    (\n      name (Jean-Luc Picard)\n      age 45\n    )";
84         var expected = "(empInfo)\n((empInfo) (employees: ((name (James Kirk)) (age 40)))"
85         + " ((name (Jean-Luc Picard)) (age 45)))";
86         AssertFormat(source, expected);
87     }
88 }
89 }

```

1.43 ./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs

```

1  using System;
2  using System.Linq;
3  using Xunit;
4
5  namespace Link.Foundation.Links.Notation.Tests
6  {
7      public static class NestedParserTests
8      {
9          [Fact]
10         public static void SignificantWhitespaceTest()
11         {
12             var source = @"
13 users
14     user1
15         id
16             43
17         name
18             first
19                 John
20             last
21                 Williams
22         location
23             New York
24         age
25             23
26     user2
27         id
28             56
29         name
30             first
31                 Igor
32             middle
33                 Petrovich
34             last
35                 Ivanov
36         location
37             Moscow
38         age
39             20";
40             var target = @"(users)
41 ((users) (user1))
42 (((users) (user1)) (id))
43 (((users) (user1)) (id)) (43))
44 ((users) (user1)) (name))
45 (((users) (user1)) (name)) (first))
46 (((((users) (user1)) (name)) (first)) (John))
47 (((users) (user1)) (name)) (last))
48 (((((users) (user1)) (name)) (last)) (Williams))
49 ((users) (user1)) (location))
50 (((users) (user1)) (location)) (New York))
51 ((users) (user1)) (age))
52 (((users) (user1)) (age)) (23))
53 ((users) (user2))
54 ((users) (user2)) (id))
55 (((users) (user2)) (id)) (56))
56 ((users) (user2)) (name))
57 (((users) (user2)) (name)) (first))
58 (((((users) (user2)) (name)) (first)) (Igor))
59 (((users) (user2)) (name)) (middle))
60 (((((users) (user2)) (name)) (middle)) (Petrovich))
61 (((users) (user2)) (name)) (last))
62 (((((users) (user2)) (name)) (last)) (Ivanov))
63 ((users) (user2)) (location))
64 (((users) (user2)) (location)) (Moscow))
65 ((users) (user2)) (age))
66 (((users) (user2)) (age)) (20))";

```

```

67         var parser = new Parser();
68         var links = parser.Parse(source);
69         var formattedLinks = links.Format();
70         Assert.Equal(target, formattedLinks);
71     }
72
73     [Fact]
74     public static void SimpleSignificantWhitespaceTest()
75     {
76         var source = @"a
77         b
78         c";
79         var target = @"(a)
80         (b)
81         (c)";
82         var parser = new Parser();
83         var links = parser.Parse(source);
84         var formattedLinks = links.Format();
85         Assert.Equal(target, formattedLinks);
86     }
87
88     [Fact]
89     public static void TwoSpacesSizedWhitespaceTest()
90     {
91         var source = @"
92 users
93     user1";
94         var target = @"(users)
95     (user1)";
96         var parser = new Parser();
97         var links = parser.Parse(source);
98         var formattedLinks = links.Format();
99         Assert.Equal(target, formattedLinks);
100     }
101
102     [Fact]
103     public static void ParseNestedStructureWithIndentationTest()
104     {
105         var source = @"parent
106 child1
107 child2";
108         var parser = new Parser();
109         var links = parser.Parse(source);
110         Assert.NotNull(links);
111         // Should create 3 links: (parent), (parent child1), (parent child2)
112         Assert.Equal(3, links.Count);
113     }
114
115     [Fact]
116     public static void IndentationConsistencyTest()
117     {
118         // Test that indentation must be consistent
119         var input = @"parent
120 child1
121 child2"; // Inconsistent indentation
122         var result = new Parser().Parse(input);
123         // This should parse but child2 won't be a child of parent due to different
124         ↪ indentation
125         Assert.NotEmpty(result);
126     }
127
128     [Fact]
129     public static void IndentationBasedChildrenTest()
130     {
131         var input = @"parent
132 child1
133 child2
134     grandchild";
135         var parser = new Parser();
136         var result = parser.Parse(input);
137         Assert.NotNull(result);
138         // Expected flattened links:
139         // (parent), (parent child1), (parent child2), ((parent child2) grandchild)
140         Assert.Equal(4, result.Count);
141     }
142
143     [Fact]
144     public static void ComplexIndentationTest()
145     {

```

```

145         var input = @"root
146     level1a
147         level2a
148         level2b
149     level1b
150         level2c";
151     var parser = new Parser();
152     var result = parser.Parse(input);
153     Assert.NotNull(result);
154     // Expected flattened links:
155     // (root), (root level1a), ((root level1a) level2a), ((root level1a) level2b),
156     // (root level1b), ((root level1b) level2c)
157     Assert.Equal(6, result.Count);
158 }
159
160 [Fact]
161 public static void NestedLinksTest()
162 {
163     var input = "(1: (2: (3: 3)))";
164     var parser = new Parser();
165     var parsed = parser.Parse(input);
166     Assert.NotEmpty(parsed);
167
168     // Validate regular formatting
169     var output = parsed.Format();
170     Assert.NotEmpty(output);
171
172     // Validate that the structure is properly nested
173     Assert.Single(parsed);
174 }
175
176 [Fact]
177 public static void IndentationParserTest()
178 {
179     var input = "parent\n  child1\n  child2";
180     var parser = new Parser();
181     var result = parser.Parse(input);
182     Assert.NotEmpty(result);
183     // Should have parent link
184     var parentLink = result.FirstOrDefault(l => l.Id == "parent");
185 }
186
187 [Fact]
188 public static void NestedIndentationParserTest()
189 {
190     var input = "parent\n  child\n    grandchild";
191     var parser = new Parser();
192     var result = parser.Parse(input);
193     Assert.NotEmpty(result);
194     // Should create nested structure with proper hierarchy
195     Assert.True(result.Count >= 1);
196 }
197
198 [Fact]
199 public static void ThreeLevelNestingRoundtripTest()
200 {
201     var input = "(1: (2: (3: 3)))";
202     var parser = new Parser();
203     var parsed = parser.Parse(input);
204
205     // Validate round-trip
206     var output = parsed.Format();
207     Assert.Equal(input, output);
208 }
209
210 [Fact]
211 public static void DeepNestedStructureRoundtripTest()
212 {
213     var input = "(a: (b: (c: (d: d))))";
214     var parser = new Parser();
215     var parsed = parser.Parse(input);
216
217     // Validate round-trip
218     var output = parsed.Format();
219     Assert.Equal(input, output);
220 }
221
222 [Fact]
223 public static void MultipleNestedLinksRoundtripTest()

```

```

224     {
225         var input = "(parent: (child1: value1) (child2: value2))";
226         var parser = new Parser();
227         var parsed = parser.Parse(input);
228
229         // Validate round-trip
230         var output = parsed.Format();
231         Assert.Equal(input, output);
232     }
233 }
234 }

```

1.44 ./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs

```

1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class NestedSelfReferenceTests
7      {
8          [Fact]
9          public static void NestedSelfReferencedObjectInPairValueTest()
10         {
11             // Test case from PARSER BUG.md
12             // This should parse a dict with two pairs, where the second pair's value
13             // is itself a self-referenced dict definition (obj 1: dict ...)
14             var notation = "(obj 0: dict ((str bmFtZQ==) (str ZG1jdDE=)) ((str b3RoZXI=) (obj 1:
↪ dict ((str bmFtZQ==) (str ZG1jdDI=)) ((str b3RoZXI=) obj 0))))";
15
16             var parser = new Parser();
17             var links = parser.Parse(notation);
18
19             // Should parse exactly one top-level link
20             Assert.Single(links);
21
22             var link = links[0];
23
24             // Top-level link should have ID "obj 0"
25             Assert.Equal("obj 0", link.Id);
26
27             // Should have: type marker + 2 pairs = 3 values
28             Assert.Equal(3, link.Values!.Count);
29
30             // First value is the type marker "dict"
31             Assert.Equal("dict", link.Values[0].Id);
32
33             // Second and third values are the two pairs
34             var pair1 = link.Values[1];
35             var pair2 = link.Values[2];
36
37             // Pair 1: ((str bmFtZQ==) (str ZG1jdDE=))
38             // This is a parenthesized expression containing two sub-expressions
39             Assert.Null(pair1.Id);
40             Assert.Equal(2, pair1.Values!.Count);
41
42             // First element of pair1: (str bmFtZQ==)
43             Assert.Null(pair1.Values[0].Id);
44             Assert.Equal(2, pair1.Values![0].Values!.Count);
45             Assert.Equal("str", pair1.Values![0].Values![0].Id);
46             Assert.Equal("bmFtZQ==", pair1.Values![0].Values![1].Id);
47
48             // Second element of pair1: (str ZG1jdDE=)
49             Assert.Null(pair1.Values[1].Id);
50             Assert.Equal(2, pair1.Values![1].Values!.Count);
51             Assert.Equal("str", pair1.Values![1].Values![0].Id);
52             Assert.Equal("ZG1jdDE=", pair1.Values![1].Values![1].Id);
53
54             // Pair 2: ((str b3RoZXI=) (obj 1: dict ...))
55             // This is a parenthesized expression containing two sub-expressions
56             Assert.Null(pair2.Id);
57             Assert.Equal(2, pair2.Values!.Count);
58
59             // First element of pair2: (str b3RoZXI=)
60             Assert.Null(pair2.Values[0].Id);
61             Assert.Equal(2, pair2.Values![0].Values!.Count);
62             Assert.Equal("str", pair2.Values![0].Values![0].Id);
63             Assert.Equal("b3RoZXI=", pair2.Values![0].Values![1].Id);
64

```

```

65         // Second element of pair2: (obj 1:dict ((str bmFtZQ==) (str ZGljdDI=)) ((str
↪ b3RoZXI=) obj 0))
66         // THIS IS THE KEY TEST - obj 1 should have its ID preserved
67         var obj1 = pair2.Values[1];
68         Assert.Equal("obj 1", obj1.Id);
69         Assert.NotNull(obj1.Values);
70         Assert.Equal(3, obj1.Values.Count); // type marker + 2 pairs
71
72         // obj 1's type marker
73         Assert.Equal("dict", obj1.Values[0].Id);
74
75         // obj 1's first pair: ((str bmFtZQ==) (str ZGljdDI=))
76         var obj1Pair1 = obj1.Values[1];
77         Assert.Equal(2, obj1Pair1.Values!.Count);
78         Assert.Null(obj1Pair1.Values[0].Id);
79         Assert.Equal(2, obj1Pair1.Values![0].Values!.Count);
80         Assert.Equal("str", obj1Pair1.Values![0].Values![0].Id);
81         Assert.Equal("bmFtZQ==", obj1Pair1.Values![0].Values![1].Id);
82         Assert.Null(obj1Pair1.Values[1].Id);
83         Assert.Equal(2, obj1Pair1.Values![1].Values!.Count);
84         Assert.Equal("str", obj1Pair1.Values![1].Values![0].Id);
85         Assert.Equal("ZGljdDI=", obj1Pair1.Values![1].Values![1].Id);
86
87         // obj 1's second pair: ((str b3RoZXI=) obj 0) - reference back to obj 0
88         var obj1Pair2 = obj1.Values[2];
89         Assert.Equal(2, obj1Pair2.Values!.Count);
90         Assert.Null(obj1Pair2.Values[0].Id);
91         Assert.Equal(2, obj1Pair2.Values![0].Values!.Count);
92         Assert.Equal("str", obj1Pair2.Values![0].Values![0].Id);
93         Assert.Equal("b3RoZXI=", obj1Pair2.Values![0].Values![1].Id);
94         Assert.Equal("obj 0", obj1Pair2.Values[1].Id);
95         Assert.Null(obj1Pair2.Values[1].Values); // Just a reference, no nested values
96     }
97
98     [Fact]
99     public static void SelfReferenceAsDirectChildWorksCorrectlyTest()
100     {
101         // This pattern should work (and did work before)
102         var notation = "(obj 0: list (int 1) (int 2) (obj 1: list (int 3) (int 4) obj 0))";
103
104         var parser = new Parser();
105         var links = parser.Parse(notation);
106
107         Assert.Single(links);
108         Assert.Equal("obj 0", links[0].Id);
109         Assert.Equal(4, links[0].Values!.Count); // list + 1 + 2 + obj 1
110
111         // The fourth value should be obj 1 with a self-reference
112         var obj1 = links[0].Values![3];
113         Assert.Equal("obj 1", obj1.Id);
114         Assert.Equal(4, obj1.Values!.Count); // list + 3 + 4 + obj 0
115         Assert.Equal("obj 0", obj1.Values[3].Id); // Reference to obj 0
116     }
117 }
118 }

```

1.45 ./Link.Foundation.Links.Notation.Tests/NestingDepthTests.cs

```

1  using System;
2  using System.Linq;
3  using System.Threading;
4  using Xunit;
5
6  namespace Link.Foundation.Links.Notation.Tests
7  {
8      /// <summary>
9      /// Reading nested groups must take time that grows with the size of the document,
10     /// not with two to the power of its nesting depth
11     /// (<see href="https://github.com/link-foundation/links-notation/issues/314">#314</see>).
12     /// Before the fix, every level of nesting doubled the work: a document nested six
13     /// levels deep took seconds. Each case runs on its own thread and fails when it has
14     /// not finished within the budget, so a regression fails instead of hanging.
15     /// </summary>
16     public class NestingDepthTests
17     {
18         /// Generous enough for a debug build on a slow machine, and far below what the
19         /// exponential reading took at the depths used here.
20         private static readonly TimeSpan Budget = TimeSpan.FromSeconds(5);
21
22         /// The depth used for the shapes that used to take exponential time.

```

```

23     private const int Deep = 32;
24
25     /// A depth that shows the reading stays linear well past the shallow cases.
26     private const int VeryDeep = 256;
27
28     private static string Closed(int depth) => new string('(', depth) + "a" + new
↪ string(')', depth);
29
30     private static string ValueAfter(int depth) => new string('(', depth) + "a" +
↪ string.Concat(Enumerable.Repeat(" b", depth));
31
32     private static string Unclosed(int depth) => new string('(', depth) + "a";
33
34     /// Unclosed groups on lines that are each indented one space deeper.
35     private static string IndentedUnclosed(int depth) =>
36     string.Join("\n", Enumerable.Range(0, depth).Select(level => new string(' ', level)
↪ + "(a)"));
37
38     /// Formats the parsed document, or names the exception parsing raised.
39     private static string ReadWithinBudget(string what, string source)
40     {
41         string? outcome = null;
42         var reader = new Thread(() =>
43         {
44             try
45             {
46                 // The documents nest deeper than the default limit on purpose; only the
47                 // time they take is measured here.
48                 outcome = new Parser(true, int.MaxValue).Parse(source).Format();
49             }
50             catch (Exception error)
51             {
52                 outcome = error.GetType().Name;
53             }
54         }, 512 * 1024 * 1024)
55         {
56             // A reader that runs past the budget must not keep the test run alive.
57             IsBackground = true,
58         };
59         reader.Start();
60         Assert.True(reader.Join(Budget), $"{what} did not finish within {Budget}");
61         return outcome!;
62     }
63
64     [Fact]
65     public void ClosedGroupsReadInLinearTime()
66     {
67         var source = Closed(Deep);
68         Assert.Equal(source, ReadWithinBudget("closed groups", source));
69     }
70
71     [Fact]
72     public void ValuesAfterGroupsReadInLinearTime()
73     {
74         var source = ValueAfter(Deep);
75         Assert.Equal($"({source})", ReadWithinBudget("values after groups", source));
76     }
77
78     [Fact]
79     public void UnclosedGroupsFailInLinearTime()
80     {
81         Assert.Equal(nameof(ParseException), ReadWithinBudget("unclosed groups",
↪ Unclosed(Deep)));
82     }
83
84     [Fact]
85     public void UnclosedGroupsOnIndentedLinesFailInLinearTime()
86     {
87         Assert.Equal(nameof(ParseException), ReadWithinBudget("unclosed indented groups",
↪ IndentedUnclosed(Deep)));
88     }
89
90     [Fact]
91     public void ValuesAfterVeryDeepGroupsReadInLinearTime()
92     {
93         var source = ValueAfter(VeryDeep);
94         Assert.Equal($"({source})", ReadWithinBudget("very deep values after groups",
↪ source));

```

```

95     }
96
97     [Fact]
98     public void GroupFollowedByValuesKeepsItsStructure()
99     {
100         var cases = new[]
101         {
102             new[] { "(a) b", "((a) b)" },
103             new[] { "(a) (b) c", "((a) (b) c)" },
104             new[] { "((a) b) c", "(((a) b) c)" },
105             new[] { "(a: b) c", "((a: b) c)" },
106             new[] { "(a)\n(b) c", "(a)\n((b) c)" },
107             new[] { "x\n (a) b\n (c)", "(x)\n((x) ((a) b))\n((x) (c))" },
108         };
109
110         var parser = new Parser();
111         foreach (var testCase in cases)
112         {
113             Assert.Equal(testCase[1], parser.Parse(testCase[0]).Format());
114         }
115     }
116 }
117 }

```

1.46 ./Link.Foundation.Links.Notation.Tests/NestingLimitTests.cs

```

1  using System;
2  using System.Linq;
3  using System.Threading;
4  using Xunit;
5
6  namespace Link.Foundation.Links.Notation.Tests
7  {
8      /// <summary>
9      /// Links nested too deeply are refused with an error rather than recursed into until
10     /// the stack overflows, which ends the process where no catch block sees it
11     /// (https://github.com/link-foundation/links-notation/issues/315).
12     /// </summary>
13     /// <remarks>
14     /// Every parenthesized group and every indentation level is one level, and the lines of
15     /// a document start at level 0. The positions asserted here are the ones the Rust and
16     /// JavaScript ports report for the same input.
17     /// </remarks>
18     public static class NestingLimitTests
19     {
20         private static string Parens(int depth) => new string('(', depth) + "a" + new
↪ string(')', depth);
21
22         private static string Values(int depth) =>
23             string.Concat(Enumerable.Repeat("a ", depth)) + "b" + new string(')', depth);
24
25         private static string Indentation(int depth) =>
26         ↪ string.Concat(Enumerable.Range(0, depth + 1).Select(level => new string(' ', level)
↪ + "a\n"));
27
28         private static ParseException TooDeep(string document, int maxDepth)
29         {
30             var error = Assert.Throws<ParseException>(() => new Parser(true,
↪ maxDepth).Parse(document));
31             Assert.Equal(maxDepth, error.MaxDepth);
32             return error;
33         }
34
35         private static bool Accepted(string document, int maxDepth) =>
36             new Parser(true, maxDepth).Parse(document).Count > 0;
37
38         [Fact]
39         public static void DefaultLimitIsSharedByEveryImplementationTest()
40         {
41             Assert.Equal(64, Parser.DefaultMaxDepth);
42             Assert.Equal(Parser.DefaultMaxDepth, new Parser().MaxDepth);
43             Assert.Equal(Parser.DefaultMaxDepth, new Parser(comments: false).MaxDepth);
44             Assert.Equal(3, new Parser { MaxDepth = 3 }.MaxDepth);
45         }
46
47         [Fact]
48         public static void NegativeLimitIsRejectedTest()
49         {
50             Assert.Throws<ArgumentOutOfRangeException>(() => new Parser(true, -1));
51             Assert.Throws<ArgumentOutOfRangeException>(() => new Parser { MaxDepth = -1 });

```

```

52     }
53
54     [Fact]
55     public static void ParenthesesUpToTheLimitAreAcceptedTest()
56     {
57         Assert.True(Accepted(Parens(3), 3));
58         Assert.True(Accepted(Values(3), 3));
59         Assert.NotEmpty(new Parser().Parse(Parens(Parser.DefaultMaxDepth)));
60         Assert.NotEmpty(new Parser().Parse(Values(Parser.DefaultMaxDepth)));
61     }
62
63     [Fact]
64     public static void EveryGroupAndEveryIndentationLevelIsOneLevelTest()
65     {
66         Assert.True(Accepted("(a)", 1));
67         Assert.True(Accepted("a b c", 0));
68         TooDeep("(a)", 0);
69         TooDeep("((a))", 1);
70         TooDeep("(a (b))", 1);
71         TooDeep("a\n (b)\n", 1);
72     }
73
74     [Fact]
75     public static void ParenthesesPastTheLimitAreRefusedAtTheGroupThatIsTooDeepTest()
76     {
77         var error = TooDeep(Parens(4), 3);
78
79         Assert.Equal((1, 4, 3), (error.Line, error.Column, error.Offset));
80         Assert.Equal("line 1, column 4: nesting depth exceeds the maximum of 3",
81 ↪ error.Summary);
82         Assert.Equal("1 | (((a)))\n | ^", error.Snippet);
83         Assert.Equal(
84 ↪ +         "Nesting too deep at line 1, column 4: nesting depth exceeds the maximum of 3\n"
85             "1 | (((a)))\n" +
86             " | ^",
87             error.Message);
88     }
89
90     [Fact]
91     public static void GroupsInValuePositionCountLikeAnyOtherGroupTest()
92     {
93         var error = TooDeep(Values(4), 3);
94
95         Assert.Equal((1, 10), (error.Line, error.Column));
96     }
97
98     [Fact]
99     public static void IndentationUpToTheLimitIsAcceptedTest()
100    {
101        Assert.True(Accepted(Indentation(3), 3));
102        Assert.NotEmpty(new Parser().Parse(Indentation(Parser.DefaultMaxDepth)));
103    }
104
105    [Fact]
106    public static void IndentationPastTheLimitIsRefusedAtTheLineThatIsTooDeepTest()
107    {
108        var error = TooDeep(Indentation(4), 3);
109
110        Assert.Equal((5, 5), (error.Line, error.Column));
111        Assert.Equal("a", error.LineText);
112    }
113
114    [Fact]
115    public static void GroupsAndIndentationAddUpTest()
116    {
117        // `(b)` on the line indented once is at level 2.
118        Assert.True(Accepted("a\n (b)\n", 2));
119        var error = TooDeep("a\n (b)\n", 1);
120
121        Assert.Equal((2, 3), (error.Line, error.Column));
122    }
123
124    [Fact]
125    public static void IndentationInsideAGroupCountsOnTopOfTheGroupTest()
126    {
127        // `b` is inside one group and indented once within it.
128        Assert.True(Accepted("(a\n b)\n", 2));
129        var error = TooDeep("(a\n b)\n", 1);

```

```

129     Assert.Equal((2, 3), (error.Line, error.Column));
130 }
131
132 [Fact]
133 public static void TrailingSpacesOnADeepLineAreNotADeeperLineTest()
134 {
135     Assert.True(Accepted("a\n b\n c \n", 2));
136     Assert.True(Accepted("a\n b\n c\n ", 2));
137 }
138
139 [Fact]
140 public static void SyntaxErrorIsNotMistakenForNestingThatIsTooDeepTest()
141 {
142     var error = Assert.Throws<ParseException>(() => new Parser().Parse("a: b: c"));
143     Assert.Null(error.MaxDepth);
144     Assert.StartsWith("Syntax error at ", error.Message);
145 }
146
147 [Fact]
148 public static void RefusesADocumentFarPastTheLimitWithoutOverflowingTheStackTest()
149 {
150     // Before the limit existed each of these overflowed the stack, which ends the
151     // process. The documents are parsed on a 1 MiB stack, smaller than any thread
152     // .NET starts by default.
153     string?[] messages = new string?[3];
154     Exception? unexpected = null;
155     var thread = new Thread(() =>
156     {
157         try
158         {
159             var documents = new[] { Parens(100 000), Values(100 000), Indentation(2 000)
160             ↪ };
161             for (var index = 0; index < documents.Length; index++)
162             {
163                 var error = Assert.Throws<ParseException>(() => new
164                 ↪ Parser().Parse(documents[index]));
165                 Assert.Equal(Parser.DefaultMaxDepth, error.MaxDepth);
166                 messages[index] = error.Message;
167             }
168             catch (Exception error)
169             {
170                 unexpected = error;
171             }
172             }, 1 << 20);
173     thread.Start();
174     thread.Join();
175
176     Assert.Null(unexpected);
177     Assert.All(messages, message => Assert.StartsWith("Nesting too deep at ", message));
178 }
179
180 [Fact]
181 public static void StreamParserReportsWhereTheNestingIsTooDeepTest()
182 {
183     var stream = new StreamParser(new Parser(true, 1));
184     stream.Write("a\nb ((c))\n");
185
186     var error = Assert.Throws<StreamParseException>(() => stream.Finish());
187
188     Assert.Equal(1, error.ParseError.MaxDepth);
189     Assert.Equal((2, 4, 5), (error.Line, error.Column, error.Offset));
190 }
191 }
192 }
193 }

```

1.47 ./Link.Foundation.Links.Notation.Tests/ParseErrorPositionTests.cs

```

1 using System;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     /// <summary>
7     /// A parse error has to say where the document stopped making sense.
8     /// </summary>
9     /// <remarks>
10     /// The positions asserted here are the ones the JavaScript and Rust ports report for

```

```

11  /// the same input, so the implementations can be held to the same contract
12  /// (https://github.com/link-foundation/links-notation/issues/302).
13  /// </remarks>
14  public static class ParseErrorPositionTests
15  {
16      private static ParseException SyntaxError(string document) =>
17          Assert.Throws<ParseException>(() => new Parser().Parse(document));
18
19      [Fact]
20      public static void ReportsTheLineAndColumnOfTheDefectTest()
21      {
22          // The example from the issue: the defect is the second colon on line 2,
23          // and the two lines after it are fine.
24          var error = SyntaxError("ci gate x\nstage: rust: nextest\nnext stage\n clippy");
25
26          Assert.Equal(2, error.Line);
27          Assert.Equal(12, error.Column);
28          Assert.Equal(':', error.Found);
29      }
30
31      [Fact]
32      public static void OffsetAgreesWithTheOtherImplementationsTest()
33      {
34          // JavaScript and Rust report offset 21, line 2, column 12 for this document.
35          var error = SyntaxError("ci gate x\nstage: rust: nextest\n");
36
37          Assert.Equal(21, error.Offset);
38          Assert.Equal(2, error.Line);
39          Assert.Equal(12, error.Column);
40      }
41
42      [Fact]
43      public static void ReportsTheLineALateDefectIsOnTest()
44      {
45          var error = SyntaxError("a\nb\nc\nd\ne: f: g\nh\n");
46
47          Assert.Equal(5, error.Line);
48          Assert.Equal(5, error.Column);
49          Assert.Equal("e: f: g", error.LineText);
50      }
51
52      [Fact]
53      public static void ReportsTheEndOfTheDocumentWhenAGroupIsNeverClosedTest()
54      {
55          var error = SyntaxError("a (b\n");
56
57          Assert.Equal(5, error.Offset);
58          Assert.Equal(2, error.Line);
59          Assert.Equal(1, error.Column);
60          Assert.Null(error.Found);
61          Assert.Contains("end of input", error.Message);
62      }
63
64      [Fact]
65      public static void ReportsAnUnmatchedClosingParenthesisTest()
66      {
67          var error = SyntaxError("a b)\n");
68
69          Assert.Equal(3, error.Offset);
70          Assert.Equal(1, error.Line);
71          Assert.Equal(4, error.Column);
72          Assert.Equal(')', error.Found);
73      }
74
75      [Fact]
76      public static void MessagePointsACaretAtTheOffendingCharacterTest()
77      {
78          var error = SyntaxError("ci gate x\nstage: rust: nextest\n");
79
80          Assert.Equal("line 2, column 12: unexpected \":\"", error.Summary);
81          Assert.Equal("2 | stage: rust: nextest\n | ^", error.Snippet);
82          Assert.Equal(
83              "Syntax error at line 2, column 12: unexpected \":\"\n2 | stage: rust: nextest\n | ^",
84              error.Message);
85      }
86
87      [Fact]
88      public static void MessageQuotesOneLineRatherThanTheRestOfTheDocumentTest()

```

```

89     {
90         var document = "ci gate x\nstage: rust: nextest\n"
91             + string.Concat(System.Linq.Enumerable.Repeat("trailing line\n", 500));
92
93         var error = SyntaxError(document);
94
95         Assert.Contains("line 2, column 12", error.Message);
96         Assert.DoesNotContain("trailing line", error.Message);
97         Assert.True(error.Message.Length < 200, $"message is {error.Message.Length}
↪ characters");
98     }
99
100     [Fact]
101     public static void MessageOfALongLineStaysAMessageTest()
102     {
103         var document = new string('a', 400) + ": " + new string('b', 400) + ": c";
104
105         var error = SyntaxError(document);
106
107         Assert.Equal(1, error.Line);
108         Assert.Equal(803, error.Column);
109         Assert.Contains("...", error.Message);
110         Assert.True(error.Message.Length < 300, $"message is {error.Message.Length}
↪ characters");
111     }
112
113     [Fact]
114     public static void KeepsCatchingCodeThatExpectsAFormatExceptionWorkingTest()
115     {
116         // The generated parser used to raise a bare FormatException; code that catches
117         // it keeps working, and now reads a position off the exception it catches.
118         var error = Assert.Throws<ParseException>(() => new Parser().Parse("a: b: c"));
119
120         Assert.IsAssignableFrom<FormatException>(error);
121         Assert.Equal(4, error.Offset);
122     }
123
124     [Fact]
125     public static void DoesNotMentionTheGrammarInternalsTest()
126     {
127         var error = SyntaxError("a: b: c");
128
129         Assert.DoesNotContain("Failed to parse", error.Message);
130         Assert.DoesNotContain("document", error.Message);
131     }
132
133     [Fact]
134     public static void ADocumentThatParsesReportsNothingTest()
135     {
136         var links = new Parser().Parse("a: b\n c: d\n");
137
138         Assert.Equal(2, links.Count);
139     }
140 }
141 }

```

1.48 ./Link.Foundation.Links.Notation.Tests/ReferenceFidelityTests.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.IO;
4  using System.Linq;
5  using System.Text;
6  using Link.Foundation.Links.Notation.Binary;
7  using Xunit;
8
9  namespace Link.Foundation.Links.Notation.Tests;
10
11  public static class ReferenceFidelityTests
12  {
13      private static IEnumerable<string> Fixtures() =>
14          File.ReadLines(Path.Combine(AppContext.BaseDirectory, "Protocol",
↪ "reference-literals.txt"))
15              .Where(line => !line.StartsWith('#'))
16              .Select(line => line == "-" ? "" :
↪ Encoding.UTF8.GetString(Convert.FromHexString(line)));
17
18      [Fact]
19      public static void NativeReferenceFidelity()
20      {

```

```

21     foreach (var reference in Fixtures())
22     {
23         var original = new[] { new Link<string>(reference, new[] { new
↳ Link<string>(reference), new Link<string>("nested", new[] { new Link<string>(reference) })
↳ }) };
24         Assert.Equal(original, new Parser().Parse(original[0].ToString()));
25         Assert.Equal(original, new Parser().Parse(original.Format(true)));
26         Assert.Equal(original, new Parser().Parse(original.Format(new FormatConfig {
↳ MaxInlineRefs = 1, PreferInline = false })));
27     }
28 }
29
30 [Fact(Timeout = 30000)]
31 public static void ReferenceLiteralsAtEveryChunkSplit()
32 {
33     foreach (var reference in Fixtures())
34     {
35         Assert.Equal(reference, ReferenceLiteral.Decode(ReferenceLiteral.Encode(reference)));
36         foreach (var literal in new[] { Link<string>.EscapeReference(reference),
↳ LinoFormat.FormatReference(reference), ReferenceLiteral.Encode(reference) })
37         {
38             var text = $"(root: {literal})\n({literal}: fixture)";
39             var expected = new[] { new Link<string>("root", new[] { new
↳ Link<string>(reference) }), new Link<string>(reference, new[] { new Link<string>("fixture")
↳ }) };
40             Assert.Equal(expected, new Parser().Parse(text));
41             for (var split = 0; split <= text.Length; split++)
42             {
43                 TestContext.Current.CancellationTokens.ThrowIfCancellationRequested();
44                 var stream = new StreamParser();
45                 stream.Write(text[..split]);
46                 stream.Write(text[split..]);
47                 Assert.Equal(expected, stream.Finish());
48             }
49         }
50     }
51 }
52
53 [Fact]
54 public static void BinaryReferenceFidelity()
55 {
56     var codec = new BinaryLinoCodec();
57     foreach (var reference in Fixtures())
58     {
59         var original = new[] { new Link<string>(reference, new[] { new
↳ Link<string>(reference) }) };
60         var decoded = codec.Decode(codec.Encode(original));
61         Assert.Equal(original, decoded);
62         Assert.Equal(original, LinoFormat.ParseDocument(LinoFormat.FormatDocument(decoded)));
63     }
64 }
65
66 [Fact]
67 public static void MalformedReferenceLiteralsAreRejected()
68 {
69     foreach (var literal in File.ReadLines(Path.Combine(AppContext.BaseDirectory,
↳ "Protocol", "invalid-reference-literals.txt")).Where(line => !line.StartsWith('#')))
70     {
71         Assert.Throws<FormatException>(() => ReferenceLiteral.Decode(literal));
72         Assert.ThrowsAny<Exception>(() => new Parser().Parse(literal));
73     }
74     Assert.Equal("é", ReferenceLiteral.Decode("~1{C3A9}"));
75     Assert.Throws<EncoderFallbackException>(() => ReferenceLiteral.Encode("\ud800"));
76     Assert.Throws<EncoderFallbackException>(() => Link<string>.EscapeReference("\ud800"));
77 }
78 }

```

1.49 ./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs

```

1 using System;
2 using Xunit;
3
4 namespace Link.Foundation.Links.Notation.Tests
5 {
6     public static class SingleLineParserTests
7     {
8         [Fact]
9         public static void SingleLinkTest()
10        {

```

```

11     var source = @"(address: source target)";
12     var parser = new Parser();
13     var links = parser.Parse(source);
14     var target = links.Format();
15     Assert.Equal(source, target);
16 }
17
18 [Fact]
19 public static void TripletSingleLinkTest()
20 {
21     var source = @"(papa has car)";
22     var parser = new Parser();
23     var links = parser.Parse(source);
24     var target = links.Format();
25     Assert.Equal(source, target);
26 }
27
28 [Fact]
29 public static void BugTest1()
30 {
31     var source = @"(ignore conan-center-index repository)";
32     var links = new Parser().Parse(source);
33     var target = links.Format();
34     Assert.Equal(source, target);
35 }
36
37 [Fact]
38 public static void QuotedReferencesTest()
39 {
40     var source = @"(a: 'b' ""c"")";
41     var target = @"(a: b c)";
42     var parser = new Parser();
43     var links = parser.Parse(source);
44     var formattedLinks = links.Format();
45     Assert.Equal(target, formattedLinks);
46 }
47
48 [Fact]
49 public static void QuotedReferencesWithSpacesTest()
50 {
51     var source = @"('a a': 'b b' ""c c"")";
52     var target = @"('a a': 'b b' 'c c')";
53     var parser = new Parser();
54     var links = parser.Parse(source);
55     var formattedLinks = links.Format();
56     Assert.Equal(target, formattedLinks);
57 }
58
59 [Fact]
60 public static void ParseSimpleReferenceTest()
61 {
62     var source = "test";
63     var parser = new Parser();
64     var links = parser.Parse(source);
65     Assert.NotNull(links);
66     Assert.Single(links);
67     // Simple reference creates a singlet link with null Id and one value
68     Assert.Null(links[0].Id);
69     Assert.NotNull(links[0].Values);
70     Assert.Single(links[0].Values!);
71     Assert.Equal("test", links[0].Values?[0].Id);
72     Assert.Null(links[0].Values?[0].Values);
73 }
74
75 [Fact]
76 public static void ParseReferenceWithColonAndValuesTest()
77 {
78     var source = "parent: child1 child2";
79     var parser = new Parser();
80     var links = parser.Parse(source);
81     Assert.NotNull(links);
82     Assert.Single(links);
83     Assert.Equal("parent", links[0].Id);
84     Assert.NotNull(links[0].Values);
85     Assert.Equal(2, links[0].Values!.Count);
86     Assert.Equal("child1", links[0].Values![0].Id);
87     Assert.Equal("child2", links[0].Values![1].Id);
88 }

```

```

89 [Fact]
90 public static void ParseMultilineLinkTest()
91 {
92     var source = "(parent: child1 child2)";
93     var parser = new Parser();
94     var links = parser.Parse(source);
95     Assert.NotNull(links);
96     Assert.Single(links);
97     Assert.Equal("parent", links[0].Id);
98     Assert.NotNull(links[0].Values);
99     Assert.Equal(2, links[0].Values!.Count);
100 }
101
102 [Fact]
103 public static void ParseValuesOnlyTest()
104 {
105     var source = ": value1 value2";
106     var parser = new Parser();
107     // Standalone ':' is now forbidden and should throw an exception
108     Assert.Throws<ParseException>(() => parser.Parse(source));
109 }
110
111 [Fact]
112 public static void SingletLinkTest()
113 {
114     // Test singlet link
115     var input = "(singlet)";
116     var result = new Parser().Parse(input);
117     Assert.Single(result);
118     Assert.Null(result[0].Id);
119     Assert.NotNull(result[0].Values);
120     Assert.Single(result[0].Values!);
121     Assert.Equal("singlet", result[0].Values?[0].Id);
122     Assert.Null(result[0].Values?[0].Values);
123 }
124
125 [Fact]
126 public static void ValueLinkTest()
127 {
128     // Test value link
129     var input = "(value1 value2 value3)";
130     var result = new Parser().Parse(input);
131     Assert.NotEmpty(result);
132 }
133
134 [Fact]
135 public static void QuotedReferencesWithSpecialCharsTest()
136 {
137     // Test quoted references
138     var input = @"("id with spaces": "value with spaces")";
139     var result = new Parser().Parse(input);
140     Assert.NotEmpty(result);
141 }
142
143 [Fact]
144 public static void SingleQuotedReferencesTest()
145 {
146     // Test single-quoted references
147     var input = "('id': 'value')";
148     var result = new Parser().Parse(input);
149     Assert.NotEmpty(result);
150 }
151
152 [Fact]
153 public static void ParseQuotedReferencesValuesOnlyTest()
154 {
155     var source = "\"has space\" 'has:colon'";
156     var parser = new Parser();
157     var links = parser.Parse(source);
158     Assert.NotNull(links);
159     Assert.Single(links);
160     Assert.Null(links[0].Id);
161     Assert.NotNull(links[0].Values);
162     Assert.Equal(2, links[0].Values!.Count);
163     Assert.Equal("has space", links[0].Values![0].Id);
164     Assert.Equal("has:colon", links[0].Values![1].Id);
165     var formatted = links.Format();
166

```

```

167     Assert.Equal("( 'has space' 'has:colon')", formatted);
168 }
169
170 [Fact]
171 public static void NestedLinksTest()
172 {
173     // Test nested links
174     var input = "(outer: (inner: value))";
175     var result = new Parser().Parse(input);
176     Assert.NotEmpty(result);
177 }
178
179 [Fact]
180 public static void HyphenatedIdentifiersTest()
181 {
182     // Test support for hyphenated identifiers like in BugTest1
183     var source = @"(conan-center-index: repository info)";
184     var parser = new Parser();
185     var links = parser.Parse(source);
186     var target = links.Format();
187     Assert.Equal(source, target);
188 }
189
190 [Fact]
191 public static void MultipleWordsInQuotesTest()
192 {
193     var source = @("New York": city state);
194     var parser = new Parser();
195     var links = parser.Parse(source);
196     var target = links.Format();
197     // Should preserve quotes for multi-word references
198     Assert.Contains("New York", target);
199 }
200
201 [Fact]
202 public static void SpecialCharactersInQuotesTest()
203 {
204     var input = @("key:with:colons": "value(with)parenns");
205     var result = new Parser().Parse(input);
206     Assert.NotEmpty(result);
207
208     input = @('key with spaces': 'value: with special chars');
209     result = new Parser().Parse(input);
210     Assert.NotEmpty(result);
211 }
212
213 [Fact]
214 public static void DeeplyNestedTest()
215 {
216     var input = "(a: (b: (c: (d: (e: value))))))";
217     var result = new Parser().Parse(input);
218     Assert.NotEmpty(result);
219 }
220
221 [Fact]
222 public static void SingleLineWithIdTest()
223 {
224     // Test single-line link with id
225     var input = "id: value1 value2";
226     var result = new Parser().Parse(input);
227     Assert.NotEmpty(result);
228 }
229
230 [Fact]
231 public static void SingleLineWithoutIdTest()
232 {
233     // Test link without id (single-line) - now forbidden
234     var input = ": value1 value2";
235     Assert.Throws<ParseException>(() => new Parser().Parse(input));
236 }
237
238 [Fact]
239 public static void MultilineWithoutIdTest()
240 {
241     // Test link without id (multi-line) - now forbidden
242     var input = "( : value1 value2)";
243     Assert.Throws<ParseException>(() => new Parser().Parse(input));
244 }
245

```

```

246 [Fact]
247 public static void SimpleRefTest()
248 {
249     var input = "simple ref";
250     var result = new Parser().Parse(input);
251     Assert.NotEmpty(result);
252 }
253
254 [Fact]
255 public static void MultiLineLinkWithIdTest()
256 {
257     var input = "(id: value1 value2)";
258     var result = new Parser().Parse(input);
259     Assert.NotEmpty(result);
260 }
261
262 [Fact]
263 public static void LinkWithoutIdMultiLineTest()
264 {
265     var input = "(: value1 value2)";
266     Assert.Throws<ParseException>(() => new Parser().Parse(input));
267 }
268
269 [Fact]
270 public static void SimpleReferenceParserTest()
271 {
272     var input = "hello";
273     var result = new Parser().Parse(input);
274     Assert.Single(result);
275     Assert.Null(result[0].Id);
276     Assert.NotNull(result[0].Values);
277     Assert.Single(result[0].Values!);
278     Assert.Equal("hello", result[0].Values?[0].Id);
279 }
280
281 [Fact]
282 public static void QuotedReferenceParserTest()
283 {
284     var input = "\"hello world\"";
285     var result = new Parser().Parse(input);
286     Assert.Single(result);
287     Assert.Null(result[0].Id);
288     Assert.NotNull(result[0].Values);
289     Assert.Single(result[0].Values!);
290     Assert.Equal("hello world", result[0].Values?[0].Id);
291 }
292
293 [Fact]
294 public static void SingletLinkParserTest()
295 {
296     var input = "(singlet)";
297     var result = new Parser().Parse(input);
298     Assert.Single(result);
299     Assert.Null(result[0].Id);
300     Assert.NotNull(result[0].Values);
301     Assert.Single(result[0].Values!);
302     Assert.Equal("singlet", result[0].Values?[0].Id);
303     Assert.Null(result[0].Values?[0].Values);
304 }
305
306 [Fact]
307 public static void ValueLinkParserTest()
308 {
309     var input = "(a b c)";
310     var result = new Parser().Parse(input);
311     Assert.Single(result);
312     Assert.Null(result[0].Id);
313     Assert.Equal(3, result[0].Values?.Count);
314 }
315
316 [Fact]
317 public static void ParseQuotedReferencesTest()
318 {
319     var input = "(\"quoted id\": \"value with spaces\")";
320     var parser = new Parser();
321     var result = parser.Parse(input);
322
323     Assert.NotEmpty(result);

```

```

324     var formatted = result.Format();
325     Assert.Contains("quoted id", formatted);
326     Assert.Contains("value with spaces", formatted);
327 }
328
329 [Fact]
330 public static void LinkWithoutIdSingleLineTest()
331 {
332     // Test that standalone ':' is forbidden and should throw
333     var input = ": value1 value2";
334     var parser = new Parser();
335
336     // C# parser forbids this syntax (like JS/Rust)
337     Assert.Throws<ParseException>(() => parser.Parse(input));
338 }
339
340 [Fact]
341 public static void SingleLineLinkWithIdTest()
342 {
343     var input = "id: value1 value2";
344     var parser = new Parser();
345     var result = parser.Parse(input);
346
347     Assert.NotEmpty(result);
348     Assert.NotNull(result[0].Id);
349     Assert.Equal("id", result[0].Id);
350 }
351
352 [Fact]
353 public static void LinkWithIdTest()
354 {
355     var input = "(id: a b c)";
356     var parser = new Parser();
357     var result = parser.Parse(input);
358
359     Assert.Single(result);
360     Assert.Equal("id", result[0].Id);
361     Assert.Equal(3, result[0].Values?.Count);
362 }
363
364 [Fact]
365 public static void QuotedReferenceTest()
366 {
367     var input = "\"quoted value\"";
368     var parser = new Parser();
369     var result = parser.Parse(input);
370
371     Assert.NotEmpty(result);
372     Assert.NotNull(result[0].Values);
373     Assert.Single(result[0].Values!);
374     Assert.Equal("quoted value", result[0].Values![0].Id);
375 }
376
377 [Fact]
378 public static void SimpleReferenceTest()
379 {
380     var input = "simplereference";
381     var parser = new Parser();
382     var result = parser.Parse(input);
383
384     Assert.NotEmpty(result);
385 }
386
387 [Fact]
388 public static void SingleLineLinkTest()
389 {
390     var input = "myid: value1 value2";
391     var parser = new Parser();
392     var result = parser.Parse(input);
393
394     Assert.Single(result);
395     Assert.Equal("myid", result[0].Id);
396     Assert.Equal(2, result[0].Values?.Count);
397 }
398
399 [Fact]
400 public static void ValuesOnlyInParenthesesTest()
401 {
402     var input = "(value1 value2)";

```

```

403     var parser = new Parser();
404     var result = parser.Parse(input);
405
406     Assert.Single(result);
407     Assert.Null(result[0].Id);
408     Assert.Equal(2, result[0].Values?.Count);
409 }
410
411 [Fact]
412 public static void ParseValuesOnlyStandaloneColonTest()
413 {
414     // Test that standalone ':' is forbidden and should throw
415     var input = ": value1 value2";
416     var parser = new Parser();
417
418     // C# parser forbids this syntax (like JS/Rust)
419     Assert.Throws<ParseException>(() => parser.Parse(input));
420 }
421
422 [Fact]
423 public static void QuotedReferencesWithSpacesInLinkTest()
424 {
425     var input = "(id: \"value with spaces\")";
426     var parser = new Parser();
427     var result = parser.Parse(input);
428
429     Assert.Single(result);
430     Assert.Equal("id", result[0].Id);
431     Assert.Single(result[0].Values!);
432     Assert.Equal("value with spaces", result[0].Values?[0].Id);
433 }
434 }
435 }

```

1.50 ./Link.Foundation.Links.Notation.Tests/StreamParserTests.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Threading.Tasks;
5  using Xunit;
6
7  namespace Link.Foundation.Links.Notation.Tests
8  {
9      public static class StreamParserTests
10     {
11         private const string Document =
12             "first loves data\n" +
13             "# streamed comment\n" +
14             "profile:\n" +
15             "    name Ada\n" +
16             "    note \"line one\nline two\"\n" +
17             "(nested:\n    child value)\n" +
18             "last sees first";
19
20         private static string[] Render(IEnumerable<Link<string>> links) =>
21             links.Select(link => link.ToString()).ToArray();
22
23         [Fact]
24         public static void MatchesCanonicalParserOneSymbolAtATime()
25         {
26             var stream = new StreamParser();
27             foreach (var character in Document)
28             {
29                 stream.Write(character.ToString());
30             }
31
32             Assert.Equal(Render(new Parser().Parse(Document)), Render(stream.Finish()));
33         }
34
35         [Fact]
36         public static void EmitsOnlyCompleteRecords()
37         {
38             var seen = new List<Link<string>>();
39             var stream = new StreamParser();
40             stream.LinkParsed += seen.Add;
41
42             Assert.Empty(stream.Write("profile:\n    name Ada\n    note \"first\nsecond\"\n"));
43             var emitted = stream.Write("n");
44
45             Assert.Single(emitted);

```

```

46     Assert.Equal(emitted, seen);
47 }
48
49 [Fact]
50 public static void SupportsLineChunksFinalRecordAndPosition()
51 {
52     var stream = new StreamParser();
53     foreach (var line in Document.SplitAfter('\n'))
54     {
55         stream.Write(line);
56     }
57
58     Assert.Equal(Render(new Parser().Parse(Document)), Render(stream.Finish()));
59     Assert.Equal(Document.Length, stream.Position.Offset);
60     Assert.Equal(0, stream.Position.Buffered);
61 }
62
63 [Fact]
64 public static void SupportsDrainResetAndBoundedMemory()
65 {
66     var seen = new List<Link<string>>();
67     var stream = new StreamParser { Collect = false, MaxBufferSize = 8 };
68     stream.LinkParsed += seen.Add;
69     for (var index = 0; index < 100; index++) stream.Write("a\n");
70
71     Assert.Empty(stream.Drain());
72     Assert.Single(stream.Finish());
73     Assert.Equal(100, seen.Count);
74
75     stream.Reset();
76     Assert.Throws<ArgumentException>(() => { stream.Write("123456789"); });
77 }
78
79 [Fact]
80 public static void ProvidesLazyAdapters()
81 {
82     var actual = StreamParser.ParseChunks(new[] { "one link\n", "two link" });
83
84     Assert.Equal(new[] { "(one link)", "(two link)" }, Render(actual));
85 }
86
87 [Fact]
88 public static void RejectsWritesAfterFinish()
89 {
90     var stream = new StreamParser();
91     stream.Finish("one");
92
93     Assert.Throws<InvalidOperationException>(() => stream.Write("two"));
94 }
95
96 [Fact]
97 public static async Task ProvidesAnAsyncEnumerableAdapter()
98 {
99     static async IEnumerable<string> Chunks()
100     {
101         yield return "one link\n";
102         await Task.Yield();
103         yield return "two link";
104     }
105
106     var actual = new List<Link<string>>();
107     await foreach (var link in StreamParser.ParseChunksAsync(
108         Chunks(),
109         cancellationTokens: TestContext.Current.CancellationTokens)) actual.Add(link);
110
111     Assert.Equal(new[] { "(one link)", "(two link)" }, Render(actual));
112 }
113 }
114
115 internal static class StreamingTestStringExtensions
116 {
117     public static IEnumerable<string> SplitAfter(this string value, char delimiter)
118     {
119         var start = 0;
120         for (var index = 0; index < value.Length; index++)
121         {
122             if (value[index] != delimiter) continue;
123             yield return value.Substring(start, index - start + 1);
124         }
125     }
126 }

```

```

125         start = index + 1;
126     }
127     if (start < value.Length) yield return value.Substring(start);
128 }
129 }

```

1.51 ./Link.Foundation.Links.Notation.Tests/TupleTests.cs

```

1  using System.Collections.Generic;
2  using Xunit;
3  using LinkType = Link.Foundation.Links.Notation.Link<string>;
4  using id = Link.Foundation.Links.Notation.id<string>;
5  using      = Link.Foundation.Links.Notation. <string>;
6
7  namespace Link.Foundation.Links.Notation.Tests
8  {
9      public class TupleTests
10     {
11         [Fact]
12         public void TupleToLinkTest()
13         {
14             var source = @"(papa (lovesMama: loves mama))
15 (son lovesMama)
16 (daughter lovesMama)
17 (all (love mama))";
18             var parser = new Parser();
19             var links = parser.Parse(source);
20             var targetFromString = links.Format();
21
22             IList<LinkType> constructedLinks = new List<LinkType>()
23             {
24                 ("papa", ( )("lovesMama", "loves", "mama")),
25                 ("son", "lovesMama"),
26                 ("daughter", "lovesMama"),
27                 ("all", ("love", "mama")),
28             };
29             var targetFromTuples = constructedLinks.Format();
30             Assert.Equal(targetFromString, targetFromTuples);
31         }
32
33         [Fact]
34         public void NamedTupleToLinkTest()
35         {
36             var source = @"(papa (lovesMama: loves mama))
37 (son lovesMama)
38 (daughter lovesMama)
39 (all (love mama))";
40             var parser = new Parser();
41             var links = parser.Parse(source);
42             var targetFromString = links.Format();
43
44             IList<LinkType> constructedLinks = new List<LinkType>()
45             {
46                 ("papa", ((id)"lovesMama", "loves", "mama")),
47                 ("son", "lovesMama"),
48                 ("daughter", "lovesMama"),
49                 ("all", ("love", "mama")),
50             };
51             var targetFromTuples = constructedLinks.Format();
52             Assert.Equal(targetFromString, targetFromTuples);
53         }
54     }
55 }

```

Index

./Link.Foundation.Links.Notation.Tests/ApiTests.cs, 52
./Link.Foundation.Links.Notation.Tests/BinaryLinksNotationTests.cs, 55
./Link.Foundation.Links.Notation.Tests/BinaryLinoCodecTests.cs, 61
./Link.Foundation.Links.Notation.Tests/CommentsTests.cs, 66
./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs, 68
./Link.Foundation.Links.Notation.Tests/EmptyReferenceTests.cs, 71
./Link.Foundation.Links.Notation.Tests/FormatConfigTests.cs, 72
./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs, 74
./Link.Foundation.Links.Notation.Tests/IndentedIdNestedValuesTests.cs, 76
./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs, 76
./Link.Foundation.Links.Notation.Tests/LinkTests.cs, 79
./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs, 80
./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs, 82
./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs, 84
./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs, 89
./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs, 91
./Link.Foundation.Links.Notation.Tests/NestedIndentationTests.cs, 92
./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs, 94
./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs, 97
./Link.Foundation.Links.Notation.Tests/NestingDepthTests.cs, 98
./Link.Foundation.Links.Notation.Tests/NestingLimitTests.cs, 100
./Link.Foundation.Links.Notation.Tests/ParseErrorPositionTests.cs, 102
./Link.Foundation.Links.Notation.Tests/ReferenceFidelityTests.cs, 104
./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs, 105
./Link.Foundation.Links.Notation.Tests/StreamParserTests.cs, 111
./Link.Foundation.Links.Notation.Tests/TupleTests.cs, 113
./Link.Foundation.Links.Notation/Binary/ArityRange.cs, 1
./Link.Foundation.Links.Notation/Binary/BinaryLinoCodec.cs, 3
./Link.Foundation.Links.Notation/Binary/BinaryNotationException.cs, 3
./Link.Foundation.Links.Notation/Binary/LinksPacket.cs, 4
./Link.Foundation.Links.Notation/Binary/LinoFormat.cs, 12
./Link.Foundation.Links.Notation/Binary/LinoMapping.cs, 14
./Link.Foundation.Links.Notation/Binary/PacketReader.cs, 22
./Link.Foundation.Links.Notation/Binary/SectionPlanner.cs, 23
./Link.Foundation.Links.Notation/Comments.cs, 25
./Link.Foundation.Links.Notation/FormatConfig.cs, 26
./Link.Foundation.Links.Notation/FormatOptions.cs, 26
./Link.Foundation.Links.Notation/FurthestFailureTracer.cs, 27
./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs, 28
./Link.Foundation.Links.Notation/IListExtensions.cs, 30
./Link.Foundation.Links.Notation/Link.cs, 32
./Link.Foundation.Links.Notation/LinkFormatExtensions.cs, 35
./Link.Foundation.Links.Notation/LinksGroup.cs, 37
./Link.Foundation.Links.Notation/ParseException.cs, 40
./Link.Foundation.Links.Notation/Parser.cs, 42
./Link.Foundation.Links.Notation/Quotes.cs, 44
./Link.Foundation.Links.Notation/ReferenceLiteral.cs, 46
./Link.Foundation.Links.Notation/StreamParseException.cs, 47
./Link.Foundation.Links.Notation/StreamParser.cs, 47
./Link.Foundation.Links.Notation/.cs, 51
./Link.Foundation.Links.Notation/id.cs, 52